



SAPIENZA
UNIVERSITÀ DI ROMA

Selezione Dinamica della Dimensione del Campione in Metodi di Ottimizzazione per il Machine Learning

Algoritmi adattivi a campione dinamico e
sottocampionamento dell'Hessiana
per problemi su larga scala

Università degli Studi La Sapienza, Roma
Corso di Laurea in Scienze Matematiche per l'Intelligenza Artificiale

Alessandro Lo Curcio • 20 agosto 2026

Ringraziamenti

Ringrazio chiunque, in qualsiasi modo, mi abbia motivato a fare di più.

Abstract

Questa tesi presenta, in forma rigorosa ma accessibile, la metodologia per la selezione dinamica della dimensione del campione negli algoritmi di ottimizzazione per il machine learning su larga scala, basandosi sul lavoro seminale di Byrd, Chin, Nocedal e Wu [1]. L'idea centrale è la seguente: anziché scegliere a priori una dimensione fissa del batch (il sottoinsieme di dati usato ad ogni iterazione), è possibile decidere durante l'algoritmo se e quando aumentarla, guidandosi da stime statistiche della varianza del gradiente stocastico. I contributi di questo lavoro sono un'analisi teorica completa dei metodi a campione dinamico (convergenza lineare deterministica e in aspettativa, stima della complessità totale), il miglioramento di alcuni risultati di convergenza mediante disuguaglianze più strette, e un'implementazione Python dei tre algoritmi (gradiente a campione dinamico, Newton-CG con Hessiana sottocampionata, estensione ai problemi con regolarizzazione L_1) con un'applicazione web interattiva per la sperimentazione. Il documento è organizzato in sette capitoli: dopo l'introduzione e il background necessario, si formula il problema di ottimizzazione, si passano in rassegna i lavori correlati, si descrivono gli algoritmi studiati, se ne discute l'implementazione interattiva con i risultati sperimentali e si traggono le conclusioni con le prospettive di lavoro futuro. I dettagli dimostrativi e i codici Python completi sono raccolti in appendice.

Indice

1	Introduzione	5
1.1	Contesto	5
1.2	Il problema: la dimensione del mini batch	5
1.3	La soluzione proposta: il campionamento dinamico	5
1.4	Contributi del lavoro	5
1.5	Struttura della tesi	6
2	Background e Notazione	6
2.1	Glossario dei simboli ricorrenti	6
2.2	Collegamento con la formulazione del problema	7
3	Formulazione del Problema	8
3.1	Apprendimento supervisionato su larga scala	8
3.2	La sfida computazionale	8
3.3	Il trade off	9
3.4	Formulazione del problema	9
3.5	Ipotesi su J	9
4	Lavori Correlati	11
4.1	Metodi di ottimizzazione stocastica	11
4.2	Campionamento dinamico	11
4.3	Metodi di secondo ordine stocastici	12
4.4	Regolarizzazione L_1 in ottimizzazione	12
5	Algoritmi Proposti	13
5.1	Metodo del Gradiente a Campione Dinamico	13
5.1.1	Formulazione della dinamica del batch	14
5.1.2	Formulazione matematica della condizione di accettazione	14
5.1.3	Regola di aggiornamento del batch	17
5.1.4	Pseudocodice (Algoritmo)	18
5.1.5	Convergenza deterministica	19
5.1.6	Analisi Stocastica e Complessità del Metodo	23
5.2	Metodo di Newton-CG con Campionamento Dinamico	31
5.2.1	Struttura del metodo	31
5.2.2	Criterio di terminazione del gradiente coniugato	32
5.2.3	Pseudocodice (Algoritmo Newton-CG)	35
5.3	Metodo Newton-CG per Modelli con Regolarizzazione L_1	39
5.3.1	Formulazione del Problema	39
5.3.2	Il gradiente generalizzato	39
5.3.3	Identificazione dell'Active Set e della Faccia Ortante	41
5.3.4	Fase di Minimizzazione nel Sottospazio	42
5.3.5	Ricerca Lineare Proiettata	42
5.3.6	Algoritmo Completo	43

6	Esperimenti e Visualizzazione Interattiva	45
6.1	Setup Sperimentale	45
6.2	Architettura Software	46
6.3	Risultati Numerici	47
6.4	Visualizzazione Interattiva	51
6.5	Validazione su un benchmark musicale: riconoscimento della nota con NSynth	51
6.6	Estensione oltre le ipotesi: implementazione vettorizzata per la rete neurale	58
7	Conclusioni e Lavoro Futuro	65
7.1	Riassunto dei risultati	65
7.2	Limiti dello studio	66
7.3	Direzioni per lavoro futuro	66
A	Dimostrazioni	67
A.1	Dimostrazione delle disuguaglianze di Polyak–Łojasiewicz	67
A.2	Dimostrazione del fattore di correzione per popolazione finita	70
A.3	Dimostrazione delle formule per α_k e β_k nel Metodo del Gradiente Coniugato	72
A.4	Spiegazione delle condizioni sul passo α_k	75
B	Codici Python completi	75
B.1	Metodo del gradiente a campione dinamico	76
B.2	Newton-CG con campionamento dinamico	79
B.3	Newton-CG per problemi L_1 -regolarizzati	83
B.4	Metodo BB-CCV (Barzilai–Borwein con campionamento dinamico)	88
C	Istruzioni per l'esecuzione	90
C.1	Dipendenze	91
C.2	Installazione	91
C.3	Esecuzione degli esperimenti numerici	91
C.4	Esecuzione dei singoli algoritmi	91
C.5	Avvio dell'applicazione web	92
C.6	Requisiti di sistema	92
D	Schemi dei metodi a varianza ridotta	92
E	Schema dell'algoritmo BB-CCV	93
E.1	Il metodo di Barzilai–Borwein	93
E.2	Schema dell'algoritmo BB-CCV	95

1 Introduzione

1.1 Contesto

Nel machine learning su larga scala si dispone di un dataset di N esempi (spesso dell'ordine di 10^6 – 10^9) e si cerca il vettore di parametri w che minimizza la perdita media $J(w)$. Il calcolo del gradiente completo $\nabla J(w)$ richiede di valutare la perdita su tutti gli esempi, con costo $O(Nm)$ ad iterazione, proibitivo se ripetuto su larga scala.

Per questo motivo gli algoritmi di ottimizzazione stocastica, che usano ad ogni passo soltanto un sottoinsieme dei dati (batch), sono diventati lo standard: la letteratura offre un ampio spettro di metodi, dal gradiente stocastico classico (SGD) [6, 2] ai metodi a varianza ridotta [10, 9].

1.2 Il problema: la dimensione del mini batch

La scelta della dimensione del mini batch è un compromesso fondamentale. Un batch piccolo rende ogni iterazione economica ma il gradiente stimato è rumoroso, e il rumore limita la precisione raggiungibile. Un batch grande fornisce una stima più accurata, ma il costo per iterazione cresce proporzionalmente alla sua dimensione. Nei metodi classici la dimensione del batch è fissa e scelta euristicamente all'inizio dell'ottimizzazione, senza alcun adattamento alla difficoltà del problema: se il batch è troppo piccolo si rischia di non convergere alla precisione desiderata; se è troppo grande si spreca risorse computazionali nelle prime iterazioni, dove una stima grossolana sarebbe più che sufficiente.

1.3 La soluzione proposta: il campionamento dinamico

La soluzione studiata in questa tesi consiste nel rendere dinamica la dimensione del batch: si parte da un campione piccolo per fare progressi rapidi ed economici e lo si aumenta progressivamente quando serve precisione. La decisione su quando e di quanto aumentare il batch è guidata da una condizione di controllo della varianza (CCV): finché la varianza campionaria del gradiente stocastico è piccola rispetto alla norma del gradiente corrente, il batch corrente è adeguato; quando il gradiente si avvicina a zero, il rumore relativo cresce e la CCV impone un batch più grande. In questo modo la risorsa computazionale viene concentrata dove è realmente necessaria, con un adattamento automatico alla difficoltà del problema.

Questa idea, introdotta da Byrd, Chin, Nocedal e Wu [1], è qui approfondita sotto tre aspetti: un'analisi teorica completa del metodo del gradiente a campione dinamico e della sua variante Newton-CG con Hessiana sottocampionata (convergenza lineare deterministica e in aspettativa, complessità totale); il raffinamento di alcuni risultati di convergenza mediante disuguaglianze più strette; e l'implementazione degli algoritmi in Python, accompagnata da un'applicazione web interattiva che ne permette la sperimentazione diretta.

1.4 Contributi del lavoro

I metodi qui analizzati – gradiente a campione dinamico, Newton-CG con Hessiana sottocampionata ed estensione ai problemi con regolarizzazione L_1 – sono stati

introdotti da Byrd, Chin, Nocedal e Wu [1]. I contributi di questa tesi riguardano l'analisi, il miglioramento di alcuni risultati di convergenza e l'implementazione:

1. Analisi teorica. Viene fornita una trattazione completa della dinamica del batch basata sul controllo della varianza: si dimostra la convergenza lineare del metodo del gradiente a campione dinamico, in forma deterministica e in aspettativa, e si ricava una stima della complessità totale competitiva con quella di SGD e migliore sui problemi mal condizionati.
2. Miglioramento dei teoremi di convergenza. Alcuni risultati vengono raffinati mediante l'uso di disuguaglianze più strette. In particolare, il fattore di contrazione del metodo del gradiente a campione dinamico passa da $1 - \frac{(1-\theta)\lambda}{2L}$ a $1 - \frac{(1-\theta)\lambda}{L}$ usando la disuguaglianza di Polyak–Łojasiewicz nella forma forte (Sezione 5.1.5).
3. Implementazioni. I tre algoritmi (gradiente a campione dinamico, Newton-CG e variante L_1) sono implementati in Python e resi disponibili in un'applicazione web interattiva, con esperimenti numerici (Capitolo 6) e codici completi (Appendice B).

1.5 Struttura della tesi

Il resto del lavoro è organizzato come segue. Il Capitolo 2 richiama la notazione usata nel resto del lavoro; il Capitolo 3 formalizza il problema dell'ottimizzazione empirica su larga scala e le ipotesi strutturali su J ; il Capitolo 4 passa in rassegna i lavori correlati; il Capitolo 5 descrive gli algoritmi proposti con i risultati di convergenza e complessità; il Capitolo 6 illustra l'implementazione e i risultati numerici; il Capitolo 7 riassume i risultati e delinea le direzioni di lavoro futuro. Le dimostrazioni, i codici Python, le istruzioni di esecuzione e gli schemi degli algoritmi aggiuntivi sono raccolti nelle Appendici A–E.

2 Background e Notazione

Questo capitolo richiama la notazione usata nel resto del lavoro.

2.1 Glossario dei simboli ricorrenti

Per facilitare la lettura, raccogliamo qui i simboli matematici che appariranno più frequentemente.

Simbolo	Significato
$w \in \mathbb{R}^m$	parametri del modello
N	numero di esempi del dataset
n_k	dimensione del batch all'iterazione k
$\mathcal{S}_k \subseteq \{1, \dots, N\}$	campione di indici all'iterazione k
$\ell(w; i)$	perdita sull'esempio i
$J(w)$	perdita media sul dataset
$J_{\mathcal{S}}(w)$	perdita media sul campione $\mathcal{S}$
λ, L	costanti di convessità forte e di Lipschitz del gradiente
$\kappa = L/\lambda$	sovrastima del numero di condizionamento (che, in norma 2 e per matrici simmetriche, è $\lambda_{\max}/\lambda_{\min}$; cfr. Sez. 3)
$\theta \in (0, 1)$	tolleranza sull'errore relativo del gradiente stimato
α_k	passo all'iterazione k
$\mathcal{V}$	varianza della popolazione del gradiente della loss, componente per componente
$\hat{\mathcal{V}}$	stima campionaria di $\mathcal{V}$ sul batch

Nota

Il significato di $\mathcal{V}$ (varianza della popolazione). Al passo k , con pesi correnti w_k , il vettore $\mathcal{V}$ misura quanto i singoli gradienti $\nabla \ell(w_k; i)$ differiscono dalla loro media $\nabla J(w_k)$:

$$\mathcal{V} := \frac{1}{N} \sum_{i=1}^N (\nabla \ell(w_k; i) - \nabla J(w_k))^2 \in \mathbb{R}^m,$$

dove il quadrato è inteso componente per componente. Se i gradienti individuali sono tra loro coerenti (bassa varianza), la media campionaria è un buon stimatore di $\nabla J(w_k)$; se invece sono molto dispersi, il batch deve essere grande perché la stima sia affidabile. Questa osservazione è il punto di partenza della Condizione di Controllo della Varianza (CCV) introdotta nella Sezione 5.

2.2 Collegamento con la formulazione del problema

La notazione introdotta in questo capitolo è usata nel prossimo per formalizzare il problema di ottimizzazione empirica su larga scala. In particolare, la varianza della popolazione $\mathcal{V}$ e la sua stima campionaria forniranno il criterio per decidere, dinamicamente, quanti dati usare a ogni iterazione.

3 Formulazione del Problema

3.1 Apprendimento supervisionato su larga scala

Nell'apprendimento supervisionato, si dispone di un insieme di esempi etichettati $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$, dove:

- $x_i \in \mathbb{R}^m$ è il vettore delle feature (caratteristiche) dell' i -esimo esempio,
- $y_i \in \mathcal{Y}$ è la corrispondente etichetta (valore da predire),
- $N \in \mathbb{N}$ è il numero totale di esempi, potenzialmente dell'ordine di 10^6 – 10^9 .

Si assume che le coppie (x_i, y_i) siano realizzazioni i.i.d. di una variabile casuale con distribuzione di probabilità congiunta P su $\mathcal{X} \times \mathcal{Y}$. L'obiettivo ideale è minimizzare il rischio ovvero l'attesa della loss sullo spazio $\mathcal{X} \times \mathcal{Y}$:

$$\min_{w \in \mathbb{R}^m} R(w) = \mathbb{E}_{(x,y) \sim P} \ell(f(w, x), y) = \int_{\mathcal{X} \times \mathcal{Y}} \ell(f(w; x), y) dP(x, y)$$

dove:

- $f(w; x) = w^T x$ è il predittore lineare parametrizzato da w ,
- $\ell : \mathbb{R} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}$ è la funzione di perdita convessa.

Poiché P è ignota, si minimizza invece il rischio empirico:

Definizione

Funzione Obiettivo Empirica.

$$\min_{w \in \mathbb{R}^m} J(w) = \min_{w \in \mathbb{R}^m} \frac{1}{N} \sum_{i=1}^N \ell(w; i), \quad (3.1)$$

dove abbiamo usato la notazione abbreviata $\ell(w; i) \triangleq \ell(f(w; x_i), y_i)$

- $w \in \mathbb{R}^m$: parametri del modello (variabile di ottimizzazione),
- N : dimensione del dataset,
- $\ell(w; i)$: perdita del modello w sull'esempio i -esimo.

3.2 La sfida computazionale

Il gradiente esatto di J richiede la somma su tutti gli N esempi:

$$\nabla J(w) = \frac{1}{N} \sum_{i=1}^N \nabla \ell(w; i).$$

Il costo di ogni $\nabla \ell(w; i)$ è $O(m)$, siccome deve essere valutato N volte, ogni valutazione del gradiente di J in un punto w costa $O(Nm)$ operazioni. Per $N = 10^8$ e $m = 10^4$ si tratta di 10^{12} operazioni per singola iterazione ovvero un costo che non è praticabile.

3.3 Il trade off

Il problema è che per minimizzare J dobbiamo calcolare $\nabla J(w)$ in tanti punti w , una volta per ogni iterazione, finché non troviamo il minimo; in particolare un singolo gradiente di J costa $O(Nm)$. Se N è enorme, una iterazione è già costosissima, e di iterazioni ne servono tante. Le strade possibili sono le seguenti: il metodo stocastico (SGD) usa un solo esempio per iterazione: il costo è basso, ma il gradiente è rumoroso, quindi servono tante iterazioni e la convergenza è lenta. Il batch completo usa tutti i dati: il gradiente è esatto, quindi servono poche iterazioni, ma ogni iterazione costa $O(Nm)$ ed è impraticabile. Il mini batch fisso è un compromesso: usa un numero intermedio di esempi, bilanciando costo e accuratezza, ma la dimensione del batch è arbitraria e fissa.

La soluzione proposta è il metodo del gradiente a campione dinamico: si parte con un batch piccolo per fare progressi veloci, e lo si aumenta gradualmente quando serve precisione, in questo modo il costo computazionale si concentra solo dove serve.

3.4 Formulazione del problema

Definizione

Perdita media sul campione $\mathcal{S}_k$: ad ogni iterazione k si seleziona un sottoinsieme $\mathcal{S}_k \subseteq \{1, \dots, N\}$ di dimensione $n_k = |\mathcal{S}_k|$ e si definisce la funzione

$$J_{\mathcal{S}_k}(w) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \ell(w; i).$$

- $\mathcal{S}_k$: campione casuale di indici all'iterazione k ,
- $n_k = |\mathcal{S}_k|$: dimensione del batch, variabile con k ,
- $J_{\mathcal{S}_k}(w)$: approssimazione stocastica dell'obiettivo reale $J(w)$.

L'algoritmo definisce una regola per aggiornare w_k e per scegliere n_{k+1} in funzione delle informazioni disponibili all'iterazione k .

3.5 Ipotesi su J

Per garantire le proprietà teoriche degli algoritmi, si assumono le seguenti condizioni.

Definizione

Convessità forte e liscezza: la funzione $J : \mathbb{R}^m \rightarrow \mathbb{R}$ è uniformemente convessa e ha gradiente Lipschitz continuo: esistono costanti $0 < \lambda \leq L$ tali che, per ogni $w, d \in \mathbb{R}^m$,

$$\lambda \|d\|_2^2 \leq d^T \nabla^2 J(w) d \leq L \|d\|_2^2.$$

- $\lambda > 0$: costante di convessità forte. Impone che $\nabla^2 J(w) \succeq \lambda I$ per ogni w , il che garantisce che J abbia un unico minimizzatore w^* .
- L : costante di Lipschitz del gradiente. Impone che $\|\nabla J(x) - \nabla J(y)\|_2 \leq L\|x - y\|_2$ per ogni x, y , il che equivale a $\nabla^2 J(w) \preceq LI$ per ogni w .

- $\kappa = L/\lambda$: numero di condizionamento (stima superiore). Per una data Hessiana $\nabla^2 J(w)$ (matrice simmetrica), il numero di condizionamento in norma 2 sarebbe $\lambda_{\max}(\nabla^2 J(w))/\lambda_{\min}(\nabla^2 J(w))$. Dato che gli autovallori sono ovunque compresi tra λ e L , si ha $\kappa \leq L/\lambda$, $\kappa = L/\lambda$ viene usato come stima superiore uniforme. Se $\kappa \gg 1$, il problema è mal condizionato e più difficile da ottimizzare.

Nota

In pratica, la convessità forte non è sempre verificata (ad esempio, la perdita logistica non è fortemente convessa). Tuttavia, è sufficiente aggiungere un termine di regolarizzazione $\frac{\gamma}{2} \|w\|^2$ alla funzione obiettivo J se è convessa, per indurre convessità forte con $\lambda = \gamma$.

Nota

Disuguaglianze fondamentali (si veda l'Appendice A.1 per la dimostrazione completa). Sia w_* l'unico minimo di J e, per semplificare le notazioni, poniamo $J(w_*) = 0$. Allora con le ipotesi strutturali esplicitate sopra valgono le seguenti disuguaglianze:

$$\|\nabla J(w)\|_2^2 \geq 2\lambda J(w), \quad J(w) \geq \frac{\lambda}{2L^2} \|\nabla J(w)\|_2^2.$$

4 Lavori Correlati

Questo capitolo passa in rassegna la letteratura pertinente, organizzata per temi: metodi di ottimizzazione stocastica, metodi con campionamento dinamico, metodi di secondo ordine stocastici e regolarizzazione L_1 .

4.1 Metodi di ottimizzazione stocastica

Il punto di partenza di tutti i metodi stocastici è il lavoro pionieristico di Robbins e Monro [6], che ha introdotto il metodo del gradiente stocastico (SGD): ad ogni iterazione si stima il gradiente su un singolo esempio (o un piccolo batch) e si aggiorna w con un passo decrescente. Il costo per iterazione è indipendente da N , ma la varianza dello stimatore limita la velocità di convergenza e la precisione raggiungibile, come analizzato in dettaglio da Bottou e Bousquet [2] e nella rassegna più recente di Bottou, Curtis e Nocedal [5], che hanno formalizzato il compromesso tra costo per iterazione e velocità di convergenza su larga scala.

Per ridurre la varianza dello stimatore sono state proposte diverse famiglie di metodi. I metodi a varianza ridotta combinano la stima stocastica con una correzione periodica basata su un gradiente (quasi) esatto: SVRG di Johnson e Zhang [9] mantiene una copia dei parametri su cui ricalcola un gradiente completo a intervalli regolari (Schema D.1 in Appendice D), mentre SAGA di Defazio et al. [10] mantiene una tavola dei gradienti individuali e corregge la stima a ogni iterazione (Schema D.2 in Appendice D). Questi metodi ottengono una convergenza lineare per problemi fortemente convessi, ma richiedono di memorizzare o ricalcolare quantità globali e la dimensione del batch rimane comunque fissata a priori.

Tutti questi metodi condividono una caratteristica: la dimensione del batch è fissa o comunque determinata da una regola prestabilita, non da una condizione di accuratezza verificata durante l'esecuzione. È proprio questo il punto che il campionamento dinamico intende superare.

4.2 Campionamento dinamico

L'idea di far crescere la dimensione del batch durante l'ottimizzazione non è nuova, ma è stata formalizzata in modo sistematico da Byrd, Chin, Nocedal e Wu [1], il lavoro di riferimento di questa tesi. Gli autori propongono di adattare la dimensione del campione alla varianza stimata del gradiente: il batch viene aumentato quando tale varianza è troppo grande rispetto alla norma del gradiente stesso, ottenendo così un compromesso controllato tra costo per iterazione e accuratezza della direzione di discesa.

Approcci correlati adottano regole di crescita della dimensione del batch basate su criteri diversi. Ad esempio, una strategia semplice e diffusa è la crescita geometrica $n_k = \lceil a^k \rceil$, che però richiede la conoscenza (o una stima) del numero di condizionamento κ . Alcuni metodi propongono invece di adattare la dimensione del batch in modo euristico in base all'andamento della funzione obiettivo o della perdita di validazione, senza una garanzia teorica. Il vantaggio dell'approccio basato sulla varianza è duplice: da un lato fornisce una giustificazione statistica alla regola di aggiornamento, dall'altro si adatta automaticamente alla difficoltà locale del problema, senza richiedere la stima di κ .

4.3 Metodi di secondo ordine stocastici

I metodi del primo ordine convergono lentamente su problemi mal condizionati, poiché il numero di condizionamento κ del problema entra nel tasso di convergenza. I metodi di Newton incorporano l'informazione di curvatura dell'Hessiana e raggiungono un tasso di convergenza locale quadratico (o superlineare), a costo di risolvere ad ogni iterazione un sistema lineare $H_k d = -g_k$; si veda il testo di Nocedal e Wright [3] per una trattazione completa.

Su larga scala la costruzione esplicita dell'Hessiana è impraticabile, e sono stati sviluppati metodi che ne usano soltanto approssimazioni. I metodi inexact Newton [11] risolvono il sistema di Newton in modo approssimato, controllando il residuo del risolutore interno tramite un parametro di forcing; il metodo del gradiente coniugato (CG) applicato al sistema di Newton, in forma Hessian free, permette di calcolare i prodotti Hessiana-vettore senza costruire la matrice [12, 3]. In ambito stocastico, Byrd et al. [4] hanno proposto un metodo che sottocampiona sia il gradiente sia l'Hessiana, mostrando che un numero di iterazioni del CG che cresce con la precisione desiderata mantiene la convergenza. Il metodo Newton-CG presentato in questa tesi si colloca in questa linea di ricerca, con due innovazioni rispetto ai lavori precedenti: la dimensione del campione per il gradiente è regolata dinamicamente dalla CCV, e il criterio di arresto del CG interno è adattivo, basato sulla varianza campionaria dei prodotti Hessiana-vettore, in modo da non sprecare iterazioni interne quando l'approssimazione dell'Hessiana è limitata dal sottocampionamento.

4.4 Regolarizzazione L_1 in ottimizzazione

La regolarizzazione L_1 consiste nell'aggiungere alla funzione obiettivo un termine di penalità proporzionale alla norma L_1 dei parametri,

$$\nu \|w\|_1 = \nu \sum_{i=1}^m |w_i|,$$

dove $\nu > 0$ è un coefficiente che ne controlla l'intensità. L'effetto è duplice: da un lato i coefficienti grandi vengono penalizzati, scoraggiando l'adattamento eccessivo ai dati; dall'altro – ed è la proprietà più importante – la soluzione risulta *sparsa*, cioè con molti coefficienti esattamente nulli. L'intuizione è la seguente: a differenza della norma L_2 , il cui costo marginale tende a zero per coefficienti piccoli, la norma L_1 “costa” sempre ν per ogni unità di coefficiente, anche vicino a zero. Conviene quindi azzerare le variabili il cui contributo alla perdita non supera tale costo, ed è per questo che la L_1 è uno strumento fondamentale per l'interpretabilità dei modelli ad alta dimensionalità (è il principio alla base del LASSO).

La non differenziabilità della norma L_1 nei punti in cui un coefficiente si annulla richiede però di adattare i metodi classici del primo ordine. Un approccio diffuso è il *soft thresholding*: ad ogni passo si applica alla soluzione l'operatore di soglia

$$S_\nu(w_i) = \text{sign}(w_i) \max\{|w_i| - \nu, 0\},$$

che riduce in modulo ogni coefficiente di ν e lo azzerava se scende sotto soglia (metodi proximali). In questa tesi si segue invece la strategia dell'*active set* (o delle facce ortanti): si identifica l'insieme delle variabili che all'ottimo saranno nulle e si minimizza la funzione obiettivo solo sulle variabili libere, dove la L_1 è differenziabile,

mantenendo le altre esattamente a zero tramite una ricerca lineare proiettata. È la strategia adottata dal metodo Newton-CG- L_1 descritto nel Capitolo 5.

5 Algoritmi Proposti

5.1 Metodo del Gradiente a Campione Dinamico

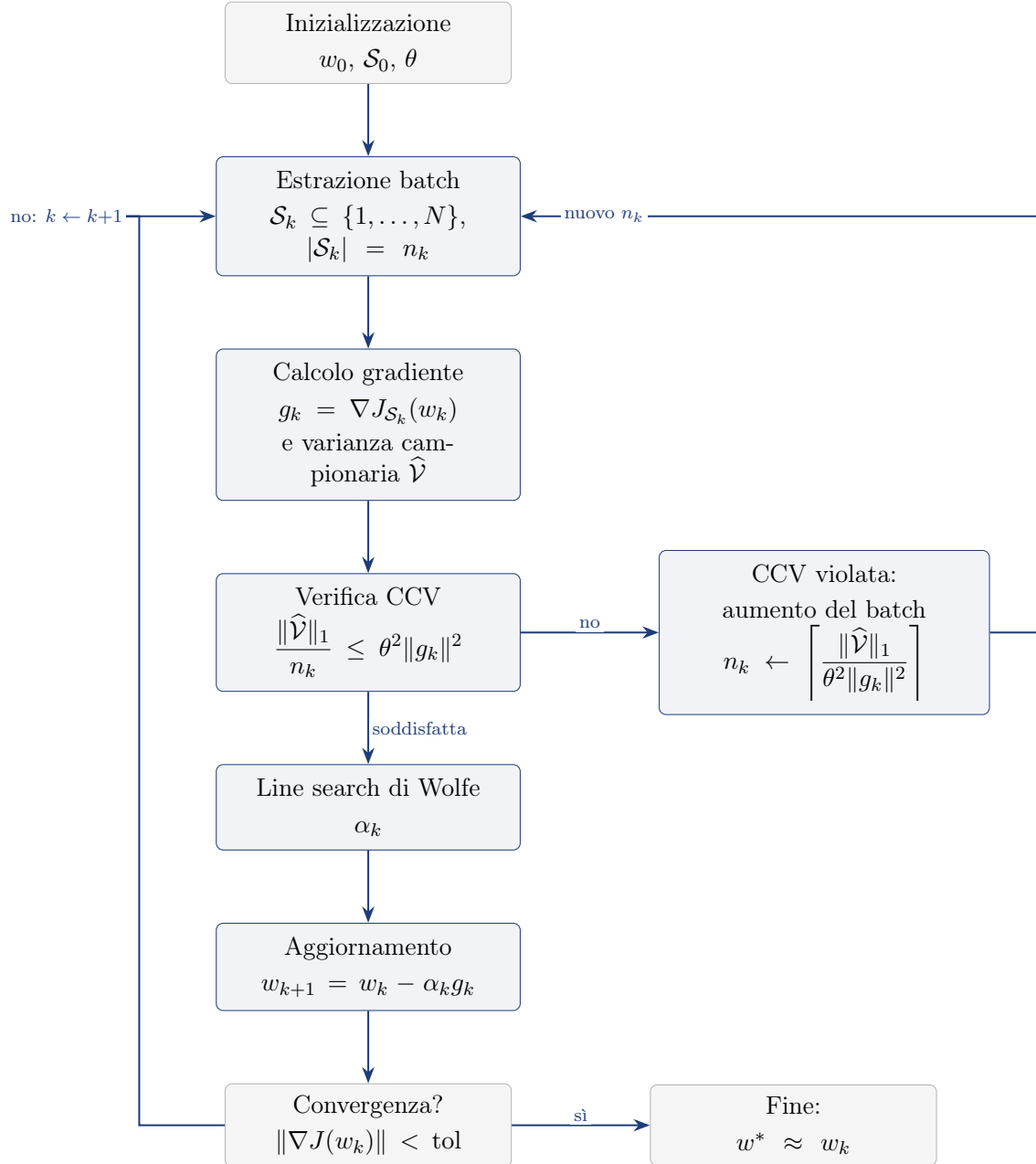


Figura 5.1: Schema dell'algoritmo con controllo della varianza campionaria (CCV): il flusso principale segue inizializzazione → estrazione batch → calcolo di gradiente e varianza → verifica della CCV → line search → aggiornamento; quando la CCV è violata la dimensione del batch viene aumentata e si ripete l'estrazione.

5.1.1 Formulazione della dinamica del batch

Il metodo del gradiente (o steepest descent) è l'algoritmo di ottimizzazione più semplice: a ogni passo, si scende nella direzione opposta al gradiente. La variante proposta non usa il gradiente esatto $\nabla J(w_k)$ (troppo costoso), ma un'approssimazione basata su un campione:

$$g_k = \nabla J_{\mathcal{S}_k}(w_k) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla \ell(w_k; i).$$

Vogliamo sapere quando il campione $\mathcal{S}_k$ è abbastanza grande da garantire che $-g_k$ sia davvero una direzione di discesa per J .

La direzione $-g_k$ è di discesa per J se l'errore del gradiente $\|g_k - \nabla J(w_k)\|$ è piccolo rispetto a $\|g_k\|$ (dimostrazione sotto). Questo errore non è calcolabile esattamente, ma può essere stimato dalla varianza campionaria di $\{\nabla \ell(w_k; i)\}_{i \in \mathcal{S}_k}$: se i gradienti dei singoli esempi nel batch sono coerenti tra loro (bassa varianza), allora la loro media è un buon estimatore di $\nabla J(w_k)$.

Nel metodo del gradiente classico, si richiede che la direzione di discesa d_k soddisfi $\nabla J(w_k)^T d_k < 0$. Qui, poiché g_k è la direzione di discesa, che approssima $\nabla J(w_k)$, abbiamo $\nabla J(w_k)^T (-g_k) = -\nabla J(w_k)^T g_k$. Se g_k è sufficientemente vicino a $\nabla J(w_k)$ in norma relativa, allora $-g_k$ è effettivamente una direzione di discesa.

5.1.2 Formulazione matematica della condizione di accettazione

La direzione $d_k = -g_k$ è di discesa per J se l'errore del gradiente stimato rispetto a quello reale è sufficientemente piccolo in norma relativa:

$$\|g_k - \nabla J(w_k)\|_2 \leq \theta \|g_k\|_2, \quad \theta \in [0, 1).$$

Per capire perché questo garantisce la discesa, poniamo $e_k = g_k - \nabla J(w_k)$, cioè $g_k = \nabla J(w_k) + e_k$. Allora

$$\nabla J(w_k)^T g_k = \nabla J(w_k)^T (\nabla J(w_k) + e_k) = \|\nabla J(w_k)\|^2 + \nabla J(w_k)^T e_k.$$

Poiché $\nabla J(w_k) = g_k - e_k$, si ha anche

$$\nabla J(w_k)^T g_k = (g_k - e_k)^T g_k = \|g_k\|^2 - e_k^T g_k.$$

Per ogni numero reale x vale $x \geq -|x|$; quindi $e_k^T g_k \geq -|e_k^T g_k|$. Usando Cauchy-Schwarz, $|e_k^T g_k| \leq \|e_k\| \|g_k\|$, da cui $e_k^T g_k \geq -\|e_k\| \|g_k\|$. Sostituendo,

$$\nabla J(w_k)^T g_k \geq \|g_k\|^2 - \|e_k\| \|g_k\| = \|g_k\| (\|g_k\| - \|e_k\|).$$

Affinché $\nabla J(w_k)^T g_k > 0$ (cioè $-g_k$ sia di discesa) è sufficiente che $\|g_k\| - \|e_k\| > 0$, ovvero $\|e_k\| < \|g_k\|$. La condizione $\|e_k\| \leq \theta \|g_k\|$ con $\theta < 1$ implica $\|e_k\| < \|g_k\|$, dunque garantisce la discesa.



Figura 5.2: Rappresentazione geometrica della condizione di discesa: se il gradiente stimato g_k giace nel cono di tolleranza attorno al gradiente esatto $\nabla J(w_k)$, con errore $\|e_k\| = \|g_k - \nabla J(w_k)\| \leq \theta \|g_k\|$, allora $-g_k$ è una direzione di discesa per J .

Dimostrazione: Per dimostrare il fatto che g_k debba essere in norma vicino al gradiente, possiamo semplicemente prendere le disuguaglianze del Lemma 1 (dimoststrate nella Sezione 5.1.5); poi, per trovare l'angolo, prendiamo la disuguaglianza che abbiamo ottenuto per il doppio prodotto scalare tra il gradiente di $J(w_k)$ e g_k durante la dimostrazione del Lemma 1. Da lì sostituiamo la definizione di prodotto scalare e isoliamo il coseno, ottenendo un quoziente con una somma di quadrati al numeratore. Da quella somma di quadrati possiamo indebolire il termine, minorando tale somma con $2AB$ (grazie a $A^2 + B^2 \geq 2AB$), e infine, dall'identità fondamentale $\sin^2 \varphi + \cos^2 \varphi = 1$, esprimiamo il seno in funzione del coseno, $\sin \varphi = \sqrt{1 - \cos^2 \varphi}$, per concludere $\sin \varphi \leq \theta$.

Poiché il gradiente della popolazione totale $\nabla J(w_k)$ è ignoto, non è possibile valutare questa condizione in modo deterministico a ogni iterazione. È importante distinguere due livelli: la condizione puntuale $\|g_k - \nabla J(w_k)\| \leq \theta \|g_k\|$ è quella usata nel Teorema deterministico per garantire la discesa a ogni passo; la Condizione di Controllo della Varianza (CCV) che introdurremo tra poco è invece una condizione in aspettativa, sufficiente per l'analisi di convergenza in media ma non una garanzia puntuale. Il mini batch $\mathcal{S}_k$ viene estratto casualmente dal dataset, rendendo g_k una variabile aleatoria con $\mathbb{E}_{\mathcal{S}_k}[g_k] = \nabla J(w_k)$. Passare dal controllo dell'errore puntuale $\|g_k - \nabla J(w_k)\|$ al controllo del suo valore atteso permette di aggirare l'ignoranza su $\nabla J(w_k)$:

$$\begin{aligned}
 \mathbb{E}_{\mathcal{S}_k} [\|g_k - \mathbb{E}_{\mathcal{S}_k}[g_k]\|_2^2] &= \mathbb{E}_{\mathcal{S}_k} \left[\sum_{j=1}^m \left(g_k^{(j)} - \mathbb{E}_{\mathcal{S}_k}[g_k^{(j)}] \right)^2 \right] \\
 &= \sum_{j=1}^m \mathbb{E}_{\mathcal{S}_k} \left[\left(g_k^{(j)} - \mathbb{E}_{\mathcal{S}_k}[g_k^{(j)}] \right)^2 \right] \\
 &= \sum_{j=1}^m \text{Var}_{\mathcal{S}_k} \left(g_k^{(j)} \right) = \|\text{Var}_{\mathcal{S}_k}(g_k)\|_1.
 \end{aligned}$$

Decomposizione della varianza dello stimatore. Definiamo la varianza della popolazione (vettore):

$$\begin{aligned}\mathcal{V} &:= \mathbb{E}_I [(\nabla \ell(w_k; I) - \mathbb{E}_I[\nabla \ell(w_k; I)])^2] \\ &= \text{Var}_I(\nabla \ell(w_k; I)) = \frac{1}{N} \sum_{i=1}^N (\nabla \ell(w_k; i) - \nabla J(w_k))^2 \in \mathbb{R}^m,\end{aligned}$$

dove il quadrato è inteso componente per componente, e distinguiamo I variabile aleatoria distribuita uniformemente in $\{1, \dots, N\}$ con i realizzazione di tale variabile.

Caso con reinserimento (indipendenza):

$$\begin{aligned}\text{Var}(g_k) &= \text{Var}\left(\frac{1}{n_k} \sum_{i \in S_k} \nabla \ell(w_k; i)\right) = \frac{1}{n_k^2} \sum_{i \in S_k} \text{Var}(\nabla \ell(w_k; i)) \\ &= \frac{1}{n_k^2} \underbrace{\text{Var}(\nabla \ell(w_k; i)) + \dots + \text{Var}(\nabla \ell(w_k; i))}_{n_k \text{ volte}} = \frac{\mathcal{V}}{n_k}.\end{aligned}$$

Anche qui la divisione va fatta componente per componente, siccome $\mathcal{V}$ è un vettore.

Caso senza reinserimento (dipendenza):

$$\text{Var}(\nabla J_{S_k}(w_k)) = \frac{\mathcal{V}}{n_k} \cdot \frac{N - n_k}{N - 1}.$$

La dimostrazione del fattore che si aggiunge qui è riportata nell'appendice. Per $N \gg n_k$ (condizione tipica nel Machine Learning), il fattore di correzione $\frac{N-n_k}{N-1}$ è circa 1, quindi $\text{Var}(g_k) \approx \mathcal{V}/n_k$: la condizione esatta si semplifica nella forma pratica riportata sotto.

Poiché $\mathcal{V}$ è ignoto, lo sostituiamo con la sua stima campionaria sul batch corrente:

$$\hat{\mathcal{V}} := \text{Var}_{i \in S_k}(\nabla \ell(w_k; i)) = \frac{1}{n_k - 1} \sum_{i \in S_k} (\nabla \ell(w_k; i) - \nabla J_{S_k}(w_k))^2.$$

Otteniamo così la Condizione di Controllo della Varianza (CCV):

Teorema

$$\frac{\|\hat{\mathcal{V}}\|_1}{n_k} \leq \theta^2 \|\nabla J_{S_k}(w_k)\|_2^2,$$

dove:

- $\|\cdot\|_1$ somma le varianze componente per componente,
- $\theta \in (0, 1)$ è la tolleranza impostata dall'utente,
- n_k è la dimensione del batch corrente.

Questa è la forma pratica per $N \gg n_k$ (corrispondente al limite $N \rightarrow \infty$ della condizione esatta che include il fattore $\frac{N-n_k}{N-1}$ derivata sopra). La condizione verifica che la varianza stimata dello stimatore sia sufficientemente piccola rispetto al gradiente corrente. Se non è soddisfatta, si aumenta n_k .

Nota

Dalla condizione di discesa (5.1.5) sappiamo che, per garantire che $-g_k$ sia una direzione di discesa, dobbiamo avere $\|e_k\|_2 \leq \theta \|g_k\|_2$, con $e_k := g_k - \nabla J(w_k)$. Elevando al quadrato:

$$\|e_k\|_2^2 \leq \theta^2 \|g_k\|_2^2.$$

Poiché $\nabla J(w_k)$ è ignoto, non possiamo calcolare $\|e_k\|_2^2$ esattamente. Tuttavia, poiché $\mathbb{E}[g_k] = \nabla J(w_k)$, si ha:

$$\mathbb{E} [\|e_k\|_2^2] = \mathbb{E} [\|g_k - \mathbb{E}[g_k]\|_2^2] = \|\text{Var}(g_k)\|_1.$$

Stimando $\|\text{Var}(g_k)\|_1 \approx \frac{\|\hat{\mathcal{V}}\|_1}{n_k}$, la disuguaglianza in valore atteso diventa esattamente la CCV:

$$\frac{\|\hat{\mathcal{V}}\|_1}{n_k} \leq \theta^2 \|g_k\|_2^2.$$

Il parametro θ controlla la tolleranza sull'errore relativo $\|e_k\|/\|g_k\|$; elevandolo al quadrato si preserva la coerenza dimensionale con la varianza (che è un errore quadratico medio).

5.1.3 Regola di aggiornamento del batch

Se la condizione CCV non è soddisfatta, si stima la nuova dimensione minima del batch. Supponendo che, per aumenti graduali, la varianza campionaria e la norma del gradiente non cambino significativamente (la stima della varianza per predire la dimensione del batch k viene fatta con la varianza del batch $k - 1$) si ottiene:¹

Definizione

Regola di Aggiornamento del Batch.

$$n_k^{\text{new}} = \left\lceil \frac{\|\hat{\mathcal{V}}\|_1}{\theta^2 \|\nabla J_{\mathcal{S}_k}(w_k)\|_2^2} \right\rceil$$

dove $\hat{\mathcal{V}} := \text{Var}_{i \in \mathcal{S}_k}(\nabla \ell(w_k; i))$ è la varianza campionaria.

Nota

Quando siamo lontani dal minimo, $\|\nabla J_{\mathcal{S}_k}(w_k)\|_2^2$ è grande: la formula di aggiornamento del batch dà un n piccolo, quindi il batch resta piccolo. Avvicinandoci al minimo, il gradiente si azzerà e il denominatore diventa piccolo: la formula impone un batch grande. Il batch cresce automaticamente dove serve precisione.

¹Nell'implementazione si aggiunge 1 per evitare che, quando il rapporto è esattamente intero, il batch non cresca, causando ripetute violazioni della CCV per fluttuazioni numeriche.



Figura 5.3: Andamento della dimensione del batch n_k in funzione dell'iterazione k per il metodo a campione dinamico (linea blu, con gradini in corrispondenza delle violazioni della CCV), confrontato con la funzione esponenziale a^k (curva oro). Il parametro $a > 1$ è stimato per minimi quadrati in scala logaritmica: minimizza $\sum_k (\ln n_k - k \ln a)^2$, da cui $\ln a = \frac{\sum_k k \ln n_k}{\sum_k k^2}$. La crescita di n_k segue quindi una legge quasi geometrica $n_k \approx a^k$, concentrando la risorsa computazionale nelle iterazioni finali, dove il gradiente è piccolo e serve precisione.

5.1.4 Pseudocodice (Algoritmo)

Algoritmo

Gradiente a Campione Dinamico

Input: w_0 (punto iniziale), $\mathcal{S}_0$ (batch iniziale), $\theta \in (0, 1)$ (tolleranza).

Ripeti per ogni $k \in \mathbb{N}$ fino a convergenza:

1. $g_k = \nabla J_{\mathcal{S}_k}(w_k)$.
2. $\hat{\mathcal{V}} = \text{Var}_{i \in \mathcal{S}_k}(\nabla \ell(w_k; i)) = \frac{1}{n_k - 1} \sum_{i \in \mathcal{S}_k} (\nabla \ell_i(w_k) - g_k)^2$.
3. Se $\frac{\|\hat{\mathcal{V}}\|_1}{n_k} > \theta^2 \|g_k\|_2^2$ aumenta il batch: $n_k \leftarrow \left\lceil \frac{\|\hat{\mathcal{V}}\|_1}{\theta^2 \|g_k\|_2^2} \right\rceil$.
4. Esegui una line search: trova $\alpha_k > 0$ tale che $J_{\mathcal{S}_k}(w_k - \alpha_k g_k) < J_{\mathcal{S}_k}(w_k)$.
5. Aggiorna: $w_{k+1} = w_k - \alpha_k g_k$.
6. Seleziona un nuovo campione $\mathcal{S}_{k+1}$ di dimensione n_k (la nuova dimensione, eventualmente aumentata).

5.1.5 Convergenza deterministica

Consideriamo l'iterazione a passo fisso

$$w_{k+1} = w_k - \alpha g_k, \quad \alpha = \frac{1 - \theta}{L},$$

dove g_k è un'approssimazione di $\nabla J(w_k)$ che soddisfa la condizione di accuratezza

$$\|g_k - \nabla J(w_k)\|_2 \leq \theta \|g_k\|_2, \quad \theta \in [0, 1).$$

Lemma 1. *Sotto la condizione di accuratezza si hanno le seguenti disuguaglianze:*

$$\|\nabla J(w_k)\| \leq (1 + \theta)\|g_k\|, \quad \|\nabla J(w_k)\| \geq (1 - \theta)\|g_k\|.$$

Inoltre,

$$\nabla J(w_k)^T g_k \geq (1 - \theta)\|g_k\|^2.$$

Dimostrazione. Poniamo $e_k = g_k - \nabla J(w_k)$. Allora per definizione $\nabla J(w_k) = g_k - e_k$. Prima dimostrazione: $\|\nabla J(w_k)\| \leq (1 + \theta)\|g_k\|$. Applicando la disuguaglianza triangolare $\|x + y\| \leq \|x\| + \|y\|$ con $x = g_k$ e $y = -e_k$:

$$\|\nabla J(w_k)\| = \|g_k + (-e_k)\| \leq \|g_k\| + \|-e_k\| = \|g_k\| + \|e_k\|.$$

Dalla condizione di accuratezza, $\|e_k\| \leq \theta\|g_k\|$. Sostituendo:

$$\|\nabla J(w_k)\| \leq \|g_k\| + \theta\|g_k\| = (1 + \theta)\|g_k\|.$$

Seconda dimostrazione: $\|\nabla J(w_k)\| \geq (1 - \theta)\|g_k\|$. Partiamo dalla disuguaglianza triangolare standard $\|a\| = \|a - b + b\| \leq \|a - b\| + \|b\|$, valida per ogni a, b . Ponendo $a = g_k$ e $b = e_k$:

$$\|g_k\| \leq \|g_k - e_k\| + \|e_k\| = \|\nabla J(w_k)\| + \|e_k\|.$$

Quindi $\|\nabla J(w_k)\| \geq \|g_k\| - \|e_k\|$. Sostituendo $\|e_k\| \leq \theta\|g_k\|$:

$$\|\nabla J(w_k)\| \geq \|g_k\| - \theta\|g_k\| = (1 - \theta)\|g_k\|.$$

Dimostrazione di $\nabla J(w_k)^T g_k \geq (1 - \theta)\|g_k\|^2$. Eleviamo al quadrato la condizione di accuratezza:

$$\|g_k - \nabla J(w_k)\|^2 \leq \theta^2 \|g_k\|^2.$$

Sviluppando il quadrato del binomio:

$$\|g_k\|^2 - 2\nabla J(w_k)^T g_k + \|\nabla J(w_k)\|^2 \leq \theta^2 \|g_k\|^2.$$

Isoliamo il termine con il prodotto scalare portando gli altri termini a destra:

$$-2\nabla J(w_k)^T g_k \leq \theta^2 \|g_k\|^2 - \|g_k\|^2 - \|\nabla J(w_k)\|^2 = (\theta^2 - 1)\|g_k\|^2 - \|\nabla J(w_k)\|^2.$$

Moltiplichiamo entrambi i membri per -1 :

$$2\nabla J(w_k)^T g_k \geq (1 - \theta^2)\|g_k\|^2 + \|\nabla J(w_k)\|^2.$$

Utilizziamo ora la disuguaglianza $\|\nabla J(w_k)\| \geq (1-\theta)\|g_k\|$ dimostrata in precedenza, elevandola al quadrato:

$$\|\nabla J(w_k)\|^2 \geq (1-\theta)^2\|g_k\|^2.$$

Sostituiamo questa stima nella disuguaglianza del Lemma:

$$2\nabla J(w_k)^T g_k \geq (1-\theta^2)\|g_k\|^2 + (1-\theta)^2\|g_k\|^2.$$

Raccogliamo $\|g_k\|^2$ e semplifichiamo i coefficienti:

$$\begin{aligned} 2\nabla J(w_k)^T g_k &\geq [(1-\theta^2) + (1-2\theta+\theta^2)]\|g_k\|^2 \\ &= [1-\theta^2+1-2\theta+\theta^2]\|g_k\|^2 = 2(1-\theta)\|g_k\|^2. \end{aligned}$$

Dividendo entrambi i membri per 2 si ottiene la tesi:

$$\nabla J(w_k)^T g_k \geq (1-\theta)\|g_k\|^2.$$

□

Teorema

Teorema di Convergenza Deterministica. Sia $J : \mathbb{R}^m \rightarrow \mathbb{R}$ una funzione λ -fortemente convessa ($\nabla^2 J(w) \succeq \lambda I$ per ogni w) e con gradiente L -Lipschitz ($\nabla^2 J(w) \preceq LI$ per ogni w). Si assuma che a ogni iterazione valga la condizione di accuratezza e si scelga il passo $\alpha = \frac{1-\theta}{L}$. Allora

$$\underbrace{J(w_{k+1}) - J(w^*)}_{=:\varepsilon_{k+1}} \leq \left(1 - \frac{(1-\theta)\lambda}{L}\right) \underbrace{(J(w_k) - J(w^*))}_{=:\varepsilon_k}.$$

Di conseguenza:

- Il fattore di riduzione per iterazione è $\rho = 1 - \frac{(1-\theta)\lambda}{L} \in (0, 1)$;
- Il numero di iterazioni per ottenere $J(w_k) - J(w^*) \leq \varepsilon$ è al più

$$k \geq \frac{L}{(1-\theta)\lambda} \log\left(\frac{J(w_0) - J(w^*)}{\varepsilon}\right) = O\left(\frac{L}{\lambda} \log \frac{1}{\varepsilon}\right).$$

Nota

Confronto con il risultato di Nocedal e Wright [3]. Il fattore di contrazione qui ottenuto è $1 - \frac{(1-\theta)\lambda}{L}$, mentre Nocedal e Wright [3] dimostrano $1 - \frac{(1-\theta)\lambda}{2L}$ (eq. 4.9). Ora $1 - \frac{(1-\theta)\lambda}{L} < 1 - \frac{(1-\theta)\lambda}{2L}$, cioè abbiamo un fattore di contrazione più piccolo (convergenza più rapida). Esso è dovuto all'uso della disuguaglianza di Polyak–Lojasiewicz nella forma forte $\|\nabla J(w)\|^2 \geq 2\lambda J(w)$ (dimostrata nell'appendice A.1), invece della forma più debole $\|\nabla J(w)\|^2 \geq \lambda(J(w) - J(w_*))$ (eq. 4.4–4.5) utilizzata da Nocedal e Wright [3]. La forma più forte è resa possibile dall'ipotesi di convessità forte ($\nabla^2 J \succeq \lambda I$), che implica direttamente $\|\nabla J(w)\|^2 \geq 2\lambda J(w)$ quando $J(w_*) = 0$.

Dimostrazione. Per semplificare poniamo $J(w^*) = 0$ (altrimenti si definisce $\tilde{J}(w) = J(w) - J(w^*)$, che soddisfa le stesse ipotesi di convessità e ha minimo nullo).

Per la L -lipschitzianità del gradiente, vale la seguente maggiorazione (conseguenza del teorema di Taylor con resto di Lagrange):

$$J(w_{k+1}) \leq J(w_k) + \nabla J(w_k)^T (w_{k+1} - w_k) + \frac{L}{2} \|w_{k+1} - w_k\|^2.$$

Sostituendo $\alpha = \frac{1-\theta}{L}$ e $w_{k+1} = w_k - \alpha g_k \Rightarrow w_{k+1} - w_k = -\alpha g_k$:

$$J(w_{k+1}) \leq J(w_k) - \frac{1-\theta}{L} \nabla J(w_k)^T g_k + \frac{L}{2} \left(\frac{1-\theta}{L}\right)^2 \|g_k\|^2.$$

Semplifichiamo il coefficiente del termine quadratico:

$$\frac{L}{2} \left(\frac{1-\theta}{L}\right)^2 = \frac{L}{2} \cdot \frac{(1-\theta)^2}{L^2} = \frac{(1-\theta)^2}{2L}.$$

Quindi:

$$J(w_{k+1}) \leq J(w_k) - \underbrace{\frac{1-\theta}{L} \nabla J(w_k)^T g_k + \frac{(1-\theta)^2}{2L} \|g_k\|^2}_{:=S}.$$

Moltiplichiamo la disuguaglianza del Lemma per $\frac{1-\theta}{2L}$:

$$\frac{1-\theta}{2L} \cdot 2 \nabla J(w_k)^T g_k \geq \frac{1-\theta}{2L} \left[(1-\theta^2) \|g_k\|^2 + \|\nabla J(w_k)\|^2 \right].$$

Semplificando il membro sinistro:

$$\frac{1-\theta}{L} \nabla J(w_k)^T g_k \geq \frac{(1-\theta)(1-\theta^2)}{2L} \|g_k\|^2 + \frac{1-\theta}{2L} \|\nabla J(w_k)\|^2.$$

Riorganizziamo la disuguaglianza appena scritta per isolare $-\frac{1-\theta}{L} \nabla J(w_k)^T g_k$, che compare nella definizione di S :

$$-\frac{1-\theta}{L} \nabla J(w_k)^T g_k \leq -\frac{(1-\theta)(1-\theta^2)}{2L} \|g_k\|^2 - \frac{1-\theta}{2L} \|\nabla J(w_k)\|^2.$$

Inseriamo questa disuguaglianza nella definizione di S :

$$J(w_{k+1}) \leq J(w_k) - \frac{(1-\theta)(1-\theta^2)}{2L} \|g_k\|^2 - \frac{1-\theta}{2L} \|\nabla J(w_k)\|^2 + \frac{(1-\theta)^2}{2L} \|g_k\|^2.$$

Raccogliamo i coefficienti di $\|g_k\|^2$:

$$-\frac{(1-\theta)(1-\theta^2)}{2L} + \frac{(1-\theta)^2}{2L} = -\frac{1-\theta}{2L} \left[(1-\theta^2) - (1-\theta) \right].$$

Calcoliamo la differenza tra parentesi:

$$(1-\theta^2) - (1-\theta) = 1-\theta^2-1+\theta = \theta-\theta^2 = \theta(1-\theta).$$

Quindi:

$$-\frac{1-\theta}{2L} \cdot \theta(1-\theta) = -\frac{\theta(1-\theta)^2}{2L}.$$

La disuguaglianza diventa:

$$J(w_{k+1}) \leq J(w_k) - \frac{\theta(1-\theta)^2}{2L} \|g_k\|^2 - \frac{1-\theta}{2L} \|\nabla J(w_k)\|^2.$$

Poiché $\frac{\theta(1-\theta)^2}{2L} \|g_k\|^2 \geq 0$, il termine $-\frac{\theta(1-\theta)^2}{2L} \|g_k\|^2$ è minore o uguale a zero, trascurarlo può solo aumentare il membro destro e la disuguaglianza rimane valida:

$$J(w_{k+1}) \leq J(w_k) - \frac{1-\theta}{2L} \|\nabla J(w_k)\|^2.$$

Infine, dalla disuguaglianza di convessità forte dimostrata nell'Appendice A.1,

$$\|\nabla J(w_k)\|^2 \geq 2\lambda J(w_k) \quad (\text{valida con } J(w_*) = 0),$$

si ottiene:

$$S \leq -\frac{1-\theta}{2L} \cdot 2\lambda J(w_k) = -\frac{\lambda(1-\theta)}{L} J(w_k).$$

Sostituendo nella disuguaglianza che definisce S :

$$J(w_{k+1}) \leq J(w_k) - \frac{\lambda(1-\theta)}{L} J(w_k) = \left(1 - \frac{\lambda(1-\theta)}{L}\right) J(w_k).$$

Applicando la disuguaglianza appena dimostrata ricorsivamente:

$$J(w_k) \leq \rho^k J(w_0), \quad \text{dove } \rho = 1 - \frac{\lambda(1-\theta)}{L}.$$

Poiché $\theta \in [0, 1)$, $\lambda > 0$ e $L > 0$, si ha $\rho \in (0, 1)$.

Imponiamo $J(w_k) \leq \varepsilon$:

$$\rho^k J(w_0) \leq \varepsilon \implies k \geq \frac{\log(J(w_0)/\varepsilon)}{\log(1/\rho)}.$$

Usando $\log(1/\rho) \geq 1 - \rho$:

$$1 - \rho = \frac{\lambda(1-\theta)}{L},$$

quindi:

$$k \geq \frac{L}{\lambda(1-\theta)} \log\left(\frac{J(w_0)}{\varepsilon}\right).$$

Questa è la stima del numero di iterazioni necessarie per ridurre l'errore al di sotto di ε . $\square$

5.1.6 Analisi Stocastica e Complessità del Metodo

Nell'analisi deterministica abbiamo assunto la condizione di accuratezza

$$\|g_k - \nabla J(w_k)\|_2 \leq \theta \|g_k\|_2, \quad \theta \in [0, 1), \quad (5.1)$$

garantendo che $-g_k$ sia una direzione di discesa per J e stabilendo la convergenza lineare (Teorema di Convergenza Deterministica).

Nel caso stocastico, $g_k = \nabla J_{\mathcal{S}_k}(w_k)$ è calcolato su un campione casuale $\mathcal{S}_k$ di dimensione n_k , rendendo g_k una variabile aleatoria. La condizione (5.1) non può essere garantita con probabilità 1 per ogni iterazione. Dobbiamo quindi sviluppare un'analisi in aspettativa, tenendo conto che n_k può variare nel corso delle iterazioni.

Impostazione del problema stocastico Definiamo

$$g_k := \nabla J_{\mathcal{S}_k}(w_k) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla \ell(w_k; i), \quad n_k := |\mathcal{S}_k|. \quad (5.2)$$

Per ogni w_k fissato, g_k è uno stimatore non distorto:

$$\mathbb{E}[g_k \mid w_k] = \nabla J(w_k). \quad (5.3)$$

Introduciamo l'ipotesi fondamentale di limitatezza della varianza dei gradienti individuali. Si noti che questa è un'ipotesi forte, necessaria per l'analisi di complessità teorica; in pratica può essere rilassata, ad esempio richiedendo la limitatezza solo in un intorno del minimo o utilizzando stime locali della varianza.

Nota

Ipotesi di limitatezza della varianza. $\exists \omega \in \mathbb{R}, \omega > 0$ tale che $\forall w_k$,

$$\|\text{Var}(\nabla \ell(w_k; I))\|_1 \leq \omega, \quad (5.4)$$

dove $\text{Var}(\nabla \ell(w_k; I)) := \mathbb{E}_I[(\nabla \ell(w_k; I) - \nabla J(w_k))^2]$ è inteso componente per componente.

Dal Lemma 1 abbiamo $\|\nabla J(w_k)\| \geq (1 - \theta)\|g_k\|$, da cui

$$\|g_k\|^2 \leq \frac{1}{(1 - \theta)^2} \|\nabla J(w_k)\|^2.$$

Dall'Appendice A.1 (disuguaglianza $J(w) \geq \frac{\lambda}{2L^2} \|\nabla J(w)\|^2$) otteniamo $\|\nabla J(w_k)\|^2 \leq \frac{2L^2}{\lambda} J(w_k)$. Combinando con la convergenza lineare del Teorema di Convergenza Deterministica, $J(w_k) \leq (1 - \frac{(1-\theta)\lambda}{L})^k J(w_0)$, segue

$$\|g_k\|^2 \leq \frac{1}{(1 - \theta)^2} \frac{2L^2}{\lambda} J(w_k) \leq \frac{1}{(1 - \theta)^2} \frac{2L^2 J(w_0)}{\lambda} \left(1 - \frac{(1 - \theta)\lambda}{L}\right)^k. \quad (5.5)$$

Va osservato che questo bound ha natura circolare: la convergenza lineare del Teorema di Convergenza Deterministica, usata nella (5.5), richiede a sua volta che la CCV sia soddisfatta a ogni iterazione, ed è proprio questo che si intende garantire con la scelta di n_k . La (5.5) va quindi intesa come una stima guida/euristica per la scelta di n_k , non come una derivazione rigorosa, in analogia con quanto fatto nel lavoro originale [1]. Questo bound adotta il tasso deterministico $\rho = 1 - \frac{(1-\theta)\lambda}{L}$ usato nel teorema precedente. Si noti che questo tasso è più favorevole (più rapido) rispetto a quello ottenuto da Byrd et al. [1], che presentano un fattore $1/2$ aggiuntivo a denominatore a causa dell'uso di una disuguaglianza più debole. Dalla Condizione di Controllo della Varianza (CCV):

$$\frac{\|\overbrace{\text{Var}_{i \in \mathcal{S}_k}(\nabla \ell(w_k; i))}^{\hat{v}}\|_1}{n_k} \leq \theta^2 \|g_k\|^2. \quad (5.6)$$

Per garantire la (5.6) in media, dobbiamo scegliere n_k sufficientemente grande. Usando l'ipotesi (5.4), la condizione (5.6) è sicuramente soddisfatta se

$$\frac{\omega}{n_k} \leq \theta^2 \|g_k\|^2,$$

ovvero se

$$n_k \geq \frac{\omega}{\theta^2 \|g_k\|^2}. \quad (5.7)$$

Osserviamo che la (5.5) fornisce un bound superiore su $\|g_k\|^2$ che decresce geometricamente con tasso $\rho = 1 - (1 - \theta)\lambda/L$. Poiché nella (5.7) la dimensione del batch è inversamente proporzionale a $\|g_k\|^2$, al restringersi del gradiente n_k deve crescere per evitare che il rumore domini la discesa. Per stimare la velocità di crescita necessaria, usiamo il bound (5.5) per valutare il tasso massimo con cui il denominatore di (5.7)

può restringersi: sostituendo la (5.5) nella (5.7) si ottiene una condizione guida sulla crescita di n_k ,

$$n_k \geq \frac{\omega}{\theta^2 \cdot \frac{1}{(1-\theta)^2} \frac{2L^2 J(w_0)}{\lambda} \left(1 - \frac{(1-\theta)\lambda}{L}\right)^k} = \frac{\omega(1-\theta)^2 \lambda}{\theta^2 2L^2 J(w_0)} \left(1 - \frac{(1-\theta)\lambda}{L}\right)^{-k}. \quad (5.8)$$

Pertanto, per far decadere il rumore almeno alla stessa velocità con cui l'errore deterministico si riduce, n_k deve crescere geometricamente come

$$n_k \geq O\left(\left(1 - \frac{(1-\theta)\lambda}{L}\right)^{-k}\right).$$

Nell'analisi stocastica imponiamo quindi:

$$n_k = \lceil a^k \rceil, \quad a > 1. \quad (5.9)$$

Le ipotesi si dividono in due categorie complementari:

- Ipotesi strutturali su J ($\lambda I \preceq \nabla^2 J(w) \preceq LI$): garantiscono la convergenza lineare se la direzione di discesa è accurata (Teorema di Convergenza Deterministica), e forniscono la maggiorazione di Taylor (A.12), che sarà usata per derivare la disuguaglianza di convergenza in aspettativa.
- Ipotesi stocastica (5.4): garantisce che la varianza dello stimatore g_k decada come $\|\text{Var}(g_k)\|_1 \leq \omega/n_k$, rendendo il rumore controllabile aumentando il batch.

Iterazione stocastica a passo fisso Consideriamo l'iterazione

$$w_{k+1} = w_k - \frac{1}{L} g_k. \quad (5.10)$$

Nell'analisi stocastica si adotta il passo $1/L$, più lungo del passo deterministico $(1-\theta)/L$ poiché il fattore $(1-\theta)$ non è necessario quando il controllo dell'errore è affidato alla crescita del batch. Questo produce un tasso di contrazione deterministico $\alpha = 1 - \lambda/L$ più piccolo rispetto a $1 - (1-\theta)\lambda/L$, e quindi un reciproco $(1 - \lambda/L)^{-1}$ più grande: il vincolo su a ammette un bound superiore più alto.

Usando la (A.12) con $y = w_{k+1}$ e $w = w_k$:

$$J(w_{k+1}) \leq J(w_k) + \nabla J(w_k)^T (w_{k+1} - w_k) + \frac{L}{2} \|w_{k+1} - w_k\|^2. \quad (5.11)$$

Sostituendo $w_{k+1} - w_k = -\frac{1}{L} g_k$:

$$J(w_{k+1}) \leq J(w_k) - \frac{1}{L} \nabla J(w_k)^T g_k + \frac{1}{2L} \|g_k\|^2. \quad (5.12)$$

Passando al valore atteso condizionato a w_k :

$$\mathbb{E}[J(w_{k+1}) \mid w_k] \leq J(w_k) - \frac{1}{L} \nabla J(w_k)^T \mathbb{E}[g_k \mid w_k] + \frac{1}{2L} \mathbb{E}[\|g_k\|^2 \mid w_k].$$

Usando (5.3):

$$\mathbb{E}[J(w_{k+1}) \mid w_k] \leq J(w_k) - \frac{1}{L} \|\nabla J(w_k)\|^2 + \frac{1}{2L} \mathbb{E}[\|g_k\|^2 \mid w_k]. \quad (5.13)$$

Per esprimere $\mathbb{E}[\|g_k\|^2 \mid w_k]$, utilizziamo l'identità $\mathbb{E}[\|X\|^2] = \|\text{Var}(X)\|_1 + \|\mathbb{E}[X]\|^2$ (valida componente per componente). Applicandola a $X = g_k$:

$$\mathbb{E}[\|g_k\|^2 \mid w_k] = \|\text{Var}(g_k)\|_1 + \|\nabla J(w_k)\|^2. \quad (5.14)$$

Sostituendo in (5.13):

$$\mathbb{E}[J(w_{k+1}) \mid w_k] \leq J(w_k) - \frac{1}{2L} \|\nabla J(w_k)\|^2 + \frac{1}{2L} \|\text{Var}(g_k)\|_1. \quad (5.15)$$

Dalla formula per la varianza di una media campionaria (derivata in precedenza) e dall'ipotesi (5.4):

$$\|\text{Var}(g_k)\|_1 \leq \frac{\|\text{Var}(\nabla \ell(w_k; i))\|_1}{n_k} \leq \frac{\omega}{n_k}. \quad (5.16)$$

Sostituendo in (5.15):

$$\mathbb{E}[J(w_{k+1}) \mid w_k] \leq J(w_k) - \frac{1}{2L} \|\nabla J(w_k)\|^2 + \frac{\omega}{2Ln_k}. \quad (5.17)$$

Dalla λ -convessità forte di J (disuguaglianza di PL dimostrata nell'Appendice A.1):

$$\|\nabla J(w_k)\|^2 \geq 2\lambda(J(w_k) - J(w_*)). \quad (5.18)$$

Sostituendo in (5.17):

$$\mathbb{E}[J(w_{k+1}) - J(w_*) \mid w_k] \leq \left(1 - \frac{\lambda}{L}\right) (J(w_k) - J(w_*)) + \frac{\omega}{2Ln_k}. \quad (5.19)$$

Prendendo il valore atteso totale e ponendo $\varepsilon_k := \mathbb{E}[J(w_k) - J(w_*)]$:

$$\varepsilon_{k+1} \leq \left(1 - \frac{\lambda}{L}\right) \varepsilon_k + \frac{\omega}{2Ln_k}. \quad (5.20)$$

Questa è l'equazione fondamentale che governa la convergenza in aspettativa.

Possiamo ora enunciare il teorema di convergenza in aspettativa.

Teorema

Teorema 5.1 (Convergenza in aspettativa). Posto $\alpha := 1 - \frac{\lambda}{L}$ e $\beta := \frac{\omega}{2L}$, per ogni $k \geq 0$ vale

$$\mathbb{E}[J(w_k) - J(w_*)] \leq \alpha^k \varepsilon_0 + \beta a \frac{\alpha^k - a^{-k}}{\alpha a - 1}, \quad (5.21)$$

dove il rapporto va inteso nel senso dei limiti se $\alpha a = 1$. Di conseguenza, posto $\rho := \max\{\alpha, 1/a\}$, se $a > 1$ si ha $\rho < 1$ e esiste una costante $C =$

$C(\varepsilon_0, \omega, \lambda, L, a)$ tale che

$$\mathbb{E}[J(w_k) - J(w_*)] \leq C\rho^k \quad \forall k \geq 0.$$

Nota

Differenza rispetto al lavoro originale [1] (Teorema 4.2). Il paper ottiene $\rho = \max\{1 - \lambda/(4L), 1/a\}$ e $C = \max\{J(w_0) - J(w_*), 2\omega/\lambda\}$ tramite un argomento induttivo che introduce un fattore $\frac{1}{2}$ aggiuntivo. La dimostrazione qui presentata ottiene invece $\rho = \max\{1 - \lambda/L, 1/a\}$ risolvendo esplicitamente la ricorrenza, il che dà un tasso deterministico $\alpha = 1 - \lambda/L$ più conservativo. Di conseguenza, l'intervallo ammissibile per a nel Corollario 5.2 è $(1, (1 - \lambda/L)^{-1}]$ anziché $(1, (1 - \lambda/(4L))^{-1}]$ come nel paper.

Dimostrazione. Iterazione della ricorrenza e forma chiusa

Dalla (5.20) possiamo esplicitare l'andamento di ε_k . Scriviamo i primi passi:

$$\begin{aligned} \varepsilon_1 &\leq \alpha\varepsilon_0 + \beta, \\ \varepsilon_2 &\leq \alpha\varepsilon_1 + \beta a^{-1} \leq \alpha(\alpha\varepsilon_0 + \beta) + \beta a^{-1} = \alpha^2\varepsilon_0 + \alpha\beta + \beta a^{-1}, \\ \varepsilon_3 &\leq \alpha\varepsilon_2 + \beta a^{-2} \leq \alpha(\alpha^2\varepsilon_0 + \alpha\beta + \beta a^{-1}) + \beta a^{-2} \\ &= \alpha^3\varepsilon_0 + \alpha^2\beta + \alpha\beta a^{-1} + \beta a^{-2}. \end{aligned}$$

Il pattern generale è

$$\varepsilon_k \leq \alpha^k \varepsilon_0 + \beta \sum_{j=0}^{k-1} \alpha^{k-1-j} a^{-j}. \quad (5.22)$$

Calcoliamo la somma geometrica finita:

$$S_k := \sum_{j=0}^{k-1} \alpha^{k-1-j} a^{-j}.$$

Ponendo $i = k - 1 - j$, si ha:

$$S_k = \sum_{i=0}^{k-1} \alpha^i a^{-(k-1-i)} = a^{-(k-1)} \sum_{i=0}^{k-1} (\alpha a)^i.$$

Se $\alpha a \neq 1$, la progressione geometrica dà:

$$S_k = a^{-(k-1)} \frac{1 - (\alpha a)^k}{1 - \alpha a} = \frac{a(\alpha^k - a^{-k})}{\alpha a - 1}. \quad (5.23)$$

Sostituendo (5.23) in (5.22) si ottiene la forma chiusa:

$$\varepsilon_k \leq \alpha^k \varepsilon_0 + \beta a \frac{\alpha^k - a^{-k}}{\alpha a - 1}, \quad (5.24)$$

che è la (5.24), cioè la (5.21) del teorema. Questa espressione è valida per $\alpha a \neq 1$; nel caso $\alpha a = 1$ si intende per limite e si ottiene $\varepsilon_k \leq (\varepsilon_0 + \beta k/\alpha)\rho^k$ (con $\rho = \alpha$), che porta allo stesso tasso esponenziale a meno di un fattore polinomiale.

Deduzione del tasso di convergenza Dalla (5.21) possiamo ricavare il tasso di convergenza studiando il comportamento asintotico. Si presentano tre casi.

- Caso $\alpha a > 1$: allora $\alpha > 1/a$, quindi α^k decresce più lentamente di a^{-k} ; il termine α^k domina. Trascurando a^{-k} rispetto a α^k :

$$\varepsilon_k \leq \alpha^k \varepsilon_0 + \frac{\beta a}{\alpha a - 1} \alpha^k = \left(\varepsilon_0 + \frac{\beta a}{\alpha a - 1} \right) \alpha^k.$$

Quindi $\rho = \alpha = 1 - \lambda/L$. Possiamo trascurare il fattore a^{-k} perché è un o -piccolo rispetto ad α^k : infatti $\lim_{k \rightarrow \infty} \frac{a^{-k}}{\alpha^k} = \lim_{k \rightarrow \infty} \left(\frac{1/a}{\alpha} \right)^k = \lim_{k \rightarrow \infty} \left(\frac{1}{\alpha a} \right)^k$ poiché $\alpha a > 1$, si ha $0 < \frac{1}{\alpha a} < 1 \implies \lim_{k \rightarrow \infty} \left(\frac{1}{\alpha a} \right)^k = 0 \implies \frac{a^{-k}}{\alpha^k} \rightarrow 0$

- Caso $\alpha a < 1$: allora $\alpha < 1/a$, quindi a^{-k} decresce più lentamente di α^k ; il termine a^{-k} domina. Trascurando α^k rispetto a a^{-k} :

$$\varepsilon_k \leq \frac{\beta a}{1 - \alpha a} a^{-k} = \frac{\beta a}{1 - \alpha a} \left(\frac{1}{a} \right)^k.$$

Quindi $\rho = 1/a$. Possiamo trascurare il fattore α^k perché è un o -piccolo rispetto a a^{-k} : infatti $\lim_{k \rightarrow \infty} \frac{\alpha^k}{a^{-k}} = \lim_{k \rightarrow \infty} \frac{\alpha^k}{(1/a)^k} = \lim_{k \rightarrow \infty} (\alpha a)^k$ poiché $0 < \alpha a < 1$, si ha $0 < \alpha a < 1 \implies \lim_{k \rightarrow \infty} (\alpha a)^k = 0 \implies \frac{\alpha^k}{a^{-k}} \rightarrow 0$.

- Caso $\alpha a = 1$: allora $\alpha = 1/a$. La somma geometrica si riduce a $S_k = k a^{-(k-1)}$, da cui:

$$\varepsilon_k \leq \alpha^k \varepsilon_0 + \beta k a^{-(k-1)} = (\varepsilon_0 + \beta k / \alpha) \rho^k, \quad \rho := \alpha = 1/a.$$

Poiché per ogni $\rho' \in (\rho, 1)$ esiste una costante C (indipendente da k) tale che $k \rho^k \leq C(\rho')^k$, la tesi vale con tasso ρ' .

Quindi per $\alpha a > 1$ si ottiene $\rho = \alpha$ e $C = \varepsilon_0 + \frac{\beta a}{\alpha a - 1}$; se $\alpha a < 1$ si ottiene $\rho = 1/a$ e $C = \varepsilon_0 + \frac{\beta a}{1 - \alpha a}$ (o, più strettamente, $C = \max\{\varepsilon_0, \frac{\beta a}{1 - \alpha a}\}$).

In tutti i casi si ha:

$$\varepsilon_k \leq C \rho^k, \quad \rho := \max\{\alpha, 1/a\}, \quad (5.25)$$

per una opportuna costante $C = C(\varepsilon_0, \omega, \lambda, L, a)$.

□

Corollario sulla complessità totale

Corollario

Corollario 5.2 (Complessità totale). Supponiamo $n_k = \lceil a^k \rceil$ con

$$a \in \left(1, \underbrace{\left(1 - \frac{\lambda}{L} \right)^{-1}}_{1/\alpha} \right],$$

e che (5.4) valga per tutti i w_k . Allora il numero totale di valutazioni dei

gradienti $\nabla \ell(w; i)$ necessarie per ottenere $\mathbb{E}[J(w_k) - J(w_*)] \leq \varepsilon$ è

$$O\left(\frac{L}{\lambda \varepsilon}\right), \quad (5.26)$$

e il lavoro totale dell'algoritmo (5.10) con campionamento dinamico (5.9) è

$$\mathcal{T}_{\text{tot}} = O\left(\frac{Lm}{\lambda \varepsilon} \max\left\{J(w_0) - J(w_*), \frac{\omega}{\lambda}\right\}\right), \quad (5.27)$$

dove m è il numero di variabili.

Dimostrazione. Con la scelta $a \leq \alpha^{-1}$, si ha $\rho = 1/a$. Dalla (5.21), il numero di iterazioni k necessario per ottenere $\mathbb{E}[J(w_k) - J(w_*)] \leq \varepsilon$ soddisfa $a^k \geq C/\varepsilon$. Poiché $\rho = 1/a$, la condizione $C\rho^k \leq \varepsilon$ equivale a $a^k \geq C/\varepsilon$. Il numero di iterazioni necessario soddisfa quindi $a^k = \Theta(C/\varepsilon)$. Il costo in valutazioni del gradiente è:

$$\sum_{j=0}^{k-1} n_j \leq 2 \sum_{j=0}^{k-1} a^j = 2 \cdot \frac{a^k - 1}{a - 1} \leq \frac{2}{a - 1} a^k = O\left(\frac{1}{\varepsilon}\right).$$

Moltiplicando per il costo $O(m)$ di ogni gradiente si ottiene (5.27). $\square$

Scelta di a e confronto Il risultato (5.27) vale se a è scelto nell'intervallo

$$a \in \left(1, \left(1 - \frac{\lambda}{L}\right)^{-1}\right].$$

Non è necessario conoscere il numero di condizionamento $\kappa = L/\lambda$ esattamente; qualsiasi sovrastima va bene:

$$a = \left(1 - \frac{1}{\kappa'}\right)^{-1}, \quad \kappa' \geq \kappa. \quad (5.28)$$

Dalla (5.27) ignorando la dipendenza dal punto iniziale (come in [2]) e assumendo il regime $\omega/\lambda \geq J(w_0) - J(w_*)$, si ottiene $\mathcal{T}_{\text{tot}} = O(m\kappa\omega/\lambda\varepsilon)$, che è il bound riportato in tabella sotto.

Confrontiamo questo risultato con i bound di complessità noti per il metodo a gradiente a campione fisso e il metodo del gradiente stocastico (SGD), calcolati da Bottou e Bousquet [2]: nella tabella, lo scalare $\bar{\alpha} \in [1/2, 1]$ è il *tasso di stima* del metodo a campione fisso, come definito in [2].

Nome algoritmo	Bound
Metodo a gradiente dinamico (questo lavoro)	$O(m\kappa\omega/\lambda\varepsilon)$
Metodo a gradiente a campione fisso	$O(m^2\kappa\varepsilon^{-1/\bar{\alpha}} \log^2(1/\varepsilon))$
Metodo del gradiente stocastico	$O(m\bar{\nu}\kappa^2/\varepsilon)$

Tabella 5.1: Bound di complessità per tre metodi. Qui m è il numero di variabili, $\kappa = L/\lambda$, ω e $\bar{\nu}$ sono definiti in (5.4) e (5.29), e $\bar{\alpha} \in [1/2, 1]$.

I metodi a campione fisso e stocastico sono definiti come:

- Metodo a campione fisso: $w_{k+1} = w_k - \frac{1}{L} \frac{\sum_{i=1}^{N_\varepsilon} \nabla \ell(w_k; i)}{N_\varepsilon},$
- Metodo del gradiente stocastico: $w_{k+1} = w_k - \frac{1}{\lambda k} \nabla \ell(w_k; i_k),$

dove N_ε specifica una dimensione del campione tale che $\mathbb{E}[|J(w_*) - J_S(w_*)|] < \varepsilon$ vale in aspettativa. I bound di complessità presentati in [2] ignorano l'effetto del punto iniziale w_0 .

Lo scalare $\bar{\alpha} \in [1/2, 1]$, chiamato *tasso di stima* [2], dipende dalle proprietà della funzione di perdita; la costante $\bar{\nu}$ è definita come

$$\bar{\nu} = \text{trace}(H^{-1}G), \quad H = \nabla^2 J(w_*), \quad G = \mathbb{E}[\nabla \ell(w_*; i) \nabla \ell(w_*; i)^T]. \quad (5.29)$$

Poiché $H \succeq \lambda I$ e ω è un bound per $\|\text{Var}(\nabla \ell(w_k; I))\|_1$, si ha $\bar{\nu} \approx \omega/\lambda$. Sostituendo nel bound per SGD:

$$O(m\bar{\nu}\kappa^2/\varepsilon) = O\left(\frac{m\omega\kappa^2}{\lambda\varepsilon}\right). \quad (5.30)$$

Il bound per il metodo dinamico (5.27) è invece $O(m\kappa\omega/(\lambda\varepsilon))$. La differenza principale è il termine κ^2 in SGD contro κ nel metodo dinamico. Concludiamo che il metodo a gradiente dinamico gode di un bound di complessità competitivo con quello del metodo del gradiente stocastico, risultando particolarmente vantaggioso per problemi mal condizionati (grande κ).

La strategia dinamica $n_k = \lceil a^k \rceil$ con a scelto in base a una stima (bound superiore) di κ garantisce un compromesso ottimale tra costo per iterazione e velocità di convergenza. In pratica, la stima di κ può essere sostituita da una regola euristica semplice (ad esempio, $a = 1.1$), e l'algoritmo si adatta automaticamente.

5.2 Metodo di Newton-CG con Campionamento Dinamico

Il metodo del gradiente (steepest descent) può convergere lentamente, specialmente su problemi mal condizionati; gli algoritmi che incorporano informazione di curvatura (secondo ordine) sulla funzione obiettivo possono essere molto più efficienti. In questa sezione descriviamo un metodo di Newton-CG (gradiente coniugato) Hessian free (non costruiamo e non memorizziamo esplicitamente la matrice Hessiana di $J(w)$, ma ne usiamo solo i prodotti per un vettore), che impiega tecniche di campionamento dinamico sia per il calcolo della funzione e del gradiente, sia per i prodotti Hessiana-vettore.

5.2.1 Struttura del metodo

Il metodo estende l'algoritmo del gradiente a campione dinamico (quello pratico, non la sua variante idealizzata con $n_k = \lceil a^k \rceil$). Rispetto ai metodi che utilizzano un campione fisso per il gradiente e un numero fisso di iterazioni del CG, questo metodo presenta due innovazioni principali:

1. Il campione $\mathcal{S}_k$ per il gradiente non è più fisso; qui si incorporano le tecniche di campionamento dinamico dell'algoritmo visto precedentemente.
2. Il numero di iterazioni del gradiente coniugato non è più fisso; si propone un criterio di terminazione automatico per il CG, che si adatta alla dimensione del campione e all'accuratezza dell'approssimazione dell'Hessiana.

Ad ogni iterazione, il metodo sceglie due campioni $\mathcal{S}_k$ e $\mathcal{H}_k$ tali che

$$|\mathcal{H}_k| \ll |\mathcal{S}_k|,$$

e definisce la direzione di ricerca d_k come soluzione approssimata del sistema lineare

$$\underbrace{\nabla^2 J_{\mathcal{H}_k}(w_k) d = -\nabla J_{\mathcal{S}_k}(w_k)}_{\text{(metodo di Newton classico)}}. \quad (5.31)$$

L'uso di due campioni distinti è cruciale:

- $\mathcal{S}_k$ per il gradiente $\nabla J_{\mathcal{S}_k}(w_k)$, per garantire una direzione di discesa affidabile.
- $\mathcal{H}_k$ per l'Hessiana $\nabla^2 J_{\mathcal{H}_k}(w_k)$, per mantenere il costo dei prodotti Hessiana-vettore sufficientemente basso.

Il sistema (5.31) viene risolto con il metodo del gradiente coniugato (CG) Hessian free: la matrice $\nabla^2 J_{\mathcal{H}_k}(w_k)$ non viene mai costruita esplicitamente; invece, si calcola direttamente il prodotto di questa Hessiana per un vettore.

Il rapporto

$$R = \frac{|\mathcal{H}_k|}{|\mathcal{S}_k|} < 1 \quad (5.32)$$

viene fissato a priori ed è mantenuto costante durante tutto l'algoritmo. Quando il campione per il gradiente $\mathcal{S}_k$ cresce (dinamicamente), la dimensione di $\mathcal{H}_k$ viene aggiornata come $|\mathcal{H}_k| = \lceil R \cdot |\mathcal{S}_k| \rceil$, in modo che $\mathcal{H}_k$ cresca proporzionalmente a $\mathcal{S}_k$. Una linea guida pratica è che il costo totale dei prodotti Hessiana-vettore nel CG non superi il costo di una valutazione del gradiente $\nabla J_{\mathcal{S}_k}$.

5.2.2 Criterio di terminazione del gradiente coniugato

Proponiamo un criterio automatico per decidere con quale accuratezza risolvere il sistema (5.31). L'idea chiave è che il residuo del CG,

$$r_k \stackrel{\text{def}}{=} \nabla^2 J_{\mathcal{H}_k}(w_k)d + \nabla J_{\mathcal{S}_k}(w_k), \quad (5.33)$$

non deve essere più piccolo dell'errore di approssimazione dell'Hessiana.

Per comprendere questo punto, scriviamo il residuo del sistema di Newton che userebbe il campione grande $\mathcal{S}_k$ sia per il gradiente che per l'Hessiana:

$$\underbrace{\nabla^2 J_{\mathcal{S}_k}(w_k)d + \nabla J_{\mathcal{S}_k}(w_k)}_{\text{Residuo del sistema}} = \underbrace{r_k}_{\text{Residuo del CG}} + \underbrace{[\nabla^2 J_{\mathcal{S}_k}(w_k) - \nabla^2 J_{\mathcal{H}_k}(w_k)] d}_{\text{Errore di Hessiana } = \Delta_{\mathcal{H}_k}(w_k; d)}. \quad (5.34)$$

Il termine $\Delta_{\mathcal{H}_k}(w_k; d)$ misura l'errore nell'approssimazione dell'Hessiana lungo la direzione d , dovuto all'uso del campione più piccolo $\mathcal{H}_k$ e questo è costante durante l'algoritmo. È chiaro dall'equazione sopra che se l'errore di Hessiana $\Delta_{\mathcal{H}_k}$ non si può eliminare, non serve ridurre r_k fino a zero. Basta ridurlo fino a quando è comparabile a $\Delta_{\mathcal{H}_k}$. Oltre quel punto, il tempo speso è sprecato.

Il nostro obiettivo è quindi bilanciare i due termini al membro destro di (5.34), terminando il CG quando il residuo r_k è dello stesso ordine dell'errore dell'hessiana.

Tuttavia, $\Delta_{\mathcal{H}_k}(w; d)$, analogamente all'errore di approssimazione del gradiente discusso precedentemente, non è direttamente calcolabile, perché richiederebbe di calcolare $\nabla^2 J_{\mathcal{S}_k}(w_k)d$ (cioè il prodotto Hessiana-vettore sul campione grande), il che vanificherebbe il vantaggio del sottocampionamento dell'Hessiana.

Seguendo la stessa strategia implementata precedentemente per il gradiente, usiamo la varianza campionaria del prodotto Hessiana-vettore per stimare l'errore.

Errore del gradiente (sez. 5.1.2)	Errore dell'Hessiana (sez. 5.2.2)
$e_k = g_k - \nabla J(w_k)$	$\Delta_{\mathcal{H}_k}(w_k; d) = [\nabla^2 J_{\mathcal{S}_k}(w_k) - \nabla^2 J_{\mathcal{H}_k}(w_k)] d$
Non calcolabile perché richiederebbe $\nabla J(w_k)$ su tutti i dati	Non calcolabile perché richiederebbe $\nabla^2 J_{\mathcal{S}_k}(w_k)d$ su tutto il campione grande
Si aggira il problema stimando $\mathbb{E}[\ e_k\ ^2]$ con la varianza campionaria dei gradienti individuali $\nabla \ell(w_k; i)$ sul batch	Si aggira il problema stimando $\mathbb{E}[\ \Delta\ ^2]$ con la varianza campionaria dei prodotti Hessiana-vettore individuali $\nabla^2 \ell(w_k; i)d$ sul sottocampione $\mathcal{H}_k$

Per comprendere la derivazione, partiamo dalla definizione di $\Delta_{\mathcal{H}_k}$.

Precedentemente avevamo definito

$$\Delta_{\mathcal{H}_k}(w_k; d) \stackrel{\text{def}}{=} [\nabla^2 J_{\mathcal{S}_k}(w_k) - \nabla^2 J_{\mathcal{H}_k}(w_k)] d.$$

Sviluppando le medie campionarie e ponendo $X_i = \nabla^2 \ell(w_k; i)d$, abbiamo

$$\nabla^2 J_{\mathcal{S}_k}(w_k)d = \frac{1}{|\mathcal{S}_k|} \sum_{i \in \mathcal{S}_k} X_i := \mu_{\mathcal{S}} \quad \text{e} \quad \nabla^2 J_{\mathcal{H}_k}(w_k)d = \frac{1}{|\mathcal{H}_k|} \sum_{i \in \mathcal{H}_k} X_i := \mu_{\mathcal{H}}.$$

Indichiamo $\Delta_{\mathcal{H}_k}(w_k; d)$ con Δ , poiché $\mathcal{H}_k$ è un sottocampione casuale estratto senza reinserimento dall'insieme $\mathcal{S}_k$, la quantità $\mu_{\mathcal{H}}$ è uno stimatore della media

della popolazione finita $\mu_{\mathcal{S}}$. L'errore di stima è $\mu_{\mathcal{H}} - \mu_{\mathcal{S}}$, il cui quadrato atteso è la varianza dello stimatore $\mu_{\mathcal{H}}$. Osserviamo che

$$\|\Delta\|^2 = \|\mu_{\mathcal{S}} - \mu_{\mathcal{H}}\|^2 = \|\mu_{\mathcal{H}} - \mu_{\mathcal{S}}\|^2 \implies \mathbb{E}[\|\Delta\|^2] = \mathbb{E}[\|\mu_{\mathcal{H}} - \mu_{\mathcal{S}}\|^2] = \text{Var}(\mu_{\mathcal{H}}).$$

A questo punto, usiamo la formula per la varianza di una media campionaria in un campionamento senza reinserimento che avevamo derivato precedentemente. Consideriamo una popolazione finita di dimensione $N = |\mathcal{S}_k|$ con elementi $X_1, \dots, X_N$, e un campione senza reinserimento di dimensione $n = |\mathcal{H}_k|$.

$$\text{Var}(\mu_{\mathcal{H}}) = \frac{\sigma^2}{n} \cdot \frac{N - n}{N - 1}.$$

Questa è la formula classica per la varianza di una media campionaria in un campionamento senza reinserimento, che include il fattore di correzione per popolazione finita $\frac{N-n}{N-1}$. Sostituendo $n = |\mathcal{H}_k|$, $N = |\mathcal{S}_k|$ e prendendo la norma $\|\cdot\|_1$ componente per componente (poiché gli X_i sono vettori), otteniamo:

$$\mathbb{E}[\|\Delta\|^2] = \frac{\overbrace{\left\| \frac{1}{|\mathcal{S}_k|} \sum_{i \in \mathcal{S}_k} (X_i - \mu_{\mathcal{S}})^2 \right\|_1}^{\sigma^2}}{|\mathcal{H}_k|} \cdot \frac{|\mathcal{S}_k| - |\mathcal{H}_k|}{|\mathcal{S}_k| - 1}.$$

Ritornando alla notazione originale, $X_i = \nabla^2 \ell(w_k; i) d$, e ricordando che σ^2 denota la varianza della popolazione $\mathcal{S}_k$ che approssimiamo con la varianza campionaria calcolata su $\mathcal{H}_k$, otteniamo la prima approssimazione:

$$\mathbb{E}[\|\Delta_{\mathcal{H}_k}(w_k; d)\|_2^2] \approx \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d)\|_1}{|\mathcal{H}_k|} \frac{(|\mathcal{S}_k| - |\mathcal{H}_k|)}{|\mathcal{S}_k| - 1}. \quad (5.35a)$$

Questo viene fatto siccome l'uso di σ^2 richiederebbe di calcolare il prodotto Hessiana-vettore su tutto il campione grande $\mathcal{S}_k$, il che vanificherebbe il vantaggio del sotto-campionamento dell'Hessiana.

Poiché nel paper si assume $|\mathcal{H}_k| \ll |\mathcal{S}_k|$ (il campione per l'Hessiana è molto più piccolo di quello per il gradiente), il rapporto $\frac{|\mathcal{S}_k| - |\mathcal{H}_k|}{|\mathcal{S}_k| - 1}$ è circa uguale a 1. Si ottiene quindi la semplificazione finale:

$$\mathbb{E}[\Delta_{\mathcal{H}_k}(w_k; d)^2] \approx \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d)\|_1}{|\mathcal{H}_k|}. \quad (5.35b)$$

Questa equazione fornisce una stima dell'errore di approssimazione dell'Hessiana per un generico vettore d . Osserviamo che la mappa $d \mapsto \nabla^2 \ell(w_k; i) d$ è lineare in d . Di conseguenza, la sua varianza (calcolata su un campione fissato) è una forma quadratica in d : esiste una matrice di covarianza campionaria $\Sigma_{\mathcal{H}_k} \in \mathbb{R}^{m \times m}$, simmetrica semidefinita positiva, tale che

$$\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d) = d^T \Sigma_{\mathcal{H}_k} d \in \mathbb{R}^m,$$

dove il membro sinistro è inteso componente per componente e la relazione vale per ogni componente $j = 1, \dots, m$ con la corrispondente matrice $\Sigma_{\mathcal{H}_k}^{(j)}$. Questa forma quadratica gode della proprietà di omogeneità

$$\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i)(\lambda d)) = \lambda^2 \text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d).$$

All'inizio dell'algoritmo, la soluzione candidata è $d_0 = 0$ e il residuo iniziale è $r_0 = \nabla J_{\mathcal{S}_k}(w_k)$. La prima direzione di ricerca del CG è quindi $p_0 = -r_0$. Per evitare di ricalcolare la varianza a ogni iterazione del CG (operazione costosa), il lavoro originale [1] propone di calcolarla una sola volta per $d = p_0$. Sfruttando l'omogeneità della forma quadratica, si definisce

$$\alpha := \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) p_0)\|_1}{\|p_0\|_2^2} = \frac{\|p_0^T \Sigma_{\mathcal{H}_k} p_0\|_1}{\|p_0\|_2^2}. \quad (5.36)$$

Poiché la varianza effettiva per una generica direzione d_j è $\|d_j^T \Sigma_{\mathcal{H}_k} d_j\|_1$, e non $\alpha \|d_j\|^2$, l'uso di α per tutte le direzioni costituisce un'approssimazione, α misura l'influenza media della matrice di covarianza campionaria dei prodotti Hessiana-vettore su un vettore generico, è una media pesata degli autovalori di $\Sigma_{\mathcal{H}_k}$ ed è sempre compreso tra $\lambda_{\min}(\Sigma_{\mathcal{H}_k})$ e $\lambda_{\max}(\Sigma_{\mathcal{H}_k})$. Il metodo assume che la forma quadratica sia sufficientemente ben condizionata perché la dipendenza angolare sia trascurabile ai fini del test di arresto; ciò giustifica la stima

$$\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) d_j)\|_1 \approx \alpha \|d_j\|_2^2 = \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) p_0)\|_1}{\|p_0\|_2^2} \|d_j\|_2^2. \quad (5.37)$$

Ora, dalla (5.35b), sostituendo la varianza stimata tramite la (5.37), definiamo

$$\gamma := \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) p_0)\|_1}{|\mathcal{H}_k| \|p_0\|_2^2}, \quad \Psi(d) := \gamma \|d\|_2^2. \quad (5.38)$$

Test di arresto del CG. La soglia $\Psi(d_j)$ definita in (5.38) viene utilizzata per decidere quando fermare il metodo del gradiente coniugato. Ad ogni iterazione j del CG, si controlla se il residuo r_{j+1} soddisfa

$$\|r_{j+1}\|_2^2 \leq \Psi(d_j) = \gamma \|d_j\|_2^2. \quad (5.39)$$

Se la condizione è verificata, il CG termina e la soluzione corrente d_{j+1} viene accettata come direzione di Newton approssimata. In caso contrario, si prosegue con la successiva iterazione del CG. Questo criterio di arresto è particolarmente efficace perché evita calcoli inutili quando l'approssimazione dell'Hessiana è limitata dal campione $\mathcal{H}_k$; si adatta alla distanza dal minimo poiché $\Psi \propto \|r_0\|^2$; previene instabilità numeriche perché Ψ cresce con $\|d_j\|^2$, fermando il CG prima che la direzione diventi eccessivamente grande.

L'algoritmo quindi calcola γ una volta sola all'inizio del CG, e poi ad ogni iterazione aggiorna $\Psi(d_j) = \gamma \|d_j\|^2$ e verifica la condizione (5.39).

5.2.3 Pseudocodice (Algoritmo Newton-CG)

Algoritmo

Newton-CG con Campionamento Dinamico

Input: w_0 (punto iniziale), $\mathcal{S}_0$ (batch iniziale per gradiente), $\mathcal{H}_0 \subset \mathcal{S}_0$ (batch iniziale per Hessiana), $R = |\mathcal{H}_0|/|\mathcal{S}_0|$ (rapporto costante), $\theta \in (0, 1)$ (tolleranza per varianza del gradiente), $c_1, c_2 \in (0, 1)$ con $c_1 < c_2$ (parametri Wolfe).

Inizializzazione: poni $k = 0$.

Ripeti per ogni iterazione k fino a convergenza:

1. Calcola la direzione di Newton approssimata d_k mediante gradiente coniugato (CG):

- (a) $d_0 = 0, r_0 = \nabla J_{\mathcal{S}_k}(w_k), p_0 = -r_0, j = 0, \Psi = 0.$

- (b) $\gamma = \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) p_0)\|_1}{|\mathcal{H}_k| \|p_0\|_2^2}.$

- (c) Finchè $\|r_j\|_2^2 > \Psi$:

- i. $s_j = \nabla^2 J_{\mathcal{H}_k}(w_k) p_j$

- ii. $\alpha_j = \frac{r_j^T r_j}{p_j^T s_j}.$

- iii. $d_{j+1} = d_j + \alpha_j p_j, r_{j+1} = r_j + \alpha_j s_j.$

- iv. $\beta_{j+1} = \frac{r_{j+1}^T r_{j+1}}{r_j^T r_j}.$

- v. $p_{j+1} = -r_{j+1} + \beta_{j+1} p_j.$

- vi. $j = j + 1, \Psi = \gamma \|d_j\|_2^2.$

- (d) $d_k = d_j$

2. Line search con condizioni di Wolfe: Trova $\alpha_k > 0$ tale che:

$$\begin{aligned} J_{\mathcal{S}_k}(w_k + \alpha_k d_k) &\leq J_{\mathcal{S}_k}(w_k) + c_1 \alpha_k \nabla J_{\mathcal{S}_k}(w_k)^T d_k, \\ \nabla J_{\mathcal{S}_k}(w_k + \alpha_k d_k)^T d_k &\geq c_2 \nabla J_{\mathcal{S}_k}(w_k)^T d_k. \end{aligned}$$

3. Aggiorna il punto: $w_{k+1} = w_k + \alpha_k d_k.$

4. Aggiornamento dinamico del batch per il gradiente:

- (a) Seleziona un nuovo campione $\mathcal{S}_{k+1}$ della stessa dimensione di $\mathcal{S}_k.$

- (b) Calcola la varianza campionaria $\hat{\mathcal{V}} = \text{Var}_{i \in \mathcal{S}_{k+1}}(\nabla \ell(w_{k+1}; i)).$

- (c) Se $\frac{\|\hat{\mathcal{V}}\|_1}{|\mathcal{S}_{k+1}|} > \theta^2 \|\nabla J_{\mathcal{S}_{k+1}}(w_{k+1})\|_2^2$, aumenta il batch:

$$|\mathcal{S}_{k+1}| \leftarrow \left\lceil \frac{\|\hat{\mathcal{V}}\|_1}{\theta^2 \|\nabla J_{\mathcal{S}_{k+1}}(w_{k+1})\|_2^2} \right\rceil.$$

5. Seleziona $\mathcal{H}_{k+1} \subseteq \mathcal{S}_{k+1}$ tale che $|\mathcal{H}_{k+1}| = R \cdot |\mathcal{S}_{k+1}|.$

6. Incrementa $k \leftarrow k + 1.$



Figura 5.4: Schema del metodo Newton-CG con campionamento dinamico: ad ogni iterazione il gradiente coniugato risolve (in modo Hessian free) il sistema di Newton con un criterio di arresto adattivo; dopo l'aggiornamento e il ricampionamento, la CCV decide se aumentare la dimensione del batch.

Nota Tecnica

L'algoritmo del gradiente coniugato serve a minimizzare l'errore di $x \approx \bar{x}$ t.c. $A\bar{x} = b$. Nel contesto del metodo di Newton, i suoi elementi vengono reinterpretati come segue:

- $A = H_k = \nabla^2 J(w_k)$ (Hessiana della funzione obiettivo, calcolata sul campione $\mathcal{H}_k$);
- $b = -g_k = -\nabla J(w_k)$ (gradiente negativo, calcolato sul campione $\mathcal{S}_k$);

- $x = d_k$ (direzione di Newton, che risolve $H_k d_k = -g_k$).

Il CG, quindi, viene usato internamente per approssimare la direzione di Newton senza costruire esplicitamente l'Hessiana.

INPUT: $A \in \mathbb{R}^{n \times n}$ (simmetrica def. pos.), $b \in \mathbb{R}^n$, $x_0 = \mathbf{1}$, $\text{tol} = 10^{-10} \|b\|_2$, $it_{\max} = 1000$

- Calcola $r = b - Ax_0$, $p = r$, residui = $\|r\|_2$
- Per $k = 1$ a $it_{\max}$:
 - $Ap = A \cdot p$
 - $\alpha = \frac{r^T p}{p^T Ap}$ (Passo ottimale)
 - $x = x + \alpha p$
 - $r' = r - \alpha Ap$
 - $\beta = \frac{r'^T r'}{r'^T r}$
 - $p = r' + \beta p$
 - $r = r'$
 - residui = $\|r\|_2$
 - Se $\|r\|_2 \leq \text{tol}$: termina

OUTPUT: x , residui

Questo metodo usa un passo α_k che si ottiene minimizzando il funzionale

$$\phi(x) = \frac{1}{2} x^T A x - b^T x$$

per $x = x_k + \alpha p_k$, annullando la derivata di tale funzionale rispetto ad α :

$$\left. \frac{d}{d\alpha} \phi(x_k + \alpha p_k) \right|_{\alpha=\alpha_k} = (Ax_k - b)^T p_k + \alpha_k p_k^T A p_k = 0.$$

Poiché $Ax_k - b = -r_k$, dove $r_k = b - Ax_k$ è il residuo, si ha:

$$-r_k^T p_k + \alpha_k p_k^T A p_k = 0 \implies \alpha_k = \frac{r_k^T p_k}{p_k^T A p_k}. \quad (5.40)$$

Ora però si può dimostrare che la formula usata in questo metodo è equivalente a questa. La dimostrazione completa è riportata nell'Appendice A.3. In sintesi, dalla condizione di stazionarietà al passo precedente e dall'aggiornamento del residuo si ottiene $r_k^T p_{k-1} = 0$, da cui segue $r_k^T p_k = r_k^T r_k$ e quindi l'equivalenza delle due formule. La formula con $r_k^T r_k$ è preferita nelle implementazioni numeriche perché è più stabile, poiché non dipende dall'ortogonalità numerica di $r_k^T p_{k-1}$, che in presenza di errori di arrotondamento non è esattamente zero.

In sostanza l'algoritmo a ogni iterazione k estrae due campioni distinti dal dataset. Un campione S_k grande per calcolare il gradiente $g_k = \nabla J_{S_k}(w_k)$, perché deve essere una stima accurata della direzione di discesa, e un sottocampione H_k con $|H_k| = R \cdot |S_k|$ e $R \ll 1$ per calcolare l'Hessiana $H_k = \nabla^2 J_{H_k}(w_k)$, poiché i prodotti Hessiana-vettore sono costosi e una stima della curvatura basta. Per aggiornare i pesi usa la direzione di Newton che si ottiene minimizzando l'espansione di Taylor per $J(w)$ al secondo ordine attorno a w_k : $J(w_k + d) \approx \tilde{J}(d) = J(w_k) + \nabla J(w_k)^T d + \frac{1}{2} d^T \nabla^2 J(w_k) d$, dove w_k è il punto corrente e d è la direzione. Troviamo la d che minimizza questo modello quadratico annullandone il gradiente rispetto a d , ottenendo il sistema lineare $\nabla^2 J(w_k) d = -\nabla J(w_k)$, che è praticamente del tipo $Ax = b$ con $A = \nabla^2 J(w_k)$ e $b = -\nabla J(w_k)$. Per risolvere questo sistema non si inverte mai la matrice (troppo costoso), ma si usa il metodo del gradiente coniugato Hessian free, facendo solo prodotti $H_k \cdot v$ senza costruire H_k .

Ora poiché l'Hessiana è calcolata su un campione piccolo H_k e quindi è approssimata, non ha senso risolvere il sistema con precisione assoluta. Il CG si ferma quando il suo residuo è confrontabile con l'errore di approssimazione dell'Hessiana, stimato dalla varianza campionaria dei prodotti $\nabla^2 \ell(w_k; i) \cdot p_0$ su H_k . In questo modo non si spreca iterazioni del CG a ridurre un errore che è già dominato dal rumore del campionamento.

Infine si esegue una line search di Armijo lungo la direzione d_k trovata, si aggiorna $w_{k+1} = w_k + \alpha_k d_k$, e come nell'algoritmo visto precedentemente si verifica la condizione di controllo della varianza (CCV) sul gradiente: se la varianza campionaria di g_k è troppo alta rispetto a $\|g_k\|^2$, si aumenta la dimensione del batch n_k per l'iterazione successiva, in modo da avere più precisione man mano che ci si avvicina al minimo.

Ora il metodo del gradiente coniugato è un metodo numerico per trovare x che approssimi la soluzione di $Ax = b$, ovvero minimizzi il funzionale $\phi(x) = \frac{1}{2} x^T A x - b^T x$ e per farlo cerca di minimizzare $\|e^{k+1}\|_A^2$ cercando in $e^k + \text{span}(p^k, p^{k-1})$; equivalentemente pone $e^{k+1} \perp_A p^k$ ed $e^{k+1} \perp_A p^{k-1}$, dalla prima condizione troviamo α_k e dalla seconda β_k (Dimostrazione nell'appendice).

5.3 Metodo Newton-CG per Modelli con Regularizzazione L_1

5.3.1 Formulazione del Problema

Nei modelli di apprendimento statistico con un numero molto elevato di parametri, l'aggiunta di un termine di regularizzazione L_1 è particolarmente vantaggiosa perché produce soluzioni sparse: molti coefficienti vengono automaticamente portati a zero, individuando le variabili effettivamente rilevanti per la predizione e rendendo il modello risultante più semplice e più facile da interpretare.

La funzione obiettivo (3.1) viene quindi modificata aggiungendo un termine di penalità sulla norma L_1 dei parametri:

$$F(w) = \frac{1}{N} \sum_{i=1}^N \ell(f(w; x_i), y_i) + \nu \|w\|_1 = J(w) + \nu \|w\|_1, \quad (5.41)$$

dove:

- $\nu > 0$ è un parametro di penalità (fisso);
- $\|w\|_1 = \sum_{j=1}^m |w_j|$ è la norma L_1 del vettore dei parametri;
- $J(w)$ è la funzione di perdita empirica definita in (3.1).

Nel contesto mini batch, si sceglie un sottoinsieme $\mathcal{S} \subseteq \{1, \dots, N\}$ del training set e si risolve il problema approssimato:

$$\min_{w \in \mathbb{R}^m} F_{\mathcal{S}}(w) = J_{\mathcal{S}}(w) + \nu \|w\|_1, \quad (5.42)$$

dove $J_{\mathcal{S}}$ è definita come in (5.1).

5.3.2 Il gradiente generalizzato

Per comprendere come si costruisce il gradiente generalizzato di una funzione contenente un termine L_1 , è utile partire dal caso più semplice in una variabile.

Caso 1D: la funzione $\nu|x|$. Consideriamo la funzione $\phi(x) = \nu|x|$, con $x \in \mathbb{R}$, $\nu \in \mathbb{R}$, $\nu > 0$. Questa funzione presenta un punto di non derivabilità in $x = 0$:

- se $x > 0$, la derivata è $\phi'(x) = +\nu$;
- se $x < 0$, la derivata è $\phi'(x) = -\nu$;
- se $x = 0$, la derivata classica non esiste.

Siccome la derivata classica non esiste, si utilizza il concetto di subgradiente (o gradiente generalizzato). Per $\phi(x) = \nu|x|$, il subgradiente in $x = 0$ è l'intervallo:

$$\partial\phi(0) = [-\nu, +\nu].$$

Questo significa che in $x = 0$ possiamo scegliere qualsiasi valore di pendenza compreso tra $-\nu$ e $+\nu$, a seconda della direzione di discesa che vogliamo seguire.

Estensione al caso multidimensionale. Consideriamo ora:

$$F(w) = \underbrace{J(w)}_{\text{parte liscia}} + \underbrace{\nu \sum_{i=1}^m |w_i|}_{\text{parte non liscia}}.$$

La parte non liscia è una somma di termini $\nu|w_i|$, uno per ogni componente di w . Poiché la somma si decompone componente per componente, possiamo calcolare il gradiente generalizzato di F separatamente per ogni coordinata.

Per la i -esima componente, abbiamo due contributi:

1. la derivata parziale della parte liscia: $\frac{\partial J(w)}{\partial w_i}$;
2. il subgradiente di $\nu|w_i|$:

$$\partial(\nu|w_i|) = \begin{cases} +\nu & \text{se } w_i > 0, \\ -\nu & \text{se } w_i < 0, \\ [-\nu, +\nu] & \text{se } w_i = 0. \end{cases}$$

Pertanto, per ogni componente i , il gradiente generalizzato $\tilde{\nabla}F(w)$ è dato da:

$$[\tilde{\nabla}F(w)]_i = \underbrace{\frac{\partial J(w)}{\partial w_i}}_{\text{gradiente di } J} + \underbrace{g_i}_{\text{subgradiente di } \nu|w_i|} \quad g_i \in \begin{cases} \{+\nu\} & \text{se } w_i > 0, \\ \{-\nu\} & \text{se } w_i < 0, \\ [-\nu, +\nu] & \text{se } w_i = 0. \end{cases}$$

Quando $w_i = 0$, abbiamo libertà di scegliere g_i in $[-\nu, +\nu]$. La scelta viene effettuata in modo che in w^* si abbia $\tilde{\nabla}F(w^*) = 0$. Per ogni coordinata i , definiamo:

$$\begin{cases} d_i := \frac{\partial J}{\partial w_i} \\ g_i \in [-\nu, +\nu] \end{cases} \implies [\tilde{\nabla}F]_i = d_i + g_i.$$

La scelta di g_i viene effettuata risolvendo il problema di proiezione:

$$g_i = \arg \min_{g \in [-\nu, \nu]} |d_i + g|,$$

cioè si sceglie il subgradiente che rende il gradiente generalizzato $d_i + g_i$ il più vicino possibile a zero. Ne derivano i tre casi seguenti:

Condizione su $d_i = \partial J / \partial w_i$	Scelta di g_i	Risultato $[\tilde{\nabla}F]_i$	Effetto
$d_i < -\nu$	$g_i = +\nu$	$d_i + \nu < 0$	$\tilde{\nabla}F < 0$: la discesa spinge w_i verso destra (aumenta)
$d_i > +\nu$	$g_i = -\nu$	$d_i - \nu > 0$	$\tilde{\nabla}F > 0$: la discesa spinge w_i verso sinistra (diminuisce)
$-\nu \leq d_i \leq +\nu$	$g_i = -d_i$	$d_i + g_i = 0$	$\tilde{\nabla}F = 0$: $w_i = 0$ è punto stazionario (minimo)

In sintesi, la regola di proiezione $g_i = \arg \min_{g \in [-\nu, \nu]} |d_i + g|$ garantisce che, nel punto di minimo w^* , si abbia esattamente $\tilde{\nabla} F(w^*) = 0$ per tutte le coordinate in cui $|d_i| \leq \nu$, mentre per le altre coordinate il gradiente viene reso il più piccolo possibile in valore assoluto.

Mettendo insieme tutti i casi, si ottiene la seguente espressione per il gradiente generalizzato:

$$[\tilde{\nabla} F_S(w)]^i = \begin{cases} \frac{\partial J_S(w)}{\partial w_i} + \nu & \text{se } w_i > 0 \text{ oppure } \left(w_i = 0 \text{ e } \frac{\partial J_S(w)}{\partial w_i} < -\nu \right), \\ \frac{\partial J_S(w)}{\partial w_i} - \nu & \text{se } w_i < 0 \text{ oppure } \left(w_i = 0 \text{ e } \frac{\partial J_S(w)}{\partial w_i} > \nu \right), \\ 0 & \text{se } w_i = 0 \text{ e } -\nu \leq \frac{\partial J_S(w)}{\partial w_i} \leq \nu. \end{cases} \quad (5.43)$$

Nota

La formula (5.43) è la versione multidimensionale del subgradiente di $F_S(w) = J_S(w) + \nu \|w\|_1$. Essa generalizza il caso 1D: quando $w_i \neq 0$, il gradiente è semplicemente la somma del gradiente di J_S e del segno di w_i moltiplicato per ν ; quando $w_i = 0$, si sceglie il subgradiente che punta nella direzione di massima discesa.

5.3.3 Identificazione dell'Active Set e della Faccia Ortante

Poiché la funzione $\nu|w_i|$ ha un angolo (una non derivabilità) proprio in $w_i = 0$, l'algoritmo non può usare la solita regola della derivata zero per decidere se quel punto è il minimo. Deve quindi fare una scelta esplicita: se la pendenza della parte liscia J è più forte del costo ν , allora conviene 'liberare' w_i (spostarla da zero); altrimenti, conviene 'attivarla' (tenerla bloccata a zero per ottenere una soluzione sparsa).

L'obiettivo della fase di predizione è determinare quali variabili sono nulle alla soluzione e identificare la faccia ortante (parte dell'iperpiano) su cui eseguire la minimizzazione. L'idea è quella di compiere un passo infinitesimo lungo la direzione di massima discesa $-\tilde{\nabla} F_{S_k}(w_k)$ e osservare il segno di ogni componente.

Definiamo il vettore $z_k \in \{-1, 0, +1\}^m$ che caratterizza la faccia ortante:

$$z_k^i = \begin{cases} +1 & \text{se } w_k^i > 0 \text{ oppure } \left(w_k^i = 0 \text{ e } \frac{\partial J_{S_k}(w_k)}{\partial w_i} < -\nu \right), \\ -1 & \text{se } w_k^i < 0 \text{ oppure } \left(w_k^i = 0 \text{ e } \frac{\partial J_{S_k}(w_k)}{\partial w_i} > \nu \right), \\ 0 & \text{altrimenti.} \end{cases} \quad (5.44)$$

Il vettore z_k definisce l'ortante

$$\Omega_k = \{d \in \mathbb{R}^m : \text{sign}(d^i) = \text{sign}(z_k^i)\}, \quad (5.45)$$

all'interno del quale la funzione $\|w\|_1$ è differenziabile e il suo gradiente è esattamente z_k . Le variabili con $z_k^i = 0$ costituiscono l'active set:

$$\mathcal{A}_k \triangleq \{i \mid z_k^i = 0\}. \quad (5.46)$$

Tali variabili verranno mantenute a zero durante la fase di minimizzazione nel sottospazio; le rimanenti sono dette libere.

5.3.4 Fase di Minimizzazione nel Sottospazio

Una volta identificata la faccia ortante, si costruisce un modello quadratico della funzione obiettivo ristretto alle variabili libere. Il modello è:

$$\begin{aligned} \min_{d \in \mathbb{R}^m} m_k(d) &\stackrel{\text{def}}{=} F_{\mathcal{S}_k}(w_k) + \tilde{\nabla} F_{\mathcal{S}_k}(w_k)^T d + \frac{1}{2} d^T \nabla^2 J_{\mathcal{H}_k}(w_k) d \\ \text{s.t. } d^i &= 0, \quad i \in \mathcal{A}_k. \end{aligned} \quad (5.47)$$

La minimizzazione viene effettuata con il metodo del gradiente coniugato Hessian free applicato al sistema lineare ridotto. Sia Y_k la matrice $n \times (n - |\mathcal{A}_k|)$ le cui colonne sono i versori delle coordinate libere; allora il sistema da risolvere è:

$$[Y_k^T \nabla^2 J_{\mathcal{H}_k}(w_k) Y_k] d^Y = -Y_k^T \tilde{\nabla} F_{\mathcal{S}_k}(w_k), \quad (5.48)$$

dove $d^Y \in \mathbb{R}^{n-|\mathcal{A}_k|}$ è il vettore delle componenti libere. La direzione di ricerca dell'algoritmo è quindi $d_k = Y_k d^Y$.

5.3.5 Ricerca Lineare Proiettata

Per garantire che il nuovo iterato rimanga all'interno della stessa faccia ortante Ω_k , si utilizza una proiezione ortogonale $P(\cdot)$ su Ω_k :

$$P(w_i) = \begin{cases} w_i & \text{se } \text{sign}(w_i) = \text{sign}(z_k^i), \\ 0 & \text{altrimenti.} \end{cases} \quad (5.49)$$

La ricerca lineare determina il passo α_k come il più grande valore nella sequenza $1, \frac{1}{2}, \frac{1}{4}, \dots$ tale che:

$$F_{\mathcal{S}_k}(P[w_k + \alpha_k d_k]) \leq F_{\mathcal{S}_k}(w_k) + \sigma \tilde{\nabla} F_{\mathcal{S}_k}(w_k)^T (P[w_k + \alpha_k d_k] - w_k) \quad (5.50)$$

(Armijo) dove $\sigma \in (0, 1)$ è una costante prefissata. Il nuovo iterato è $w_{k+1} = P[w_k + \alpha_k d_k]$.

5.3.6 Algoritmo Completo

Algoritmo

Newton-CG per Problemi L_1 -Regolarizzati

Input: w_0 (punto iniziale, tipicamente $w_0 = 0$), $\mathcal{H}_0, \mathcal{S}_0$ con $|\mathcal{H}_0| < |\mathcal{S}_0|$, $\nu > 0$ (parametro di regolarizzazione), $\eta, \sigma \in (0, 1)$, $\max_{cg}$ (limite iterazioni CG).

Per ogni iterazione $k = 0, 1, \dots$ fino a convergenza:

1. Calcola $\tilde{\nabla} F_{\mathcal{S}_k}(w_k)$ tramite (5.43) e determina z_k con (5.44). Definisci l'active set $\mathcal{A}_k$ con (5.46).
2. Minimizzazione nel sottospazio
 - (a) Calcola $\tilde{\nabla} F_{\mathcal{S}_k}(w_k) = \nabla J_{\mathcal{S}_k}(w_k) + \nu z_k$.
 - (b) Applica il gradiente coniugato al sistema (5.48). Termina quando il residuo r_k soddisfa

$$\|r_k\| \leq \eta \|Y_k^T \tilde{\nabla} F_{\mathcal{S}_k}(w_k)\|, \quad (5.51)$$

oppure dopo $\max_{cg}$ iterazioni.

- (c) Poni $d_k = Y_k d^Y$.
3. Ricerca lineare proiettata. Determina il più grande $\alpha_k \in \{1, \frac{1}{2}, \frac{1}{4}, \dots\}$ che soddisfi (5.50). Aggiorna

$$w_{k+1} = P[w_k + \alpha_k d_k].$$

4. Ricampionamento. Scegli $\mathcal{H}_{k+1}, \mathcal{S}_{k+1}$ con $|\mathcal{H}_{k+1}| = |\mathcal{H}_0|$ e $|\mathcal{S}_{k+1}| = |\mathcal{S}_0|$.

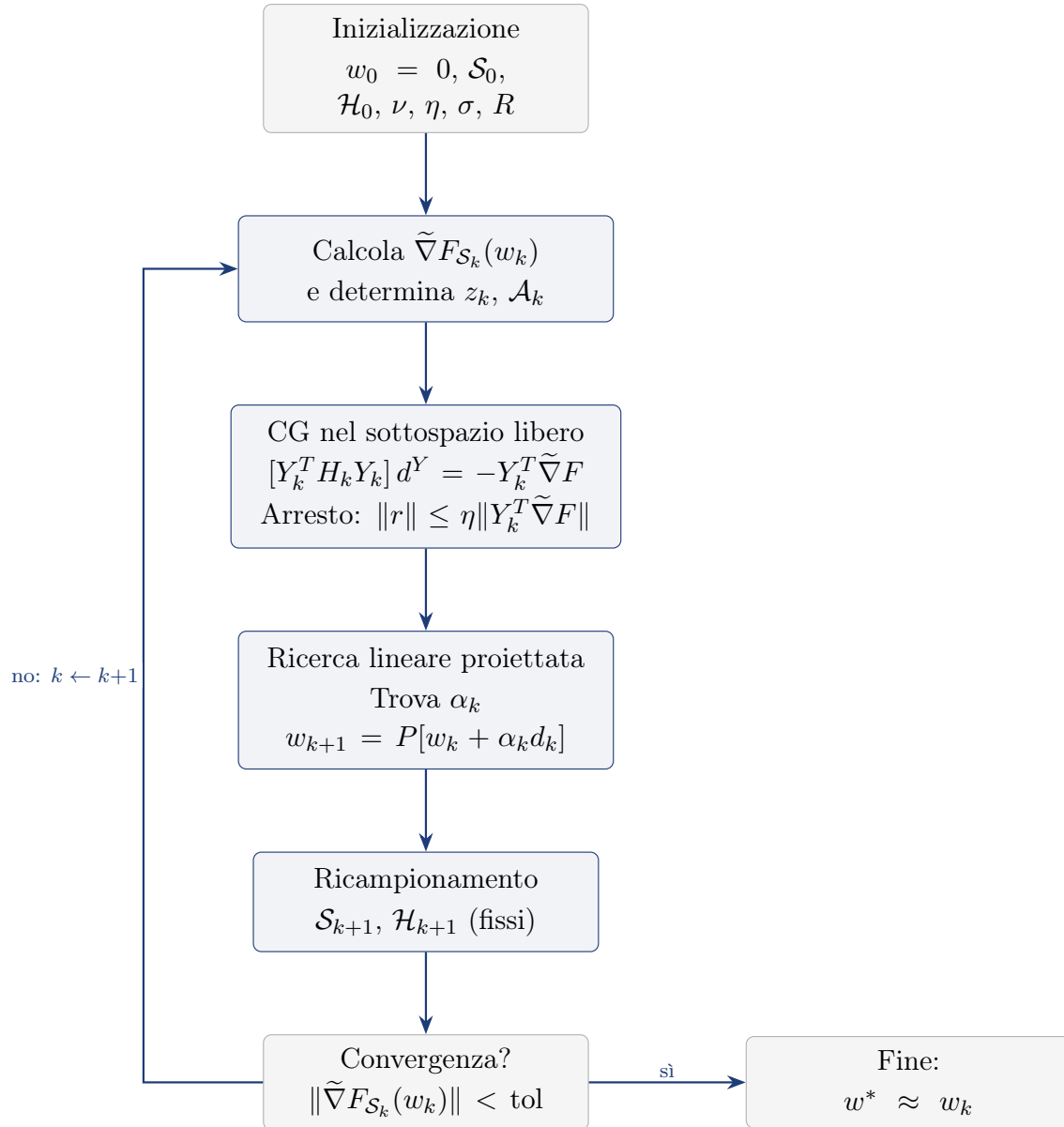


Figura 5.5: Schema del metodo Newton-CG per problemi con regolarizzazione L_1 : il gradiente generalizzato determina l'active set e la faccia ortante, il CG risolve il sistema ridotto sulle variabili libere e la ricerca lineare proiettata mantiene le variabili dell'active set a zero.

Nota

L'algoritmo appena descritto è particolarmente efficace nella produzione di soluzioni sparse: la fase di predizione identifica rapidamente le variabili nulle, mentre la fase di sottospazio accelera la convergenza sulle variabili libere sfruttando l'informazione di curvatura dell'Hessiana sottocampionata.

Metodo	Costo per iterazione	Memoria	Ipotesi aggiuntive	Complessità totale
Gradiente a campione dinamico	$O(m n_k)$	$O(m)$	CCV soddisfatta a ogni iterazione; varianza limitata (5.4)	$O(mL/\lambda\varepsilon)$
Newton-CG con campionamento dinamico	$O(m n_k + m n_h j_k)$	$O(m n_h)$ (sottocampione Hessiana)	Hessiana sdp sul sottospazio; criterio di arresto (5.35)–(5.36)	$O(mL/\lambda\varepsilon)$ iterazioni esterne
Newton-CG con regolarizzazione L_1	Come Newton-CG + costo active set	$O(m)$	Identificazione corretta della faccia ortante (5.41)–(5.46)	Come Newton-CG

Tabella 5.2: Confronto dei tre metodi proposti nel Capitolo 5. n_k è la dimensione del batch corrente, $n_h = R n_k$ quella del sottocampione per l’Hessiana, j_k il numero di iterazioni interne del gradiente coniugato, m il numero di variabili.

6 Esperimenti e Visualizzazione Interattiva

In questo capitolo descriviamo l’ambiente sperimentale utilizzato per validare gli algoritmi del Capitolo 5, l’architettura software dell’applicazione web interattiva e i risultati numerici ottenuti, confrontando il metodo del gradiente a campione dinamico con SGD e batch gradient descent, e Newton-CG con il gradiente dinamico. I codici Python completi sono riportati nell’Appendice B.

6.1 Setup Sperimentale

Gli esperimenti sono condotti con l’applicazione web interattiva descritta nella Sezione 6.2: le funzioni obiettivo e i dataset sono i preset quadratici dell’applicazione, con la costruzione del “dataset centrato” descritta nella Sezione 6.2, che consente di controllare esattamente la soluzione w^* e di misurare l’errore finale $\|w_k - w^*\|$ senza ambiguità.

Le funzioni test utilizzate sono i seguenti preset quadratici dell’applicazione.

- Quadratica ben condizionata ($\kappa \approx 1.1$): $J(w) = (w_1 - 1)^2 + (w_2 + 2)^2 + 0.1 w_1 w_2$ (preset “Quadratica ben condizionata ($\kappa \approx 1.1$)”).
- Quadratica molto mal condizionata ($\kappa \approx 100$): $J(w) = 100 (w_1 - 1)^2 + (w_2 + 2)^2$ (preset “Quadratica molto mal condizionata ($\kappa \approx 100$)”).
- Quadratica con termine incrociato: $J(w) = (w_1 - 1)^2 + (w_2 + 2)^2 + 0.5 (w_1 - 1)(w_2 + 2)$ (preset “Quadratica con termine incrociato”).

L’applicazione offre anche il preset “Quadratica mal condizionata ($\kappa \approx 20$)”, non usato nei test riportati in questo capitolo.

I test sono eseguiti con i parametri di default dell’applicazione: dimensione del dataset $N = 200$, punto iniziale $w_0 = (2, -3)$, seme casuale fissato (seed 42), passo iniziale $\alpha = 0.1$, tolleranza della CCV $\theta = 0.5$, batch iniziale $n_0 = 5$ e un budget di

30 iterazioni; la convergenza è dichiarata quando $\|\nabla J(w_k)\|_2 < 10^{-6}$. Gli algoritmi confrontati sono i tre implementati nell'applicazione – Dynamic GD, Newton-CG e Newton-CG L_1 – e il metodo BB-CCV descritto nell'Appendice E, eseguito sugli stessi preset.

Tutti i valori riportati in questo capitolo sono quelli mostrati dal pannello “Analisi” dell'applicazione: selezionando il preset, l'algoritmo e i parametri sopra indicati e premendo il pulsante “Ricalcola” e poi “Analisi”, i risultati sono riproducibili esattamente a parità di seed.

6.2 Architettura Software

Per rendere concreti i concetti teorici esposti nel Capitolo 5, è stata realizzata un'applicazione web interattiva che implementa i tre algoritmi in un ambiente visivo e modificabile dall'utente. Il codice sorgente è disponibile al seguente repository GitHub:

<https://github.com/Alessandro1040/selezione-dinamica-batch>

L'applicazione esegue il codice Python (funzione obiettivo, gradiente, algoritmo) direttamente nel browser tramite il runtime Python Pyodide, consentendo all'utente di modificare loss e algoritmo e vedere immediatamente il risultato. La visualizzazione delle superfici e dei percorsi di ottimizzazione è affidata a Plotly (libreria JavaScript). L'interfaccia permette di:

- selezionare o scrivere una funzione obiettivo $J(w)$ con gradiente $\nabla J(w)$ e Hessiana $\nabla^2 J(w)$, scegliendo tra preset predefiniti o codice personalizzato;
- scegliere tra i tre algoritmi discussi (gradiente dinamico, Newton-CG con campionamento dinamico, Newton-CG con regolarizzazione L_1), il cui codice Python è visibile e modificabile;
- variare i parametri dell'algoritmo in tempo reale;
- visualizzare il percorso di ottimizzazione sulla superficie di $J(w)$ (in 3D per funzioni 2D, in 2D per funzioni 1D), con possibilità di scorrere le iterazioni, avviare una riproduzione automatica e ruotare la visualizzazione;
- monitorare metriche in tempo reale (valore di $J(w_k)$, norma del gradiente, dimensione del batch);
- eseguire un'analisi di convergenza che mostra l'errore e $J(w_k)$ per ogni iterazione.

Costruzione del dataset sintetico centrato. Nel codice dell'applicazione, la superficie plottata è la funzione obiettivo teorica $J(w)$, definita dall'utente nel preset della loss. Questa è la funzione che si vorrebbe minimizzare: viene usata per disegnare il grafico, per calcolare i valori del percorso e per mostrare le metriche come $J(w_k)$ e $\|\nabla J(w_k)\|$. L'algoritmo però non minimizza direttamente $J(w)$, ma utilizza un dataset sintetico di N funzioni di perdita individuali $\ell(w; i)$, con $i = 1, \dots, N$, costruito a partire da $J(w)$ perturbando i suoi coefficienti; ogni funzione $\ell(w; i)$ rappresenta il contributo del singolo esempio i . A ogni iterazione l'algoritmo seleziona un batch

$\mathcal{S}_k$ di dimensione n_k e calcola il gradiente del batch come $g_k = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla \ell(w_k; i)$; la line search di Armijo usa la loss del batch $J_{\mathcal{S}_k}(w) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \ell(w; i)$. La media su tutti gli N dati è $\hat{J}_N(w) = \frac{1}{N} \sum_{i=1}^N \ell(w; i)$.

Il dataset è costruito in modo che le medie campionarie dei coefficienti delle perturbazioni coincidano esattamente con i coefficienti di $J(w)$; pertanto $\hat{J}_N(w) = J(w)$ per ogni w e $\nabla \hat{J}_N(w) = \nabla J(w)$ per ogni w , indipendentemente da N . Questa proprietà è ottenuta sottraendo a ogni perturbazione la propria media campionaria. Quindi $J(w)$ è la funzione plottata, mentre l'algoritmo minimizza $\hat{J}_N(w)$ (o sue approssimazioni su batch). Nel codice $\hat{J}_N(w)$ coincide esattamente con $J(w)$ per ogni N , a differenza di un problema reale dove la coincidenza si ha solo nel limite $N \rightarrow \infty$. In sintesi, la loss teorica $J(w)$ è usata per il plot e le metriche, la loss campionaria totale $\hat{J}_N(w)$ per la condizione di arresto, la loss del batch $J_{\mathcal{S}_k}(w)$ per la line search, e il gradiente del batch g_k per l'aggiornamento dei pesi.

L'implementazione segue la struttura algoritmica del Capitolo 5, con due estensioni rispetto alla formulazione originale [1]: la CCV estesa al subgradiente per L_1 (nel paper il sample size per il caso L_1 è assunto fisso, e gli autori lasciano l'estensione al caso dinamico come lavoro futuro) e il dataset sintetico centrato descritto sopra. Per il resto, l'implementazione è fedele al paper: l'Hessiana nel Newton-CG è calcolata su un sottocampione di dimensione $|\mathcal{H}_k| = R \cdot |\mathcal{S}_k|$; il CG per il caso L_1 usa la tolleranza fissa η della (5.51); la line search di Wolfe è quella degli algoritmi pratici del paper, mentre i passi fissi compaiono solo nell'analisi teorica di complessità.

6.3 Risultati Numerici

Presentiamo i risultati numerici ottenuti nel setup descritto nella Sezione 6.1, eseguendo gli algoritmi sui preset quadratici dell'applicazione web (Sezione 6.4). Le Tabelle 6.1–6.3 riportano la norma dell'errore $\|w_k - w_*\|_2$ a ogni iterazione k per i quattro algoritmi – Dynamic GD, Newton-CG, Newton-CG L_1 e BB-CCV (Appendice E) – sui tre problemi test: quadratico ben condizionato ($\kappa \approx 1.1$), molto mal condizionato ($\kappa \approx 100$) e con termine incrociato, con i quattro algoritmi affiancati in ordine: (a) Dynamic GD, (b) Newton-CG, (c) Newton-CG L_1 , (d) BB-CCV. I valori sono quelli riportati dal pannello “Analisi” dell'applicazione.

Tabella 6.1: Errore $\|w_k - w_*\|_2$ a ogni iterazione sul problema quadratico ben condizionato ($\kappa \approx 1.1$), per i quattro algoritmi. I valori sono quelli riportati dal pannello *Analisi* dell'applicazione (Sezione 6.4).

(a) Dynamic GD		(b) Newton-CG		(c) Newton-CG L_1		(d) BB-CCV	
k	$\ w_k - w_*\ _2$	k	$\ w_k - w_*\ _2$	k	$\ w_k - w_*\ _2$	k	$\ w_k - w_*\ _2$
0	1.4142×10^0	0	1.4142×10^0	0	1.4142×10^0	0	1.4142×10^0
1	1.1516×10^0	1	1.2758×10^0	1	1.2683×10^0	1	1.1516×10^0
2	9.5519×10^{-1}	2	1.2758×10^0	2	1.1466×10^0	2	3.8420×10^{-1}
3	7.8850×10^{-1}	3	1.1736×10^0	3	1.0501×10^0	3	9.3399×10^{-2}
4	6.7471×10^{-1}	4	1.1736×10^0	4	9.5532×10^{-1}	4	2.0089×10^{-1}
5	5.5214×10^{-1}	5	1.1736×10^0	5	8.4970×10^{-1}	5	1.6584×10^{-1}
6	4.3385×10^{-1}	6	1.1736×10^0	6	7.7334×10^{-1}	6	7.9543×10^{-2}
7	3.8792×10^{-1}	7	1.0613×10^0	7	6.9434×10^{-1}	7	1.0292×10^{-1}
8	3.5366×10^{-1}	8	9.7253×10^{-1}	8	6.3606×10^{-1}	8	1.1777×10^{-1}
9	3.0927×10^{-1}	9	8.7008×10^{-1}	9	5.6016×10^{-1}	9	1.1707×10^{-1}
10	2.5516×10^{-1}	10	7.9049×10^{-1}	10	5.0464×10^{-1}	10	1.1673×10^{-1}
11	2.3129×10^{-1}	11	7.9049×10^{-1}	11	4.5898×10^{-1}	11	1.1662×10^{-1}
12	2.1401×10^{-1}	12	7.1398×10^{-1}	12	4.2560×10^{-1}	12	1.1662×10^{-1}
13	2.0084×10^{-1}	13	7.1398×10^{-1}	13	3.9782×10^{-1}		
14	1.8252×10^{-1}	14	6.6137×10^{-1}	14	3.5221×10^{-1}		
15	1.6601×10^{-1}	15	6.6137×10^{-1}	15	3.1656×10^{-1}		
16	1.6395×10^{-1}	16	6.0964×10^{-1}	16	2.8037×10^{-1}		
17	1.5471×10^{-1}	17	5.5345×10^{-1}	17	2.5757×10^{-1}		
18	1.4730×10^{-1}	18	5.5345×10^{-1}	18	2.3404×10^{-1}		
19	1.4134×10^{-1}	19	5.5345×10^{-1}	19	2.1740×10^{-1}		
20	1.3654×10^{-1}	20	5.1627×10^{-1}	20	2.0263×10^{-1}		
21	1.3268×10^{-1}	21	4.6784×10^{-1}	21	1.8691×10^{-1}		
22	1.2957×10^{-1}	22	4.3254×10^{-1}	22	1.7550×10^{-1}		
23	1.2707×10^{-1}	23	4.3254×10^{-1}	23	1.6345×10^{-1}		
24	1.2505×10^{-1}	24	4.3254×10^{-1}	24	1.5147×10^{-1}		
25	1.2342×10^{-1}	25	4.3254×10^{-1}	25	1.4067×10^{-1}		
26	1.2211×10^{-1}	26	4.3254×10^{-1}	26	1.3109×10^{-1}		
27	1.2105×10^{-1}	27	4.0264×10^{-1}	27	1.2247×10^{-1}		
28	1.2020×10^{-1}	28	4.0264×10^{-1}	28	1.1474×10^{-1}		
29	1.1951×10^{-1}	29	4.0264×10^{-1}	29	1.0786×10^{-1}		
30	1.1895×10^{-1}	30	4.0264×10^{-1}	30	1.0167×10^{-1}		

Tabella 6.2: Errore $\|w_k - w_*\|_2$ a ogni iterazione sul problema quadratico molto mal condizionato ($\kappa \approx 100$), per i quattro algoritmi. I valori sono quelli riportati dal pannello *Analisi* dell'applicazione (Sezione 6.4).

(a) Dynamic GD		(b) Newton-CG		(c) Newton-CG L_1		(d) BB-CCV	
k	$\ w_k - w_*\ _2$	k	$\ w_k - w_*\ _2$	k	$\ w_k - w_*\ _2$	k	$\ w_k - w_*\ _2$
0	1.4142×10^0	0	1.4142×10^0	0	1.4142×10^0	0	1.4142×10^0
1	9.9721×10^{-1}	1	1.2807×10^0	1	1.3506×10^0	1	9.9721×10^{-1}
2	9.8193×10^{-1}	2	1.1526×10^0	2	1.2993×10^0	2	9.8012×10^{-1}
3	9.7324×10^{-1}	3	1.1526×10^0	3	1.2413×10^0	3	9.7212×10^{-1}
4	9.5355×10^{-1}	4	1.0395×10^0	4	1.2035×10^0	4	9.5909×10^{-1}
5	9.4387×10^{-1}	5	9.3153×10^{-1}	5	1.1638×10^0	5	9.5065×10^{-1}
6	9.2830×10^{-1}	6	9.3153×10^{-1}	6	1.1300×10^0	6	9.3418×10^{-1}
7	8.8226×10^{-1}	7	9.3153×10^{-1}	7	1.1031×10^0	7	9.1599×10^{-1}
8	8.6034×10^{-1}	8	8.5161×10^{-1}	8	1.0869×10^0	8	8.9981×10^{-1}
9	8.4937×10^{-1}	9	8.5161×10^{-1}	9	1.0653×10^0	9	8.9001×10^{-1}
10	8.0714×10^{-1}	10	8.5161×10^{-1}	10	1.0466×10^0	10	8.8062×10^{-1}
11	7.9682×10^{-1}	11	7.7216×10^{-1}	11	1.0363×10^0	11	1.0991×10^{-14}
12	7.5723×10^{-1}	12	7.7216×10^{-1}	12	1.0264×10^0		
13	7.4751×10^{-1}	13	6.9289×10^{-1}	13	1.0192×10^0		
14	7.1041×10^{-1}	14	6.9289×10^{-1}	14	1.0119×10^0		
15	7.0126×10^{-1}	15	6.9289×10^{-1}	15	1.0059×10^0		
16	6.6649×10^{-1}	16	6.2258×10^{-1}	16	1.0007×10^0		
17	6.5788×10^{-1}	17	5.5683×10^{-1}	17	9.9489×10^{-1}		
18	6.2529×10^{-1}	18	4.9795×10^{-1}	18	9.9248×10^{-1}		
19	6.1717×10^{-1}	19	4.5157×10^{-1}	19	9.8932×10^{-1}		
20	6.0177×10^{-1}	20	4.0302×10^{-1}	20	9.8703×10^{-1}		
21	5.8679×10^{-1}	21	3.6487×10^{-1}	21	9.8468×10^{-1}		
22	5.7227×10^{-1}	22	3.2788×10^{-1}	22	9.8245×10^{-1}		
23	5.6488×10^{-1}	23	3.0656×10^{-1}	23	9.8032×10^{-1}		
24	5.3689×10^{-1}	24	3.0656×10^{-1}	24	9.7874×10^{-1}		
25	5.2993×10^{-1}	25	2.6912×10^{-1}	25	9.7684×10^{-1}		
26	5.0370×10^{-1}	26	2.4956×10^{-1}	26	9.7511×10^{-1}		
27	4.9714×10^{-1}	27	2.4956×10^{-1}	27	9.7353×10^{-1}		
28	4.8474×10^{-1}	28	2.4956×10^{-1}	28	9.7221×10^{-1}		
29	4.7267×10^{-1}	29	2.2864×10^{-1}	29	9.7081×10^{-1}		
30	4.6668×10^{-1}	30	2.0969×10^{-1}	30	9.6954×10^{-1}		

Tabella 6.3: Errore $\|w_k - w_*\|_2$ a ogni iterazione sul problema quadratico con termine incrociato, per i quattro algoritmi. I valori sono quelli riportati dal pannello *Analisi* dell'applicazione (Sezione 6.4).

(a) Dynamic GD		(b) Newton-CG		(c) Newton-CG L_1		(d) BB-CCV	
k	$\ w_k - w_*\ _2$	k	$\ w_k - w_*\ _2$	k	$\ w_k - w_*\ _2$	k	$\ w_k - w_*\ _2$
0	1.4142×10^0	0	1.4142×10^0	0	1.4142×10^0	0	1.4142×10^0
1	1.1925×10^0	1	1.2661×10^0	1	1.2573×10^0	1	1.1925×10^0
2	1.0131×10^0	2	1.2661×10^0	2	1.1244×10^0	2	2.9864×10^{-1}
3	8.5375×10^{-1}	3	1.1493×10^0	3	1.0125×10^0	3	3.6502×10^{-2}
4	7.4106×10^{-1}	4	1.1493×10^0	4	9.0888×10^{-1}	4	1.3817×10^{-1}
5	6.1967×10^{-1}	5	1.1493×10^0	5	7.9719×10^{-1}	5	1.1954×10^{-1}
6	5.0270×10^{-1}	6	1.1493×10^0	6	7.2330×10^{-1}	6	6.5108×10^{-2}
7	4.3324×10^{-1}	7	1.0340×10^0	7	6.4246×10^{-1}	7	4.4710×10^{-2}
8	3.8973×10^{-1}	8	9.3915×10^{-1}	8	5.7882×10^{-1}	8	1.4514×10^{-2}
9	3.1558×10^{-1}	9	8.2530×10^{-1}	9	4.9223×10^{-1}	9	2.9908×10^{-3}
10	2.5451×10^{-1}	10	7.3866×10^{-1}	10	4.3017×10^{-1}	10	6.4737×10^{-4}
11	2.1774×10^{-1}	11	7.3866×10^{-1}	11	3.7902×10^{-1}	11	4.6638×10^{-4}
12	1.9163×10^{-1}	12	6.5807×10^{-1}	12	3.3589×10^{-1}	12	5.7492×10^{-4}
13	1.5804×10^{-1}	13	6.5807×10^{-1}	13	3.0307×10^{-1}	13	6.0513×10^{-4}
14	1.2980×10^{-1}	14	5.9670×10^{-1}	14	2.7238×10^{-1}	14	6.1389×10^{-4}
15	1.1956×10^{-1}	15	5.9670×10^{-1}	15	2.3455×10^{-1}	15	6.1588×10^{-4}
16	1.0807×10^{-1}	16	5.3249×10^{-1}	16	2.0858×10^{-1}	16	6.1644×10^{-4}
17	9.1075×10^{-2}	17	4.7713×10^{-1}	17	1.8380×10^{-1}		
18	6.9281×10^{-2}	18	4.7713×10^{-1}	18	1.5958×10^{-1}		
19	5.8837×10^{-2}	19	4.2913×10^{-1}	19	1.4076×10^{-1}		
20	5.1543×10^{-2}	20	3.8109×10^{-1}	20	1.1881×10^{-1}		
21	4.2974×10^{-2}	21	3.8109×10^{-1}	21	9.3580×10^{-2}		
22	3.5601×10^{-2}	22	3.5167×10^{-1}	22	7.6246×10^{-2}		
23	2.9857×10^{-2}	23	3.5167×10^{-1}	23	5.9908×10^{-2}		
24	2.5128×10^{-2}	24	3.5167×10^{-1}	24	4.5124×10^{-2}		
25	2.1212×10^{-2}	25	3.5167×10^{-1}	25	3.1758×10^{-2}		
26	1.7954×10^{-2}	26	3.5167×10^{-1}	26	1.9901×10^{-2}		
27	1.5232×10^{-2}	27	3.5167×10^{-1}	27	1.0200×10^{-2}		
28	1.2950×10^{-2}	28	3.5167×10^{-1}	28	7.2958×10^{-3}		
29	1.1032×10^{-2}	29	3.5167×10^{-1}	29	1.3420×10^{-2}		
30	9.4161×10^{-3}	30	3.5167×10^{-1}	30	2.0877×10^{-2}		

Osservazioni comparative. Dai valori delle Tabelle 6.1–6.3 emergono tre indicazioni. (i) Sul problema ben condizionato ($\kappa \approx 1.1$) i più veloci sono Dynamic GD, Newton-CG L_1 e BB-CCV, il più lento Newton-CG ($\approx 4 \times 10^{-1}$): la curvatura quasi isotropa rende un passo scalare ben calibrato in ogni direzione, mentre Newton-CG paga la soluzione del sistema di Newton senza guadagnarne. (ii) Sul problema molto mal condizionato ($\kappa \approx 100$) il numero di condizionamento entra nel tasso di convergenza dei metodi del primo ordine (Sezione 4): Dynamic GD, Newton-CG L_1 e Newton-CG restano bloccati sopra 10^{-1} , mentre BB-CCV raggiunge la precisione macchina in 11 iterazioni, perché il passo di Barzilai–Borwein stima la curvatura dai soli gradienti già calcolati (Appendice E) e si adatta all'anisotropia, dove un passo fisso non può essere insieme abbastanza piccolo nella direzione ripida e abbastanza grande in quella piatta. (iii) Con il termine incrociato BB-CCV è di nuovo il migliore ($\approx 6 \times 10^{-4}$ in 16 iterazioni), seguito da Dynamic GD e Newton-CG L_1 ; Newton-CG resta attorno a 10^{-1} . In sintesi, BB-CCV è il migliore sui problemi considerati, in particolare sui mal condizionati; Dynamic GD basta dove la curvatura è ben bilanciata ma collassa dove non lo è, dove serve l'informazione di curvatura.

6.4 Visualizzazione Interattiva

L'applicazione web descritta nella Sezione 6.2 consente di osservare direttamente i concetti teorici discussi in questa tesi. L'utente può:

- sperimentare l'effetto di θ : selezionando un valore piccolo di θ , la CCV diventa stringente e il batch cresce presto; con θ grande, il batch resta piccolo più a lungo e l'algoritmo procede più rapidamente, ma con un errore finale più alto;
- osservare la dinamica del batch: il grafico della dimensione del batch in funzione dell'iterazione mostra i “gradini” di crescita, che si verificano esattamente quando la varianza stimata supera la soglia;
- confrontare i tre algoritmi sulla stessa funzione obiettivo, variando in tempo reale i parametri (batch iniziale, rapporto R , passo, regolarizzazione ν), e visualizzando i percorsi sovrapposti sulla superficie di $J(w)$;
- studiare la convergenza: la tabella delle iterazioni riporta $J(w_k)$ e l'errore, permettendo di verificare empiricamente il tasso di convergenza lineare del metodo dinamico;
- modificare il codice: il codice Python di ogni algoritmo è visibile e modificabile nel browser, consentendo di introdurre varianti e di verificarne l'effetto senza installare nulla.

Il codice completo degli algoritmi eseguiti nell'applicazione è riportato nell'Appendice B; le istruzioni per l'esecuzione e per l'avvio dell'applicazione sono nell'Appendice C.

6.5 Validazione su un benchmark musicale: riconoscimento della nota con NSynth

I test delle Sezioni precedenti usano i preset quadratici sintetici dell'applicazione. Per verificare i metodi su un problema di apprendimento reale, li applichiamo al riconoscimento della *nota* suonata (pitch class) sul dataset **NSynth** [13]: un benchmark di riferimento per la ricerca audio, contenente circa 306 000 note monofoniche di quattro secondi (16 kHz) generate da 1006 strumenti di biblioteche campione commerciali, ciascuna annotata con la nota e la famiglia strumentale di appartenenza.

Per contenere il costo del download (lo split di addestramento completo occupa circa 24 GB), usiamo lo split *validation* (12 678 clip) come insieme di addestramento e lo split *test* (4 096 clip) come insieme di valutazione. Per costruzione del dataset gli strumenti dei due split sono disgiunti: il test misura quindi la generalizzazione a strumenti mai incontrati durante l'addestramento.

Features audio. Da ogni clip estraiamo il *chromagramma*: le dodici componenti cromatiche, cioè l'energia in ciascuna delle 12 classi di altezza (C , $C\sharp$, D , $\dots$, B), indipendenti dall'ottava, e ne prendiamo media e deviazione standard nel tempo, ottenendo 24 features per clip. Il chromagramma è una rappresentazione musicalmente significativa e, essendo per costruzione indipendente dall'ottava, è particolarmente adatto a questo compito.

Problema di apprendimento. Il compito è la classificazione multinomiale in dodici classi di altezza mediante regressione logistica (softmax). La perdita empirica è

$$J(w) = -\frac{1}{N} \sum_{i=1}^N \log p_{y_i}(x_i; w) + \frac{\lambda}{2} \|W\|_F^2,$$

con $p_c(x; w) \propto \exp(e_c^\top Wx)$, dove $W \in \mathbb{R}^{12 \times 25}$ (24 features + bias) e $w = \text{vec}(W)$: 300 parametri in tutto. I metodi a gradiente e Newton usano la regolarizzazione L_2 con $\lambda = 10^{-4}$; il metodo Newton-CG L_1 risolve la versione L_1 -regolarizzata con $\nu = 10^{-3}$ (Sezione 5.3). I parametri degli algoritmi sono quelli della Sezione 6.1: $\theta = 0.5$, $n_0 = 64$, $\alpha = 1$, budget di 300 iterazioni, $R = 0.1$ e seed 42; la dimensione del batch è selezionata dinamicamente dalla CCV.

Il modello è una regressione logistica multinomiale: a ogni clip x (vettore di 24 features) assegna a ogni nota c un punteggio

$$s_c(x) = w_{c,1}x_1 + \dots + w_{c,24}x_{24} + b_c,$$

combinazione lineare delle features, poi convertito in probabilità dalla softmax. È “lineare” perché la decisione è una somma pesata degli input, senza interazioni tra le features (in due dimensioni due classi sarebbero separate da una retta); l’unica non-linearità è la softmax, che serve solo a produrre probabilità. La scelta è deliberata: la loss è convessa e liscia, quindi la teoria della tesi (CCV, convergenza, complessità) si applica direttamente, a differenza delle reti profonde su spettrogrammi (loss non convessa, milioni di parametri). Rispetto a una rete profonda il modello è più semplice: input di 24 statistiche aggregate invece dello spettrogramma e decisione lineare senza features apprese.

Il codice del notebook `nsynth_nota_riproduzione` è organizzato in un piccolo numero di **funzioni** che si richiamano tra loro: la Figura 6.1 ne mostra la mappa completa, mentre le Figure 6.2–6.7 mostrano il dettaglio di ciascuna, (ingressi $\rightarrow$ uscite) e le formule esatte implementate. I nomi coincidono con quelli del notebook, così gli schemi si confrontano direttamente con il codice.

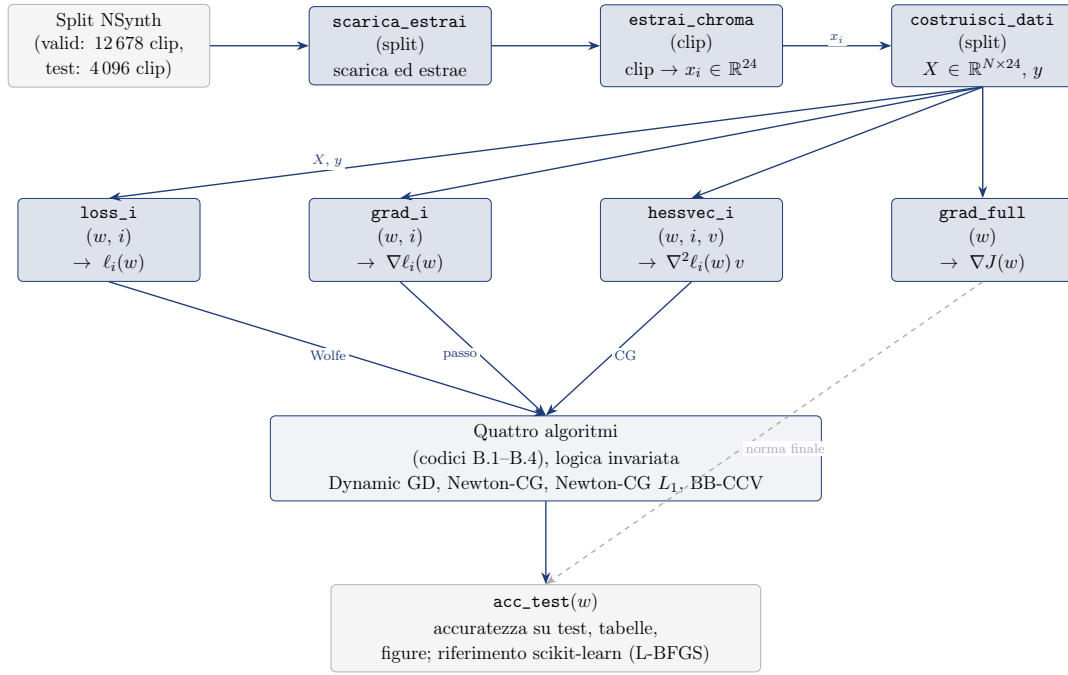


Figura 6.1: Mappa delle funzioni del notebook `nsynth_nota_riproduzione`. Dalle clip di uno split si estraggono i vettori x_i (Figura 6.2), raccolti in X con le etichette delle note da `costruisci_dati`; le quattro funzioni del nucleo forniscono agli algoritmi dei codici B.1–B.4 proprio gli oggetti che essi richiedono: la loss per la line search di Wolfe, il gradiente per il passo, il prodotto Hessiano–vettore per il gradiente coniugato; `grad_full` e `acc_test` servono alle metriche di valutazione. Freccie continue: flusso dei dati; frecce tratteggiate: chiamate tra funzioni.

Estrazione delle features (audio $\rightarrow$ vettore). La funzione `estrai_chroma` converte una clip in un vettore $x_i \in \mathbb{R}^{24}$ di statistiche cromatiche: calcola il chromagramma, cioè l’energia in ciascuna delle 12 classi di altezza in ogni frame temporale, e lo riassume nel tempo con media e deviazione standard (Figura 6.2). Ogni componente misura quindi quanta energia porta una determinata classe di nota, mediata (o la sua variabilità) sull’intera clip.

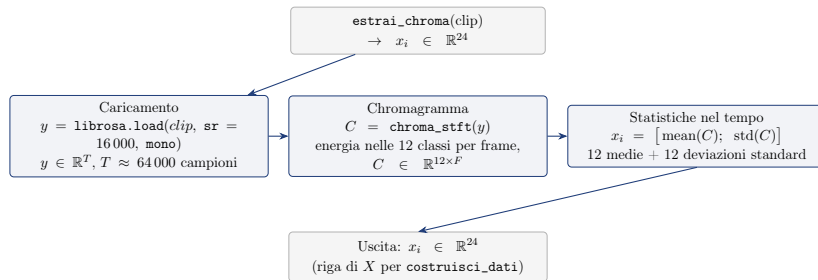


Figura 6.2: Funzione `estrai_chroma`: da clip audio a vettore di 24 statistiche cromatiche. Il chromagramma C misura l’energia in ciascuna delle 12 classi di altezza in ogni frame temporale; media e deviazione standard nel tempo producono il vettore x_i .

Costruzione del dataset. La funzione `costruisci_dati` passa le clip di uno split a `estrai_chroma` e costruisce la matrice X e il vettore delle etichette y : per ogni

clip l'etichetta è la *pitch class* p mod 12 della nota annotata ($0 = C, \dots, 11 = B$). Le colonne sono poi standardizzate con media e deviazione standard calcolate sul solo training set, applicando le stesse statistiche a test (Figura 6.3).

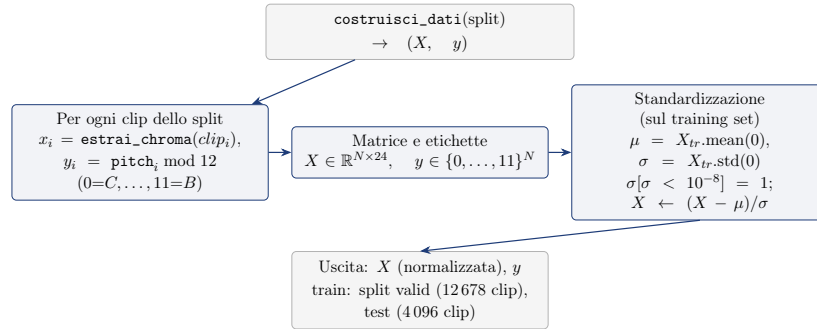


Figura 6.3: Funzione `costruisci_dati`: costruisce la matrice delle features e le etichette della pitch class per uno split, poi le normalizza con media e deviazione standard calcolate sul solo training set (le stesse statistiche sono applicate a test).

Le funzioni per-esempio. I codici B.1–B.4 operano per-esempio: a ogni iterazione richiamano in loop `loss_i`, `grad_i` e `hessvec_i` sul sottocampione corrente, con w fissato. Le tre funzioni condividono una cache sui pesi (le quantità che dipendono solo da w sono ricalcolate una volta sola). Il modello è una regressione logistica multinomiale: la Figura 6.4 mostra la loss di un singolo esempio, la Figura 6.5 il suo gradiente e la Figura 6.6 il prodotto Hessiano–vettore.

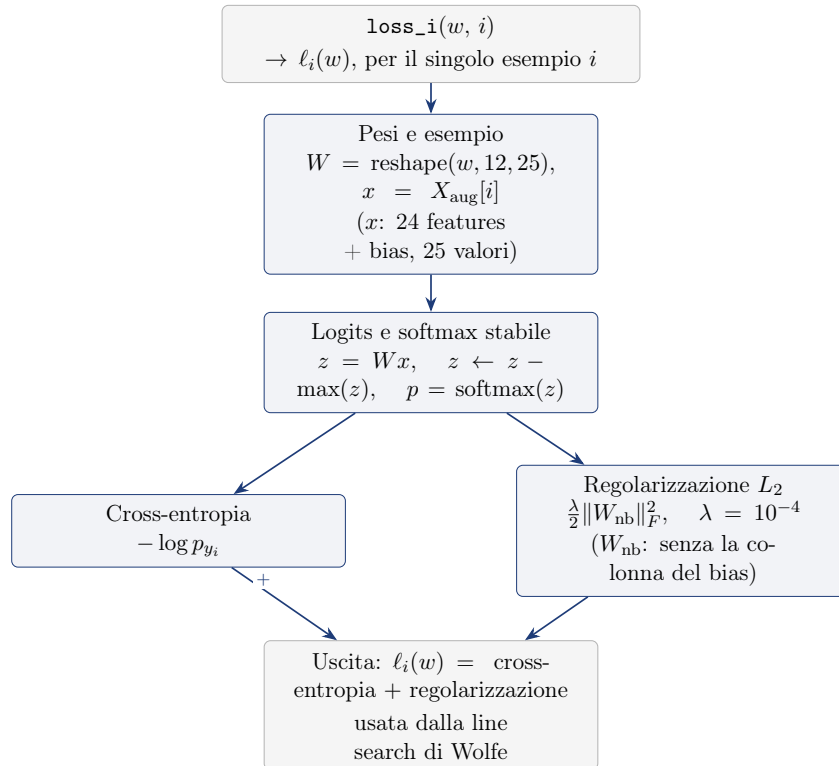


Figura 6.4: Funzione `loss_i`: logits della regressione logistica multinomiale, softmax stabile (shift del massimo), cross-entropia per l'esempio i e regolarizzazione L_2 sui pesi; l'uscita è la somma dei due contributi.

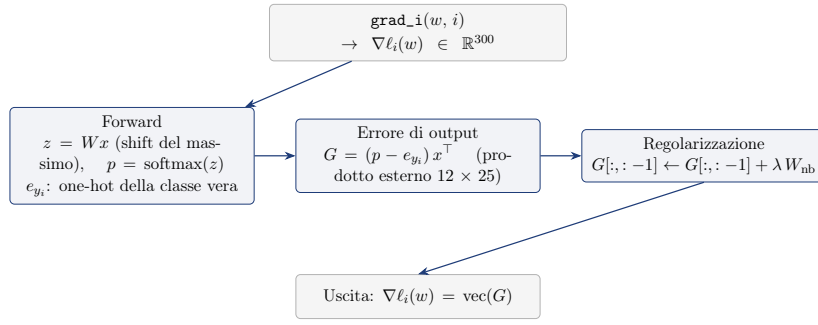


Figura 6.5: Funzione `grad_i`: gradiente per-esempio della regressione logistica. L'errore $p - e_{y_i}$ tra le probabilità e la codifica one-hot della classe vera, esteriorizzato con l'input x , dà la matrice 12×25 ; la colonna del bias (l'ultima) non riceve regolarizzazione.

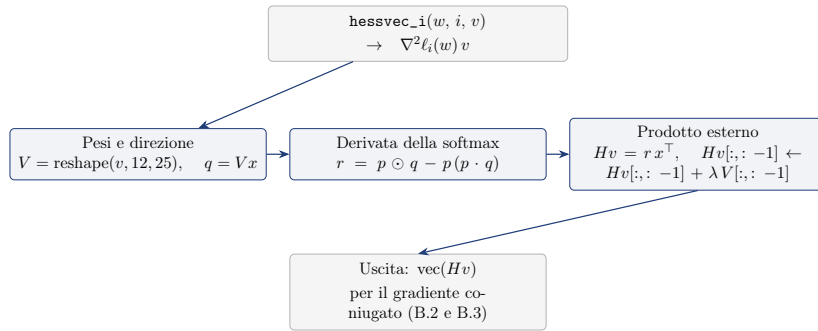


Figura 6.6: Funzione `hessvec_i`: prodotto Hessiano-vettore per-esempio. L'Hessiana della regressione logistica applicata a v richiede la derivata della softmax lungo la direzione $q = Vx$; la regolarizzazione L_2 contribuisce solo sulle colonne dei pesi.

Gradiente sul training set completo. La funzione `grad_full` calcola in forma vettorizzata il gradiente di J su tutto il training set, con la stessa formula di `grad_i` (Figura 6.7). *Vettorizzata* significa che il calcolo non è ripetuto in un ciclo sui singoli esempi (una chiamata a `grad_i` per ciascuna delle N clip), ma eseguito una sola volta su matrici che raccolgono l'intero dataset: un unico prodotto $Z = X_{\text{aug}} W^\top$ dà i logits di tutte le clip, la softmax è applicata per righe e la media $(P - Y)^\top X_{\text{aug}} / N$ somma i contributi in forma matriciale. Il risultato è matematicamente identico alla somma dei contributi individuali, ma le operazioni vettoriali di NumPy delegano i prodotti a kernel di algebra lineare ottimizzati e risultano molto più veloci. La funzione serve per le metriche di valutazione (norma del gradiente finale) e per il subgradiente del metodo L_1 .

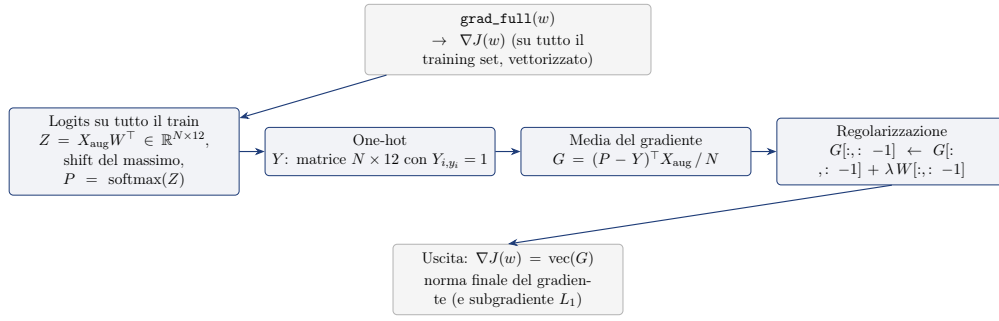


Figura 6.7: Funzione **grad_full**: gradiente di J su tutto il training set in forma matriciale. Le probabilità di tutte le N clip sono calcolate in un unico prodotto; la media degli errori $(P - Y)$ pesati con le features dà il gradiente, usato per la norma finale e per il subgradiente del metodo L_1 .

Abbiamo applicato gli stessi metodi anche al riconoscimento della famiglia strumentale (10 classi, features di mel), dove il modello lineare si ferma a circa il 59% di accuratezza contro il 20.8% della classe maggioritaria: il limite è nella rappresentazione, non negli algoritmi di ottimizzazione. Riportiamo qui il compito del pitch class perché più pulito e direttamente confrontabile con il solutore di riferimento.

I risultati sono riportati nella Tabella 6.4 e nelle Figure 6.8–6.9.

Metodo	Acc. test	$\ \nabla J(w)\ _2$	Batch finale	Tempo (s)	Coeff. non nulli
Dynamic GD	88.0%	6.8×10^{-2}	5 297	7.3	—
Newton-CG	91.4%	2.8×10^{-6}	12 678	6.3	—
Newton-CG L_1	89.8%	8.5×10^{-7}	12 678	1.1	146/300
BB-CCV	90.8%	3.1×10^{-3}	12 678	2.4	—
Riferimento: scikit-learn (L-BFGS) = 91.5%					

Tabella 6.4: Riconoscimento della nota (12 classi di altezza) su NSynth con features chroma (24 dimensioni): accuratezza di test dopo 300 iterazioni di budget e norma del gradiente finale. Newton-CG L_1 raggiunge il criterio di arresto $\|\nabla J\|_2 < 10^{-6}$.

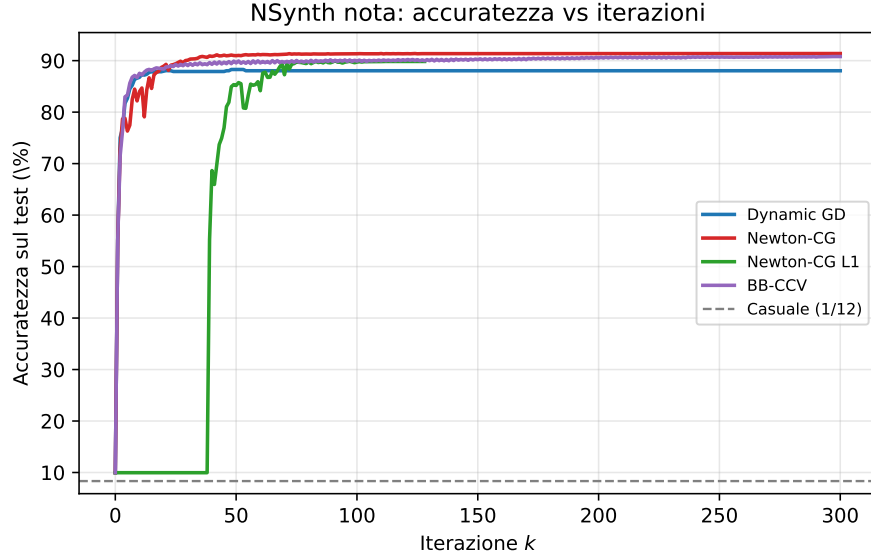


Figura 6.8: Accuratezza sul test NSynth per il riconoscimento della nota in funzione delle iterazioni k . La linea tratteggiata indica la predizione casuale ($1/12 \approx 8.3\%$).

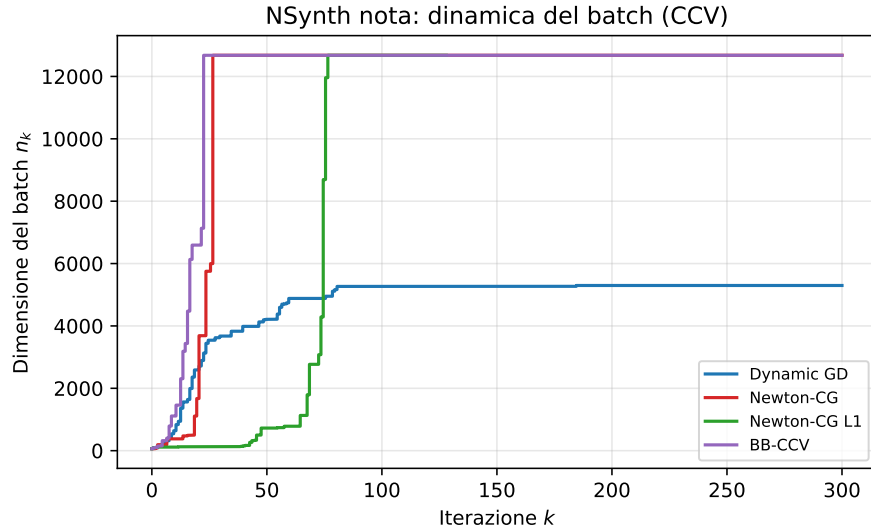


Figura 6.9: Dimensione del batch n_k per il riconoscimento della nota: anche su questo compito la CCV aumenta il campione man mano che il gradiente si riduce.

Tutti i metodi superano di gran lunga la predizione casuale (8.3%): Dynamic GD raggiunge l'88.0%, BB-CCV il 90.8% e Newton-CG il 91.4%, in linea con il solutore di riferimento (L-BFGS di scikit-learn, 91.5%): il metodo del secondo ordine converge quindi allo stesso ottimo del solutore standard, con una differenza inferiore a un decimo di punto percentuale. Il metodo Newton-CG L_1 ($\nu = 10^{-3}$) raggiunge l'89.8% e termina entro il criterio di arresto in 128 iterazioni, azzerando il 51% dei coefficienti (146 restano non nulli su 300): la regolarizzazione L_1 seleziona le componenti cromatiche rilevanti per il riconoscimento della nota. Anche su questo compito la CCV aumenta la dimensione del batch man mano che il gradiente si riduce (Figura 6.9). Il solo Dynamic GD fa eccezione: la norma del suo gradiente

ristagna a $\approx 6.8 \times 10^{-2}$ (non scende mai sotto la tolleranza di arresto) e perciò la CCV si stabilizza a $n_k = 5\,297$ invece di spingere il campione fino a N . In sintesi, gli stessi algoritmi validati sui problemi sintetici raggiungono un'accuratezza elevata ($\approx 91\%$) sul compito musicale del riconoscimento della nota, in linea con il solutore di riferimento.

Riproducibilità. Tutti gli esperimenti di questa sezione (estrazione delle features, esecuzione dei quattro metodi, figure e tabelle) sono riproducibili con il notebook Colab `nsynth_nota_riproduzione`, https://colab.research.google.com/github/Alessandro1040/selezione-dinamica-batch/blob/main/tesi/nsynth/nsynth_nota_riproduzione.ipynb; il codice è anche disponibile nel repository GitHub (<https://github.com/Alessandro1040/selezione-dinamica-batch>), nella cartella `tesi/nsynth/`.

Precisione sui valori numerici. I dati riportati in questa sezione (accuratezze, norme del gradiente, dimensioni del batch e tempi) si riferiscono alla run di riferimento eseguita sul computer dell'autore, non a un'esecuzione in Colab. Il notebook Colab riproduce lo stesso codice con lo stesso seme casuale (seed 42), ma la riproducibilità non è esatta: gli algoritmi sono stocastici e i calcoli in virgola mobile dipendono dall'ambiente di esecuzione (versione di NumPy e libreria BLAS sottostante, OpenBLAS in Colab contro Accelerate su macOS), per cui piccole differenze di arrotondamento, accumulate sulle 300 iterazioni, producono valori finali leggermente diversi. Le differenze sono dell'ordine di qualche decimo di punto percentuale e non cambiano le conclusioni; i tempi di esecuzione in Colab sono invece molto più lunghi (minuti, contro pochi secondi sulla macchina locale).

6.6 Estensione oltre le ipotesi: implementazione vettorizzata per la rete neurale

Per verificare come si comportano i quattro metodi quando la funzione obiettivo **non** è convessa, gli stessi algoritmi dei codici B.1–B.4 sono stati applicati a una **rete neurale**, una famiglia di modelli in cui la loss dipende in modo non lineare dai parametri e non è convessa. Le regole degli algoritmi (line search di Wolfe, controllo della varianza CCV, gradiente coniugato, proiezione sull'ortante) restano *invariate*; cambia solo il modo in cui si calcolano gradiente, loss e prodotto Hessiano–vettore. Nelle versioni originali questi oggetti si ottengono sommando, per ogni esempio del batch, il contributo individuale: con una rete di $\approx 234\,000$ parametri e $N = 12\,678$ esempi questo costo (e la memoria necessaria) rende l'esecuzione impraticabile su Colab. Si usano quindi versioni *batch*, dette *vettorizzate*: applicano le stesse formule a matrici che rappresentano l'intero batch in una sola volta, i prodotti $X_b W_1$, $a_1 W_2$ e $a_2 W_3$ del forward e i prodotti matriciali del passaggio all'indietro, invece di ripetere il calcolo in un ciclo per ogni esempio (come definito nella Sezione 6.5). La corrispondenza con le versioni per-esempio è verificata numericamente (errori dell'ordine di 10^{-15}).

L'implementazione è organizzata in un piccolo numero di **funzioni** che si richiamano tra loro: la Figura 6.10 ne mostra la mappa completa, mentre le Figure 6.11–6.16 mostrano il dettaglio di ciascuna, (ingressi $\rightarrow$ uscite) e le formule esatte implementate. I nomi coincidono con quelli del notebook Colab `nsynth_net_riproduzione`, così gli schemi si confrontano direttamente con il codice.

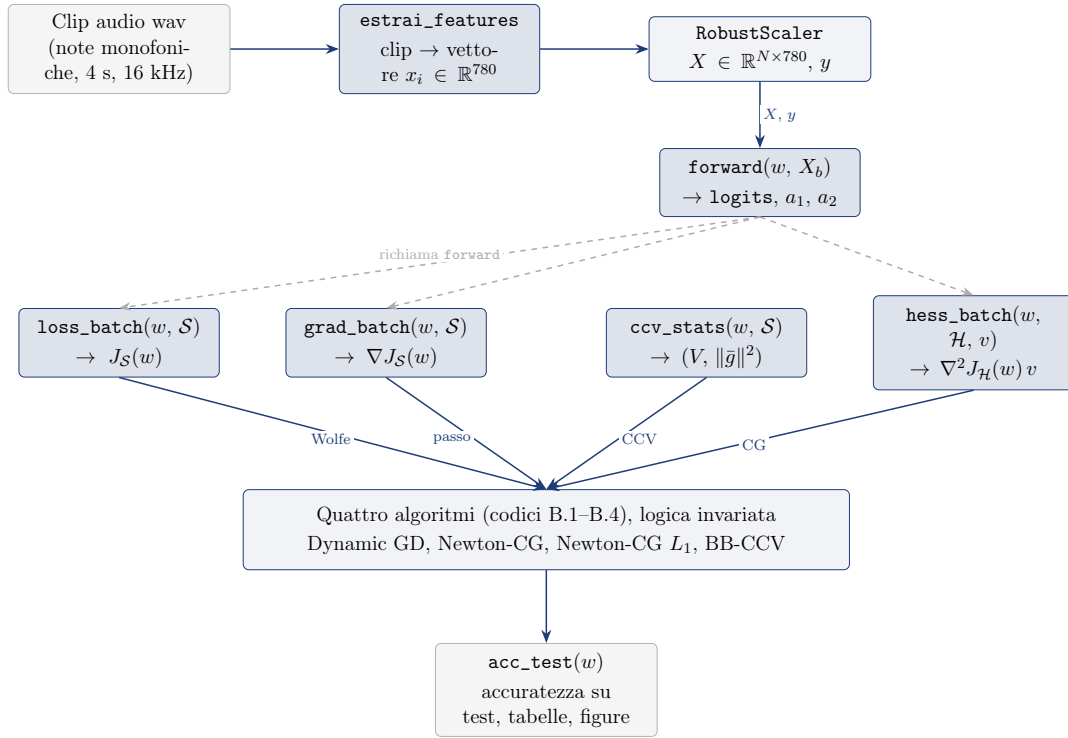


Figura 6.10: Mappa delle funzioni dell’implementazione vettorizzata. Dalle clip si estraggono i vettori x_i (Figura 6.11), normalizzati nella matrice X ; **forward** calcola le attivazioni della rete e le quattro funzioni che la richiamano forniscono agli algoritmi dei codici B.1–B.4 proprio gli oggetti che essi richiedono: la loss per la line search di Wolfe, il gradiente per il passo, la varianza per la verifica CCV e il prodotto Hessiano–vettore per il gradiente coniugato; **acc_test** valuta infine la soluzione sul test. Freccie continue: flusso dei dati; frecce tratteggiate: chiamate tra funzioni.

Estrazione delle features (audio $\rightarrow$ vettore). Una clip è un segnale $y \in \mathbb{R}^T$ di 64 000 campioni a 16 kHz, cioè un vettore di pressioni d’aria campionate, da cui nessun algoritmo lavora direttamente. La funzione **estrai_features** lo converte in un vettore $x_i \in \mathbb{R}^{780}$ di statistiche che riassumono *come suona* la clip: separa prima il segnale in una parte armonica y_h (il tono) e una percussiva y_p (gli attacchi), poi calcola su y_h sei famiglie di descrittori (timbro, altezza, forma spettrale, dinamica) riassumendo ogni descrittore nel tempo con media e deviazione standard (Figura 6.11); i valori sono infine normalizzati con un **RobustScaler**. Un programmatore non deve conoscere l’audio: basta sapere che x_i è l’input della rete e che ogni sua componente misura una caratteristica statistica del suono.

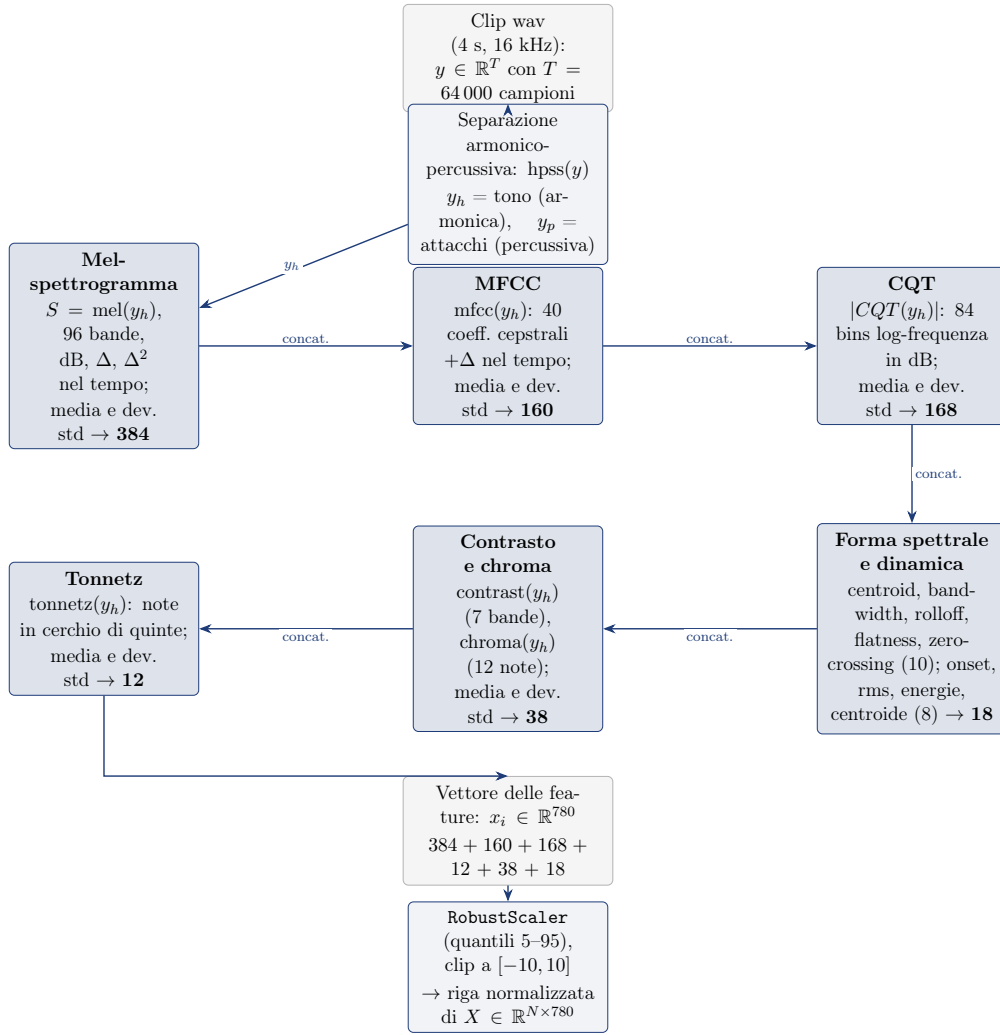


Figura 6.11: `estrai_features`: da clip audio a vettore di feature. Il segnale y è separato in armonica (y_h) e percussiva (y_p); sei famiglie di descrittori calcolati su y_h (tonnetz, spettrale e dinamica anche su y) producono medie e deviazioni standard nel tempo, concatenate nel vettore $x_i \in \mathbb{R}^{780}$ (le cifre indicano le dimensioni di ogni blocco); infine `RobustScaler` normalizza le colonne sui quantili 5–95 e i valori sono limitati a $[-10, 10]$. Le frecce etichettate *concat.* indicano che i blocchi, calcolati indipendentemente, sono concatenati nell'ordine mostrato.

Forward della rete. La rete è una funzione f_w che trasforma il batch $X_b \in \mathbb{R}^{m \times 780}$ in un vettore di punteggi per le 10 famiglie strumentali, alternando trasformazioni affini e attivazioni non lineari tanh: z_l è l'ingresso dello strato l , $a_l = \tanh(z_l)$ la sua uscita, e l'ultimo strato produce i *logits*, che la softmax convertirà in probabilità. I parametri sono raccolti nel vettore $w = [W_1; b_1; W_2; b_2; W_3; b_3]$ di $\approx 234\,000$ valori (Figura 6.12); la funzione restituisce anche le attivazioni a_1, a_2 , che servono alla backpropagation.

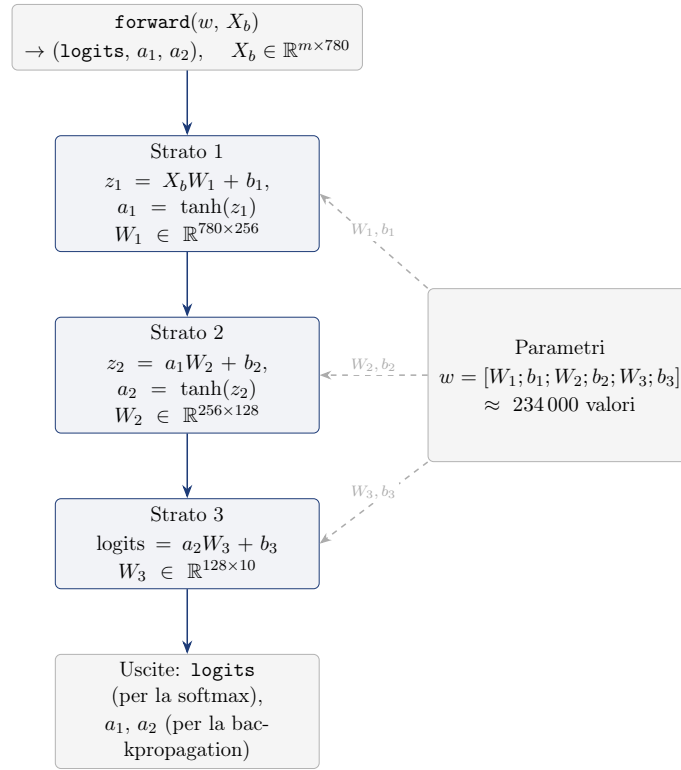


Figura 6.12: Funzione **forward** (architettura della rete): tre trasformazioni affini intervallate dall'attivazione $\tanh$, con le dimensioni delle matrici; w è l'intero vettore dei parametri, da cui ogni strato legge i propri pesi W_l e bias b_l . I logits sono le uscite grezze dell'ultimo strato.

Loss sul batch. La funzione **loss_batch** restituisce la perdita media sul sotto-campione $\mathcal{S}$, $J_{\mathcal{S}}(w) = -\frac{1}{m} \sum_{i \in \mathcal{S}} \log p_{y_i} + \frac{\lambda}{2} \sum_{l=1}^3 \|W_l\|_F^2$: le probabilità p si ottengono con la softmax dai logits della **forward**, e la regolarizzazione L_2 (con $\lambda = 10^{-4}$) è applicata solo alle matrici dei pesi W_l , non ai bias. La Figura 6.13 riassume i passaggi.

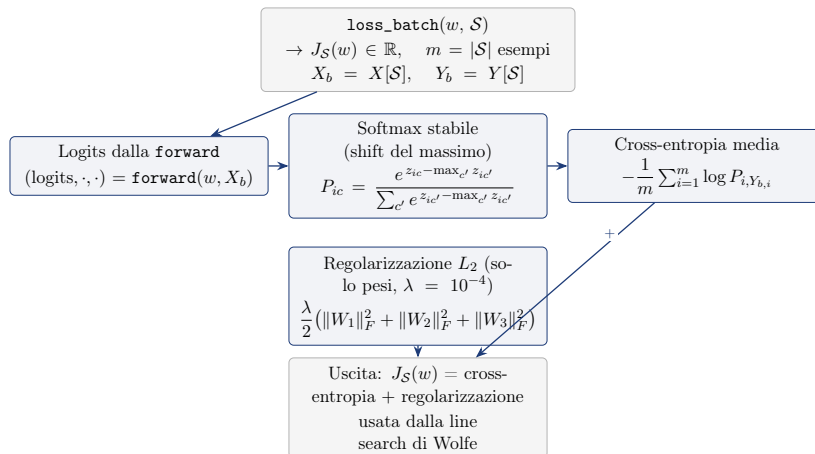


Figura 6.13: Funzione **loss_batch**: logits dalla **forward**, softmax stabile, cross-entropia media sul batch e regolarizzazione L_2 sui pesi; l'uscita è la somma dei due contributi.

Gradiente del batch. Per minimizzare la loss serve il gradiente rispetto ai parametri. Lo si calcola con la **backpropagation**: dopo il forward si definisce l'errore $\delta = (P - Y_b)/m$ tra le probabilità P e la codifica one-hot Y_b delle classi vere, e lo si propaga all'indietro attraverso i tre strati usando la derivata di tanh, che vale $1 - a^2$; a ogni peso si somma il contributo di regolarizzazione λW_l . Il risultato è la media dei gradienti individuali, $\nabla J_S(w) = \frac{1}{m} \sum_{i \in S} \nabla \ell(w; i)$, calcolata però in un solo passaggio matriciale sull'intero batch (Figura 6.14).

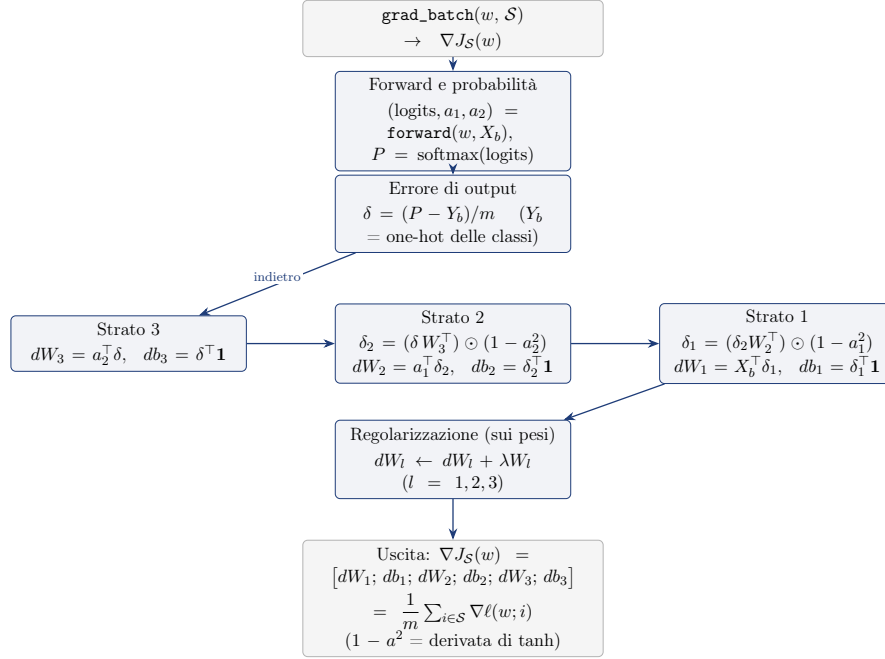


Figura 6.14: Funzione `grad_batch`: backpropagation vettorizzata. L'errore δ tra probabilità e one-hot è propagato all'indietro con la derivata di tanh ($1 - a^2$); a ogni matrice dei pesi si somma λW_l . Il risultato coincide con la media dei gradienti per-esempio del codice B.1.

Varianza per-esempio per la CCV. La condizione di controllo della varianza (codice B.1) richiede la varianza campionaria dei gradienti individuali sul batch corrente. In forma esatta essa vale $V = \frac{\sum_i \|g_i\|^2 - n_k \|\bar{g}\|^2}{n_k - 1}$: per calcolarla non serve materializzare la matrice ($n_k \times D$) dei gradienti individuali (che richiederebbe $n_k \cdot D$ valori, troppi per la memoria di Colab), basta accumulare la somma dei quadrati delle norme $\sum_i \|g_i\|^2$ calcolando i gradienti a blocchi di 32 esempi. Il valore ottenuto è *identico* a quello del codice B.1. La Figura 6.15 riassume la funzione `ccv_stats` e la sua dipendenza da `grad_batch` e `per_example_sq_norms`.

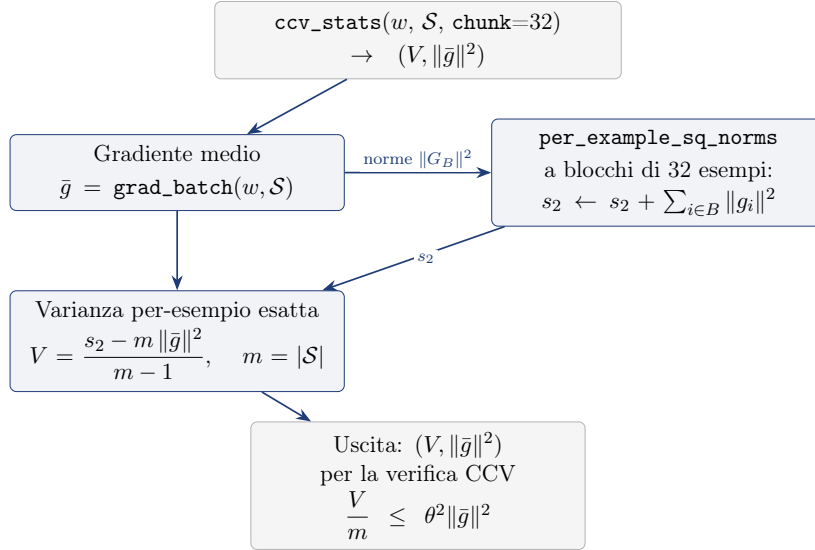


Figura 6.15: Funzione `ccv_stats`: gradiente medio dal `grad_batch`, somma dei quadrati delle norme per-esempio calcolata a blocchi di 32 esempi (per non materializzare la matrice $n_k \times D$) e combinazione nella formula esatta della varianza campionaria; l'uscita alimenta la verifica CCV del codice B.1.

Prodotto Hessiano–vettore. Il metodo Newton-CG richiede, a ogni iterazione del gradiente coniugato, il prodotto tra l'Hessiana della loss media sul sottocampione $\mathcal{H}_k$ e un vettore v . Poiché l'Hessiana della media è la media delle Hessiane, questo prodotto coincide con la media dei prodotti per-esempio; `hess_batch` lo calcola con un'unica chiamata di differenziazione automatica sulla funzione $f(w') = J_{\mathcal{H}_k}(w')$ (Figura 6.16). Per contenere il costo si pone $\gamma = 0$ nel criterio di arresto del CG (la stima della varianza degli Hessiani per-esempio sarebbe proibitiva) e si usa $\alpha = 1$ come passo iniziale delle line search, come nel resto della tesi.

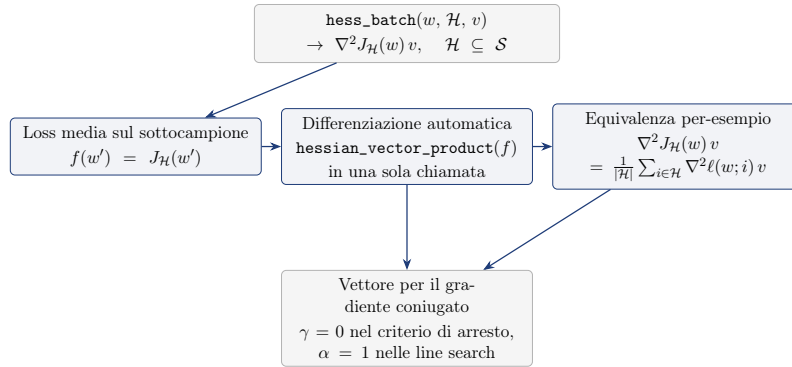


Figura 6.16: Funzione `hess_batch`: il prodotto Hessiano–vettore sul sottocampione $\mathcal{H}$ è calcolato definendo la loss media f e chiamando `hessian_vector_product` una sola volta; il valore coincide con la media dei prodotti per-esempio.

Il notebook `nsynth_net_riproduzione`, disponibile nella cartella `tesi/nsynth/` del repository, implementa fedelmente le funzioni qui descritte: la cella di validazione interna verifica numericamente, su un piccolo batch, che le funzioni batch coincidano con le versioni per-esempio (errori $< 10^{-8}$), garantendo che l'uso delle versioni vettorizzate non altera i risultati degli algoritmi.

Risultati dell'esperimento. I quattro metodi sono stati applicati alla rete con 200 iterazioni, seed 42, batch iniziale 128 e soglia CCV $\theta = 0,3$; per contenere il costo delle iterazioni la dimensione del batch è stata limitata a $n_k \leq 2048$ (senza limite la CCV tenderebbe a far crescere il batch verso l'intero training set, $N = 12\,678$). La Tabella 6.5 riporta per ogni metodo l'accuratezza sul test set (4096 clip), la norma del gradiente finale, la dimensione del batch finale, il tempo di addestramento e, per il metodo con regolarizzazione L_1 , il numero di coefficienti non nulli. La Figura 6.17 mostra l'accuratezza di test al variare dell'iterazione k , mentre la Figura 6.18 mostra la dinamica della dimensione del batch n_k scelta dal controllo CCV.

Tabella 6.5: Rete neurale su NSynth: risultati dei quattro metodi (200 iterazioni, cap del batch a 2048, seed 42; tempi su Apple M5).

Metodo	Acc. test	$\ \nabla J(w)\ _2$	Batch fin.	Tempo (s)	Coeff. $\neq 0$
Dynamic GD	96.9%	4.1×10^{-1}	2048	148.8	—
Newton-CG	99.2%	3.0×10^{-2}	2048	210.2	—
Newton-CG L_1	97.3%	7.0×10^{-1}	2048	196.9	58 412/234 122
BB-CCV	96.2%	8.4×10^{-2}	2048	216.9	—

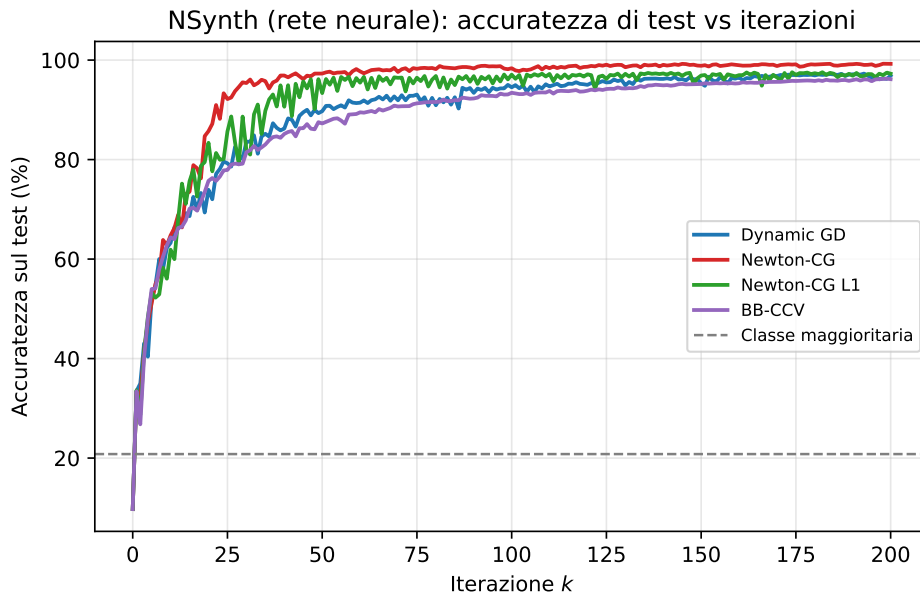


Figura 6.17: NSynth (rete neurale): accuratezza sul test set in funzione delle iterazioni per i quattro metodi; la linea tratteggiata indica la frequenza della classe maggioritaria (20.8%).

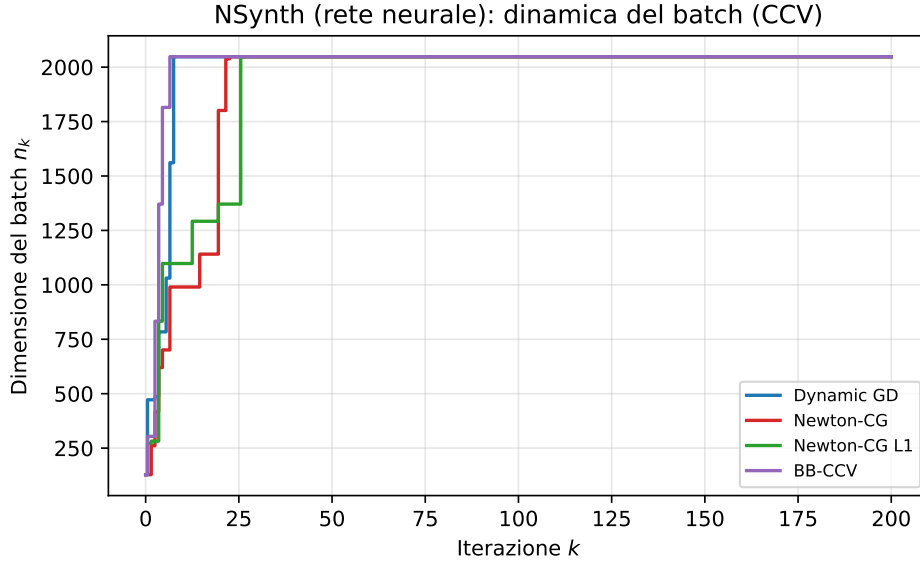


Figura 6.18: NSynth (rete neurale): dimensione del batch n_k scelta dal controllo CCV a ogni iterazione per i quattro metodi (tetto a 2048).

Il codice completo è disponibile nel notebook Colab `nsynth_net_riproduzione`, https://colab.research.google.com/github/Alessandro1040/selezione-dinamica-batch/blob/main/tesi/nsynth/nsynth_net_riproduzione.ipynb; un secondo notebook, `nsynth_net_test`, https://colab.research.google.com/github/Alessandro1040/selezione-dinamica-batch/blob/main/tesi/nsynth/nsynth_net_test.ipynb, carica i pesi addestrati e lo scaler e verifica i modelli su clip reali con le etichette vere: per una clip scelta (per indice o nome) stampa la famiglia vera e le predizioni dei quattro metodi — ad esempio la clip `keyboard_electronic_078-053-050` del test set è classificata come `keyboard` da tutti i metodi con probabilità superiore al 99% — e permette inoltre di caricare un proprio file audio e di ascoltare le clip. L'intero repository, con script, figure e note operative, è disponibile su <https://github.com/Alessandro1040/selezione-dinamica-batch>.

7 Conclusioni e Lavoro Futuro

7.1 Riassunto dei risultati

In questo lavoro abbiamo studiato la selezione dinamica della dimensione del campione negli algoritmi di ottimizzazione per il machine learning su larga scala, seguendo l'approccio di Byrd, Chin, Nocedal e Wu [1]. I risultati principali sono i seguenti.

In primo luogo, abbiamo sviluppato un'analisi teorica completa del metodo del gradiente a campione dinamico. Abbiamo mostrato che, se la varianza stimata del gradiente stocastico è controllata dalla tolleranza θ (Condizione di Controllo della Varianza), la direzione di ricerca è una direzione di discesa e l'algoritmo converge linearmente, sia in forma deterministica sia in aspettativa. La complessità totale — il numero di valutazioni di gradienti individuali necessarie per ottenere un errore ε — è $O(mL/(\lambda\varepsilon))$, un bound competitivo con quelli noti per SGD e significativamente migliore su problemi mal condizionati, dove il termine κ^2 di SGD diventa κ .

In secondo luogo, abbiamo analizzato e implementato il metodo di Newton-CG con sottocampionamento dell'Hessiana, nel quale due campioni distinti vengono usati per il gradiente e per l'Hessiana. Il criterio di arresto del gradiente coniugato interno è adattivo: il CG viene fermato quando il suo residuo è dello stesso ordine di grandezza dell'errore di approssimazione dell'Hessiana, stimato dalla varianza campionaria dei prodotti Hessiana-vettore. Questo evita iterazioni interne inutili e rende il metodo particolarmente efficace su problemi mal condizionati, come confermato dagli esperimenti del Capitolo 6.

In terzo luogo, abbiamo studiato il campionamento dinamico per i problemi con regolarizzazione L_1 , utilizzando il gradiente generalizzato, l'identificazione dell'active set e la ricerca lineare proiettata. Abbiamo infine realizzato un'applicazione web interattiva che implementa i tre algoritmi e ne permette la sperimentazione visuale su funzioni obiettivo modificabili dall'utente.

7.2 Limiti dello studio

Lo studio presenta alcuni limiti che è importante esplicitare.

1. Ipotesi di convessità forte. L'analisi di convergenza richiede che J sia λ -fortemente convessa, un'ipotesi non sempre verificata nei problemi reali (ad esempio la perdita logistica non lo è). Come discusso nel Capitolo 3, l'ipotesi può essere aggirata aggiungendo una regolarizzazione $\frac{\gamma}{2}\|w\|^2$, che induce convessità forte con $\lambda = \gamma$, ma introduce un bias nel problema.
2. Limitazione della varianza. L'analisi di complessità si basa sull'ipotesi di limitatezza globale della varianza (Eq. (5.4) nel Capitolo 5), che può non valere in problemi con perdite non limitate o con code pesanti. In pratica questa ipotesi può essere rilassata richiedendo la limitatezza solo in un intorno del minimo, ma le garanzie teoriche in tal caso sono più deboli.
3. Dataset sintetico centrato. L'implementazione interattiva usa un dataset sintetico "centrato", in cui la media campionaria dei gradienti individuali coincide esattamente con il gradiente della funzione obiettivo teorica $J(w)$. Nei problemi reali questa coincidenza vale solo asintoticamente (nel limite $N \rightarrow \infty$), e la varianza dei gradienti individuali ha una struttura più ricca di quella dei dati sintetici.

7.3 Direzioni per lavoro futuro

Dai limiti appena discussi emergono naturalmente alcune direzioni di lavoro futuro.

1. Rilassamento dell'ipotesi di convessità forte. Nella nostra analisi la condizione di Polyak-Łojasiewicz (PL) interviene solo come conseguenza della convessità forte: l'ipotesi $\nabla^2 J(w) \succeq \lambda I$ implica $\|\nabla J(w)\|^2 \geq 2\lambda J(w)$ (Appendice A.1) e da questa disuguaglianza si ricava la convergenza lineare del Teorema di Convergenza Deterministica. Un primo obiettivo è quindi invertire il ruolo delle due ipotesi: assumere direttamente la condizione PL, $\|\nabla J(w)\|^2 \geq 2\mu J(w)$ con $\mu > 0$ (forma forte, come in [7, 8]), eliminando la richiesta di convessità forte. Poiché la convessità forte è un caso particolare della PL (basta prendere

$\mu = \lambda$), la generalizzazione non comporta alcuna perdita di risultati: le dimostrazioni restano valide sostituendo λ con μ e sopprimendo il passaggio in cui la PL è dedotta dall'Hessiana. Il vantaggio è duplice: l'analisi copre una classe più ampia di funzioni, per le quali la PL garantisce comunque la convergenza lineare del valore di funzione pur senza richiedere la convessità (forte); e le dimostrazioni risultano più semplici, perché la PL diventa un'ipotesi di partenza invece di essere derivata.

2. Rapporto di sottocampionamento dell'Hessiana dinamico. Nel metodo Newton-CG il rapporto $R = |\mathcal{H}_k|/|\mathcal{S}_k|$ tra la dimensione del campione per l'Hessiana e quella per il gradiente è attualmente fisso. Un'estensione naturale è rendere dinamico anche questo rapporto, ad esempio aumentandolo quando la curvatura del problema cambia rapidamente o quando il criterio di arresto del CG richiede troppe iterazioni interne, bilanciando il costo dei prodotti Hessiana-vettore con la qualità della direzione di Newton.

A Dimostrazioni

A.1 Dimostrazione delle disuguaglianze di Polyak–Lojasiewicz

Sia w_* l'unico minimo di J e, per semplificare le notazioni, poniamo $J(w_*) = 0$. Allora con le ipotesi strutturali esplicitate sopra valgono le seguenti disuguaglianze:

$$\|\nabla J(w)\|_2^2 \geq 2\lambda J(w), \quad J(w) \geq \frac{\lambda}{2L^2} \|\nabla J(w)\|_2^2.$$

Come prima cosa cerchiamo una rappresentazione integrale esatta di $J(w)$: per ottenere una formula che coinvolga esplicitamente l'Hessiana, applichiamo il Teorema Fondamentale del Calcolo due volte.

Prima applicazione: definiamo $\varphi(t) = J(w_* + t(w - w_*))$ per $t \in [0, 1]$. Allora $\varphi(0) = J(w_*) = 0$, $\varphi(1) = J(w)$, e $\varphi'(t) = \nabla J(w_* + t(w - w_*))^T (w - w_*)$. Per il TFC:

$$J(w) = \underbrace{J(w_*)}_{\varphi(0)=0} + \int_0^1 \nabla J(w_* + t(w - w_*))^T (w - w_*) dt. \quad (\text{A.1})$$

Seconda applicazione: applichiamo ora il TFC alla funzione $\psi(t) = \nabla J(w_* + t(w - w_*))$. Si ha $\psi(0) = \nabla J(w_*) = 0$, $\psi(1) = \nabla J(w)$, e $\psi'(t) = \nabla^2 J(w_* + t(w - w_*))(w - w_*)$. Quindi per ogni $t \in [0, 1]$:

$$\nabla J(w_* + t(w - w_*)) = \int_0^t \nabla^2 J(w_* + s(w - w_*))(w - w_*) ds. \quad (\text{A.2})$$

Sostituiamo (A.2) nella (A.1):

$$J(w) = \int_0^1 \left(\int_0^t \nabla^2 J(w_* + s(w - w_*))(w - w_*) ds \right)^T (w - w_*) dt.$$

Poiché $(w - w_*)$ è costante rispetto agli integrali, possiamo riscrivere:

$$J(w) = \int_0^1 \int_0^t (w - w_*)^T \nabla^2 J(w_* + s(w - w_*))(w - w_*) ds dt.$$

La regione di integrazione è $0 \leq s \leq t \leq 1$, che è equivalente a $0 \leq s \leq 1$, $s \leq t \leq 1$. Scambiando l'ordine di integrazione:

$$J(w) = \int_0^1 \int_s^1 (w - w_*)^T \nabla^2 J(w_* + s(w - w_*))(w - w_*) dt ds.$$

L'integrale interno in dt vale $\int_s^1 dt = 1 - s$. Quindi otteniamo la formula finale:

$$J(w) = \int_0^1 (1 - t)(w - w_*)^T \nabla^2 J(w_* + t(w - w_*))(w - w_*) dt. \quad (\text{A.3})$$

Si consideri ora la derivazione della disuguaglianza $\|\nabla J(w)\|^2 \geq 2\lambda J(w)$. Scriviamo la rappresentazione integrale di $J(y)$ a partire da $J(w)$:

$$J(y) = J(w) + \int_0^1 \nabla J(w + t(y - w))^T (y - w) dt. \quad (\text{A.4})$$

Applicando nuovamente il TFC al gradiente:

$$\nabla J(w + t(y - w)) = \nabla J(w) + \int_0^t \nabla^2 J(w + s(y - w))(y - w) ds. \quad (\text{A.5})$$

Sostituendo (A.5) in (A.4):

$$J(y) = J(w) + \nabla J(w)^T (y - w) + \int_0^1 \int_0^t (y - w)^T \nabla^2 J(w + s(y - w))(y - w) ds dt.$$

Dalla convessità forte $\nabla^2 J \succeq \lambda I$, l'integrale doppio è maggiorato dal basso da:

$$\lambda \|y - w\|^2 \int_0^1 \int_0^t 1 ds dt = \frac{\lambda}{2} \|y - w\|^2.$$

Pertanto:

$$J(y) \geq J(w) + \nabla J(w)^T (y - w) + \frac{\lambda}{2} \|y - w\|^2. \quad (\text{A.6})$$

Minimizzando il membro destro rispetto a y si ottiene:

$$\min_y \left[J(w) + \nabla J(w)^T (y - w) + \frac{\lambda}{2} \|y - w\|^2 \right] = J(w) - \frac{1}{2\lambda} \|\nabla J(w)\|^2.$$

Dalla (A.6) segue che $J(y)$ è sempre maggiore o uguale al termine quadratico in y ; quindi il minimo di J sarà maggiore o uguale al minimo di tale termine. Poiché $y = w_*$ è il minimo di J e $J(w_*) = 0$, si ha:

$$0 = J(w_*) \geq J(w) - \frac{1}{2\lambda} \|\nabla J(w)\|^2,$$

da cui la tesi $\|\nabla J(w)\|^2 \geq 2\lambda J(w)$.

Si consideri ora la derivazione della disuguaglianza $J(w) \geq \frac{\lambda}{2L^2} \|\nabla J(w)\|^2$. Dalla rappresentazione integrale del gradiente che si ottiene valutando la (A.2) in $t = 1$:

$$\nabla J(w) = \int_0^1 \nabla^2 J(w_* + t(w - w_*))(w - w_*) dt. \quad (\text{A.7})$$

Passando alla norma:

$$\|\nabla J(w)\| = \left\| \int_0^1 \nabla^2 J(w_* + t(w - w_*))(w - w_*) dt \right\|.$$

Per la disuguaglianza triangolare in forma integrale:

$$\|\nabla J(w)\| \leq \int_0^1 \|\nabla^2 J(w_* + t(w - w_*))(w - w_*)\| dt.$$

Usando la limitatezza $\|\nabla^2 J\| \leq L$, si ha:

$$\|\nabla J(w)\| \leq \int_0^1 L \|w - w_*\| dt = L \|w - w_*\|. \quad (\text{A.8})$$

Da cui:

$$\|\nabla J(w)\| \leq L \|w - w_*\| \implies \|w - w_*\| \geq \frac{1}{L} \|\nabla J(w)\|. \quad (\text{A.9})$$

Elevando al quadrato (A.9):

$$\|w - w_*\|^2 \geq \frac{1}{L^2} \|\nabla J(w)\|^2. \quad (\text{A.10})$$

Dalla (A.3) invece:

$$J(w) = \int_0^1 (1-t) (w - w_*)^T \nabla^2 J(w_* + t(w - w_*)) (w - w_*) dt.$$

Usando la convessità forte $\nabla^2 J \succeq \lambda I$:

$$J(w) \geq \|w - w_*\|^2 \lambda \int_0^1 (1-t) dt.$$

L'integrale vale:

$$\int_0^1 (1-t) dt = \left[t - \frac{t^2}{2} \right]_0^1 = \frac{1}{2}.$$

Quindi:

$$J(w) \geq \frac{\lambda}{2} \|w - w_*\|^2. \quad (\text{A.11})$$

Combinando (A.10) e (A.11):

$$J(w) \geq \frac{\lambda}{2} \|w - w_*\|^2 \geq \frac{\lambda}{2} \cdot \frac{1}{L^2} \|\nabla J(w)\|^2 = \frac{\lambda}{2L^2} \|\nabla J(w)\|^2.$$

Questa è la seconda disuguaglianza cercata.

Con passaggi simili a quelli che sono stati fatti per dimostrare la (A.6) possiamo verificare che applicando il teorema di Taylor con resto integrale e usando $\nabla^2 J \preceq LI$ otteniamo

$$J(y) \leq J(w) + \nabla J(w)^T (y - w) + \frac{L}{2} \|y - w\|^2. \quad (\text{A.12})$$

A.2 Dimostrazione del fattore di correzione per popolazione finita

Il fattore $\frac{N-n_k}{N-1}$ è il Fattore di Correzione per Popolazione Finita. Per dimostrare formalmente il motivo per cui moltiplichiamo per questo fattore nel caso in cui accettassimo la possibilità di prendere più volte gli stessi dati nei batch possiamo prima dimostrare la necessità di questo fattore nel caso generale e poi applicarlo al nostro caso specifico: Consideriamo $\bar{X} = \frac{1}{n} \sum_{i \in \mathcal{S}} X_i = \frac{1}{n} \sum_{i=1}^N I_i X_i$ con

$$I_i = \begin{cases} 1 & \text{se } X_i \in \mathcal{S} \\ 0 & \text{se } X_i \notin \mathcal{S} \end{cases} \implies \begin{cases} \mathbb{E}[I_i] = \frac{n}{N}, & \mathbb{E}[I_i I_\ell] = \frac{n(n-1)}{N(N-1)} \\ \text{Var}(I_i) = \frac{n(N-n)}{N^2}, & \text{Cov}(I_i, I_\ell) = -\frac{n(N-n)}{N^2(N-1)} \end{cases}$$

$$\text{Var} \left(\sum_{i \in \mathcal{S}} X_i \right) = \sum_{i=1}^N X_i^2 \text{Var}(I_i) + \sum_{\substack{i, \ell \in \{1, \dots, N\} \\ i \neq \ell}} X_i X_\ell \text{Cov}(I_i, I_\ell)$$

Per verificarlo, basta espandere la definizione di varianza applicata alla combinazione lineare $\sum_{i=1}^N X_i I_i$: si scrive

$$\text{Var} \left(\sum_{i=1}^N X_i I_i \right) = \mathbb{E} \left[\left(\sum_{i=1}^N X_i (I_i - \mathbb{E}[I_i]) \right)^2 \right],$$

si sviluppa il quadrato della somma usando $\left(\sum_{i=1}^N a_i \right)^2 = \sum_{i=1}^N a_i^2 + \sum_{i \neq \ell} a_i a_\ell$ con $a_i = X_i (I_i - \mathbb{E}[I_i])$ (e questa servirà anche dopo, vera perché: $\underbrace{\sum_i \sum_\ell a_i a_\ell}_{(\sum_i a_i)^2} =$

$\sum_i a_i^2 + \sum_{i \neq \ell} a_i a_\ell \iff \sum_{i \neq \ell} a_i a_\ell = (\sum_i a_i)^2 - \sum_i a_i^2$), si applica la linearità dell'aspettazione e si riconoscono le formule per la varianza e covarianza di I

$$\begin{aligned} &= \frac{n(N-n)}{N^2} \sum_{i=1}^N X_i^2 - \frac{n(N-n)}{N^2(N-1)} \sum_{i \neq \ell} X_i X_\ell \\ &= \frac{n(N-n)}{N^2} \left[\sum_{i=1}^N X_i^2 - \frac{1}{N-1} \sum_{i \neq \ell} X_i X_\ell \right] \\ &\quad \sum_{i=1}^N X_i = N\mu \\ &\quad \sum_{i=1}^N X_i^2 = N(\mu^2 + \sigma^2) \end{aligned}$$

Infatti $\sigma^2 = \frac{1}{N} \sum_{i=1}^N (X_i - \mu)^2 \iff N\sigma^2 = \sum_{i=1}^N (X_i^2 - 2\mu X_i + \mu^2) \iff N\sigma^2 = \sum_{i=1}^N X_i^2 - 2\mu \sum_{i=1}^N X_i + N\mu^2 \iff N\sigma^2 = \sum_{i=1}^N X_i^2 - 2\mu(N\mu) + N\mu^2 \iff N\sigma^2 = \sum_{i=1}^N X_i^2 - 2N\mu^2 + N\mu^2 \iff N\sigma^2 = \sum_{i=1}^N X_i^2 - N\mu^2 \iff \sum_{i=1}^N X_i^2 = N\sigma^2 + N\mu^2 = N(\mu^2 + \sigma^2)$

$$\sum_{i \neq \ell} X_i X_\ell = \left(\sum_{i=1}^N X_i \right)^2 - \sum_{i=1}^N X_i^2 = N^2 \mu^2 - N(\mu^2 + \sigma^2)$$

$$\begin{aligned}
\sum_{i=1}^N X_i^2 - \frac{1}{N-1} \sum_{i \neq \ell} X_i X_\ell &= N(\mu^2 + \sigma^2) - \frac{1}{N-1} [N^2 \mu^2 - N(\mu^2 + \sigma^2)] \\
&= N(\mu^2 + \sigma^2) - \frac{N^2 \mu^2}{N-1} + \frac{N(\mu^2 + \sigma^2)}{N-1} \\
&= N(\mu^2 + \sigma^2) \left(1 + \frac{1}{N-1}\right) - \frac{N^2 \mu^2}{N-1} = N(\mu^2 + \sigma^2) \frac{N}{N-1} - \frac{N^2 \mu^2}{N-1} \\
&= \frac{N^2(\mu^2 + \sigma^2)}{N-1} - \frac{N^2 \mu^2}{N-1} = \frac{N^2 \sigma^2}{N-1} = N \sigma^2 \frac{N}{N-1} \\
\text{Var} \left(\sum_{i \in \mathcal{S}} X_i \right) &= \frac{n(N-n)}{N^2} \cdot N \sigma^2 \frac{N}{N-1} = \frac{n(N-n)}{N-1} \sigma^2 \\
\text{Var}(\bar{X}) &= \frac{1}{n^2} \text{Var} \left(\sum_{i \in \mathcal{S}} X_i \right) = \frac{1}{n^2} \cdot \frac{n(N-n)}{N-1} \sigma^2 = \frac{\sigma^2}{n} \cdot \frac{N-n}{N-1}
\end{aligned}$$

Nella dimostrazione generale con le variabili X_i , poniamo:

- $X_i \longleftrightarrow \nabla \ell(w_k; i)$ (gradiente della funzione di perdita sull'esempio i -esimo, valutato in w_k)
- $\mu \longleftrightarrow \nabla J(w_k) = \mathbb{E}_{i \sim \text{Unif}\{1, \dots, N\}} [\nabla \ell(w_k; i)] = \frac{1}{N} \sum_{i=1}^N \nabla \ell(w_k; i)$
- $\sigma^2 \longleftrightarrow \mathcal{V}_j(w_k)$ per ogni $j = 1, \dots, m$, dove

$$\mathcal{V}(w_k) := \frac{1}{N} \sum_{i=1}^N (\nabla \ell(w_k; i) - \nabla J(w_k))^2 \in \mathbb{R}^m$$

è la versione vettoriale di σ^2 nel caso multidimensionale.

Applicando la formula generale $\text{Var}(\bar{X}) = \frac{\mathcal{V}}{n_k} \cdot \frac{N-n_k}{N-1}$ a ogni componente $j = 1, \dots, m$:

$$\text{Var}(\nabla J_{\mathcal{S}_k}(w_k)) = \frac{\mathcal{V}(w_k)}{n_k} \cdot \frac{N-n_k}{N-1}$$

e quindi, in norma $\|\cdot\|_1$:

$$\|\text{Var}(\nabla J_{\mathcal{S}_k}(w_k))\|_1 = \frac{\|\mathcal{V}(w_k)\|_1}{n_k} \cdot \frac{N-n_k}{N-1}.$$

Poiché $\mathcal{V}(w_k)$ è ignoto, lo stimiamo con la varianza campionaria

$$\hat{\mathcal{V}}(w_k) := \frac{1}{n_k - 1} \sum_{i \in \mathcal{S}_k} (\nabla \ell(w_k; i) - \nabla J_{\mathcal{S}_k}(w_k))^2,$$

ottenendo la condizione di controllo della varianza:

$$\frac{\|\hat{\mathcal{V}}\|_1}{n_k} \cdot \frac{N-n_k}{N-1} \leq \theta^2 \|\nabla J_{\mathcal{S}_k}(w_k)\|^2.$$

A.3 Dimostrazione delle formule per α_k e β_k nel Metodo del Gradiente Coniugato

Consideriamo il funzionale quadratico

$$\phi(x) = \frac{1}{2}x^T A x - b^T x, \quad A = A^T > 0. \quad (\text{A.13})$$

Sia x^* il minimo, soluzione di $Ax^* = b$. Definiamo

$$\begin{cases} e^k = x^* - x^k, \\ r^k = b - Ax^k, \end{cases} \quad \xRightarrow{Ax^* = b} \quad r^k = Ae^k. \quad (\text{A.14})$$

L'aggiornamento del metodo è

$$\begin{cases} x^{k+1} = x^k + \alpha_k p^k, \\ p^k = r^k + \beta_k p^{k-1}, \quad p^0 = r^0 \end{cases} \quad (\text{A.15})$$

da cui, per la definizione di e^k ,

$$e^{k+1} = e^k - \alpha_k p^k. \quad (\text{A.16})$$

Il metodo minimizza $\|e^{k+1}\|_A^2$ nel piano $e^k + \text{span}(p^k, p^{k-1})$. Definiamo

$$F(\alpha, \gamma) = \|e^k + \alpha p^k + \gamma p^{k-1}\|_A^2. \quad (\text{A.17})$$

Poiché $\|z\|_A^2 = z^T A z$, con $z = e^k + \alpha p^k + \gamma p^{k-1}$:

$$\begin{aligned} F(\alpha, \gamma) &= (e^k + \alpha p^k + \gamma p^{k-1})^T A (e^k + \alpha p^k + \gamma p^{k-1}) \\ &= \|e^k\|_A^2 + 2\alpha(e^k)^T A p^k + 2\gamma(e^k)^T A p^{k-1} \\ &\quad + \alpha^2 \|p^k\|_A^2 + 2\alpha\gamma(p^k)^T A p^{k-1} + \gamma^2 \|p^{k-1}\|_A^2. \end{aligned} \quad (\text{A.18})$$

Annullando le derivate parziali:

$$\begin{cases} (e^k)^T A p^k + \alpha \|p^k\|_A^2 + \gamma (p^k)^T A p^{k-1} = 0, \\ (e^k)^T A p^{k-1} + \alpha (p^k)^T A p^{k-1} + \gamma \|p^{k-1}\|_A^2 = 0. \end{cases} \quad (\text{A.19})$$

Per la scelta di α_{k-1} (minimo lungo p^{k-1}):

$$\frac{d}{d\alpha} \phi(x^{k-1} + \alpha p^{k-1}) \Big|_{\alpha=\alpha_{k-1}} = 0 \implies \nabla \phi(x^k)^T p^{k-1} = 0 \implies (r^k)^T p^{k-1} = 0. \quad (\text{A.20})$$

Usando $r^k = Ae^k$ e la simmetria di A :

$$(e^k)^T A p^{k-1} = (Ae^k)^T p^{k-1} = (r^k)^T p^{k-1} = 0. \quad (\text{A.21})$$

Si noti l'ordine logico della deduzione: si applica prima la (A.21) alla seconda equazione del sistema (A.19) il termine $(e^k)^T A p^{k-1}$ si annulla e si ottiene $\gamma \|p^{k-1}\|_A^2 = 0$, da cui $\gamma = 0$; la proprietà (A.23), cioè $(p^k)^T A p^{k-1} = 0$, segue poi come conseguenza della prima equazione.

La minimizzazione nel piano implica le condizioni di ortogonalità

$$e^{k+1} \perp_A p^k \quad \text{e} \quad e^{k+1} \perp_A p^{k-1}. \quad (\text{A.22})$$

Dalla seconda, con $e^{k+1} = e^k - \alpha_k p^k$ e (A.21):

$$(e^k - \alpha_k p^k)^T A p^{k-1} = 0 \implies -\alpha_k (p^k)^T A p^{k-1} = 0 \xrightarrow{\alpha_k \neq 0} (p^k)^T A p^{k-1} = 0. \quad (\text{A.23})$$

Sfruttando (A.21) e (A.23) nel sistema (A.19):

$$\begin{cases} (e^k)^T A p^k + \alpha \|p^k\|_A^2 = 0, \\ \gamma \|p^{k-1}\|_A^2 = 0, \end{cases} \implies \gamma = 0, \quad \alpha = -\frac{(e^k)^T A p^k}{\|p^k\|_A^2}. \quad (\text{A.24})$$

Ora, $(e^k)^T A p^k = (r^k)^T p^k$ e da (A.20) $r^k \perp p^{k-1}$; quindi

$$(r^k)^T p^k = (r^k)^T (r^k + \beta_k p^{k-1}) = \|r^k\|^2. \quad (\text{A.25})$$

Pertanto

$$\alpha_k = \frac{\|r^k\|^2}{\|p^k\|_A^2} = \frac{(r^k)^T r^k}{(p^k)^T A p^k}. \quad (\text{A.26})$$

Per β_k , da (A.23) e $p^k = r^k + \beta_k p^{k-1}$:

$$(r^k + \beta_k p^{k-1})^T A p^{k-1} = 0 \implies \beta_k = -\frac{(r^k)^T A p^{k-1}}{(p^{k-1})^T A p^{k-1}}. \quad (\text{A.27})$$

Dall'aggiornamento del residuo:

$$r^k = r^{k-1} - \alpha_{k-1} A p^{k-1} \implies A p^{k-1} = \frac{r^{k-1} - r^k}{\alpha_{k-1}}. \quad (\text{A.28})$$

Al numeratore di (A.27):

$$(r^k)^T A p^{k-1} = \frac{(r^k)^T r^{k-1} - (r^k)^T r^k}{\alpha_{k-1}} \stackrel{(r^k)^T r^{k-1}=0}{=} -\frac{(r^k)^T r^k}{\alpha_{k-1}}. \quad (\text{A.29})$$

Dimostrazione di $(r^k)^T r^{k-1} = 0$: Mostriamo per induzione che $r^k \perp p^j$ per $j \leq k-1$.

- Base: $k=1$, (A.20) dà $r^1 \perp p^0$.
- Passo: supponiamo $r^{k-1} \perp p^j$ per $j \leq k-2$. Per $r^k = r^{k-1} - \alpha_{k-1} A p^{k-1}$ e $j \leq k-2$:

$$(r^k)^T p^j = (r^{k-1})^T p^j - \alpha_{k-1} (p^{k-1})^T A p^j = 0.$$

Inoltre (A.20) dà $(r^k)^T p^{k-1} = 0$. Quindi $r^k \perp p^j$ per $j \leq k-1$.

In particolare $r^k \perp p^{k-2}$. Da $p^{k-1} = r^{k-1} + \beta_{k-1} p^{k-2}$:

$$0 = (r^k)^T p^{k-1} = (r^k)^T r^{k-1} + \beta_{k-1} (r^k)^T p^{k-2} \implies (r^k)^T r^{k-1} = 0. \quad (\text{A.30})$$

Al denominatore di (A.27), usando (A.26) per $k-1$:

$$(p^{k-1})^T A p^{k-1} = \frac{(r^{k-1})^T r^{k-1}}{\alpha_{k-1}}. \quad (\text{A.31})$$

Sostituendo (A.29) e (A.31) in (A.27):

$$\beta_k = -\frac{-\frac{(r^k)^T r^k}{\alpha_{k-1}}}{\frac{(r^{k-1})^T r^{k-1}}{\alpha_{k-1}}} = \frac{(r^k)^T r^k}{(r^{k-1})^T r^{k-1}}. \quad (\text{A.32})$$

Abbiamo quindi dimostrato che le condizioni di ortogonalità (A.22) portano a:

$$\alpha_k = \frac{(r^k)^T r^k}{(p^k)^T A p^k}, \quad \beta_k = \frac{(r^k)^T r^k}{(r^{k-1})^T r^{k-1}} \quad (\text{A.33})$$

con

$$p^k = r^k + \beta_k p^{k-1}, \quad x^{k+1} = x^k + \alpha_k p^k. \quad (\text{A.34})$$

Equivalenza delle formule per α_k . Nel documento (Nota Tecnica) si afferma che le formule $\alpha_k = \frac{r_k^T p_k}{p_k^T A p_k}$ e $\alpha_k = \frac{r_k^T r_k}{p_k^T A p_k}$ sono equivalenti. Dimostriamo questa equivalenza.

Prendendo la condizione di stazionarietà (A.20), ovvero $(r^k)^T p^{k-1} = 0$, e sostituendo l'aggiornamento del residuo (A.28), $r^k = r^{k-1} - \alpha_{k-1} A p^{k-1}$, si ottiene:

$$\begin{aligned} 0 &= (r^k)^T p^{k-1} = (r^{k-1} - \alpha_{k-1} A p^{k-1})^T p^{k-1} = (r^{k-1})^T p^{k-1} - \alpha_{k-1} (p^{k-1})^T A p^{k-1} \\ &\implies (r^{k-1})^T p^{k-1} - \alpha_{k-1} (p^{k-1})^T A p^{k-1} = 0. \end{aligned} \quad (\text{A.35})$$

Ora, l'aggiornamento del residuo nel metodo del gradiente coniugato è:

$$r_k = r_{k-1} - \alpha_{k-1} A p_{k-1}. \quad (\text{A.36})$$

Moltiplichiamo scalarmente la (A.36) per p_{k-1} :

$$r_k^T p_{k-1} = r_{k-1}^T p_{k-1} - \alpha_{k-1} p_{k-1}^T A p_{k-1}.$$

Ma il membro destro è esattamente zero per la (A.35). Quindi:

$$r_k^T p_{k-1} = 0. \quad (\text{A.37})$$

Questo risultato è valido per ogni $k \geq 1$ e deriva unicamente dalla line search esatta al passo precedente.

Ora, nella direzione di ricerca del gradiente coniugato si ha:

$$p_k = r_k + \beta_k p_{k-1}. \quad (\text{A.38})$$

Moltiplichiamo scalarmente la (A.38) per r_k :

$$r_k^T p_k = r_k^T r_k + \beta_k r_k^T p_{k-1}.$$

Usando la (A.37), il secondo termine si annulla:

$$r_k^T p_k = r_k^T r_k. \quad (\text{A.39})$$

Sostituendo la (A.39) nella formula $\alpha_k = \frac{r_k^T p_k}{p_k^T A p_k}$, otteniamo:

$$\alpha_k = \frac{r_k^T r_k}{p_k^T A p_k}. \quad (\text{A.40})$$

Abbiamo quindi dimostrato che:

$$\frac{r_k^T p_k}{p_k^T A p_k} = \frac{r_k^T r_k}{p_k^T A p_k}.$$

Le due formule per α_k sono quindi equivalenti. La formula con $r_k^T r_k$ è preferita nelle implementazioni numeriche perché è numericamente più stabile, poiché non dipende dall'ortogonalità numerica di $r_k^T p_{k-1}$, che in presenza di errori di arrotondamento non è esattamente zero.

A.4 Spiegazione delle condizioni sul passo α_k

La condizione di Armijo introduce un limite superiore per il valore della funzione lungo la direzione di ricerca: il passo α è accettato se $f(x_k + \alpha p)$ si trova al di sotto della retta $f(x_k) + c_1 \alpha \nabla f(x_k)^T p$, e rifiutato altrimenti. La condizione di curvatura di Wolfe impone invece un limite inferiore per la derivata $f'(\alpha)$ lungo la direzione.

La condizione di Armijo richiede che

$$f(x_k + \alpha p) \leq f(x_k) + c_1 \alpha \nabla f(x_k)^T p,$$

con $c_1 \in (0, 1)$ (tipicamente 10^{-4}). La retta al membro destro ha pendenza $c_1 \nabla f(x_k)^T p$, meno ripida della tangente in $\alpha = 0$, e funge da upper bound: passi che fanno superare questa retta vengono rifiutati e ridotti. Più c_1 è piccolo, più la condizione è debole e più passi grandi sono accettati.

La condizione di curvatura di Wolfe, con $g(\alpha) = f(x_k + \alpha p)$ e $g'(\alpha) = \nabla f(x_k + \alpha p)^T p$, richiede

$$g'(\alpha) \geq c_2 g'(0), \quad c_2 \in (c_1, 1),$$

dove $g'(0) = \nabla f(x_k)^T p < 0$. Poiché $c_2 < 1$, $c_2 g'(0)$ è meno negativo di $g'(0)$: la condizione è un lower bound sulla derivata, che scarta i passi troppo piccoli (per $\alpha \rightarrow 0^+$, $g'(\alpha) \rightarrow g'(0)$, che non soddisfa la disuguaglianza).

La condizione $0 < c_1 < c_2 < 1$ garantisce che le due disuguaglianze siano compatibili: Armijo (limite superiore su f) e Wolfe (limite inferiore su g') definiscono un intervallo non vuoto di α accettabili. Se $c_1 \geq c_2$, le due condizioni possono diventare contraddittorie. In sintesi, Armijo impedisce passi troppo grandi, Wolfe impedisce passi troppo piccoli.

B Codici Python completi

In questa appendice sono riportati i codici Python completi dei tre algoritmi discussi nel Capitolo 5 e implementati nell'applicazione interattiva (Capitolo 6). I codici fanno riferimento alle seguenti funzioni ausiliarie, definite dall'utente e dipendenti dal dataset:

- `loss_i(w, i)`: perdita individuale $\ell(w; i)$,
- `grad_i(w, i)`: gradiente individuale $\nabla \ell(w; i)$,
- `hessvec_i(w, i, v)`: prodotto $\nabla^2 \ell(w; i) v$,
- `grad_full(w)`: gradiente esatto $\nabla J(w)$ (usato solo per il criterio di arresto),
- N : dimensione del dataset.

B.1 Metodo del gradiente a campione dinamico

Dynamic GD

```
import numpy as np

def dynamic_gd(w0, theta, max_iter, alpha, batch0):
    w = np.array(w0, dtype=float)
    n = max(batch0, 2)
    history = [w.copy().tolist()]
    batch_sizes = [n]
```

Definisce una funzione che prende in input w_0 , θ , $\max_iter$, α , batch0 , poi inizializza il vettore dei pesi w_0 (dove $w_0 \in \mathbb{R}^d$, con d pari al numero di parametri del modello), imposta la dimensione del batch iniziale $n = n_0 = \max(\text{batch0}, 2)$ in modo che non possa essere minore di 2, e inizializza la lista ‘history’ (che conterrà i pesi w_k a ogni iterazione k) e ‘batch_sizes’ (che conterrà le corrispondenti dimensioni n_k di ogni batch).

Dynamic GD

```
#
for k in range(max_iter):
    indices = np.random.choice(N, size=n, replace=False)
```

Per ogni iterazione $k \in \{0, \dots, \max_iter\}$ crea una lista indices ovvero un campione casuale $\mathcal{S}_k = \{i_1, \dots, i_n\}$ di n indici distinti da $\{1, \dots, N\}$, estratto senza reinserimento (ovvero uniforme tra tutti i $\binom{N}{n}$ possibili sottoinsiemi). N è definito come il numero di esempi totali, nel codice Python, per questioni di indicizzazione, l'insieme è $\{0, \dots, N-1\}$.

Dynamic GD

```
#
grads = np.array([grad_i(w, i) for i in indices])
g = np.mean(grads, axis=0)
```

grads colleziona un insieme di gradienti di loss su tanti esempi diversi (uno per ogni indice) quindi g è l'approssimazione che viene fatta del gradiente della loss usando il batch. Matematicamente grads è una lista di $\nabla \ell(w_k; i)$ per ogni indice i con cui si costruisce il batch mentre $g = g_k = \nabla J_{\mathcal{S}_k}(w_k) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla \ell(w_k; i)$, dove $n_k = |\mathcal{S}_k|$ è la dimensione del batch corrente.

Dynamic GD

```
#
if n > 1:
    var_vec = np.var(grads, axis=0, ddof=1)
else:
    var_vec = np.zeros_like(g)
```

```

V_norm1 = np.sum(var_vec)

gg = np.dot(g, g)
if gg > 1e-16:
    if V_norm1 / n > theta**2 * gg:
        n_new = int(np.ceil(V_norm1 / (theta**2 * gg))) + 1
        n = min(n_new, N)

```

Matematicamente, var_vec è un vettore che calcola la varianza campionaria di ogni componente del gradiente sui dati del batch: $\hat{\mathcal{V}} := \text{Var}_{i \in \mathcal{S}_k}(\nabla \ell(w_k; i)) = \frac{1}{n_k-1} \sum_{i \in \mathcal{S}_k} (\nabla \ell(w_k; i) - g_k)^2 \in \mathbb{R}^d$, dove $g_k = \nabla J_{\mathcal{S}_k}(w_k)$ come già definito. Poi V_norm1 ne prende la norma L_1 : $\|\hat{\mathcal{V}}\|_1 = \sum_{j=1}^d [\hat{\mathcal{V}}]_j$. Questa quantità viene confrontata con $\theta^2 \|g_k\|_2^2$ per decidere se aumentare il batch, quindi se la CCV è soddisfatta allora la dimensione del batch viene aumentata come descritto dalla regola di aggiornamento del batch. Se $n = 1$ (mai vero, il controllo è superfluo) la varianza campionaria non è definita (divisione per 0), quindi si pone $\hat{\mathcal{V}} = 0$

Dynamic GD

```

#
def J_batch(w_curr):
    return np.mean([loss_i(w_curr, i) for i in indices])

c1 = 1e-4
step = alpha
J_curr = J_batch(w)
g_norm2 = np.dot(g, g)

```

Definisce la funzione di perdita sul batch corrente $J_batch = J_{\mathcal{S}_k}(w) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \ell(w; i)$, definisce lo step che viene inizializzato come α , definisce la variabile J_curr chiamando la funzione J_batch sui parametri w e definisce g_norm2 come $g_k^T g_k = \|g_k\|_2^2$

Dynamic GD

```

#
def J_batch(w_curr):
    return np.mean([loss_i(w_curr, i) for i in indices])

# Line search Wolfe
c1, c2 = 1e-4, 0.9
step = alpha
J_curr = J_batch(w)
g_norm2 = np.dot(g, g)
d = -g
gd = -g_norm2
if g_norm2 > 1e-16:
    for _ in range(30):
        w_new = w + step * d

```

```

        if J_batch(w_new) <= J_curr + c1 * step * gd:
            g_new = np.mean([grad_i(w_new, i) for i in
                             indices], axis=0)
            if np.dot(g_new, d) >= c2 * gd:
                break
            step *= 0.5
        else:
            step = 0.0

    w = w + step * d

```

Si definisce la funzione di perdita sul batch corrente $J_{\mathcal{S}_k}(w)$. La direzione di ricerca è fissata come $d_k = -g_k$ e il prodotto scalare $g_k^\top d_k = -\|g_k\|_2^2$ misura la pendenza iniziale lungo tale direzione. Se il gradiente non è trascurabile ($\|g_k\|_2^2 > 10^{-16}$), si esegue una line search con backtracking che richiede le condizioni di Wolfe. $J_{\mathcal{S}_k}(w_k + \alpha_k d_k) \leq J_{\mathcal{S}_k}(w_k) + c_1 \alpha_k g_k^\top d_k$ con $c_1 = 10^{-4}$. Se questa è soddisfatta, si calcola il gradiente nel punto di prova sullo stesso batch e si verifica la seconda condizione (curvatura): il gradiente nel nuovo punto deve essere meno negativo lungo d_k , ovvero $\nabla J_{\mathcal{S}_k}(w_k + \alpha_k d_k)^\top d_k \geq c_2 g_k^\top d_k$ con $c_2 = 0.9$. Se una delle due condizioni fallisce, il passo viene dimezzato. Dopo al più 30 tentativi, se nessuno step è accettabile si pone $\alpha_k = 0$. Infine si aggiorna $w_{k+1} = w_k + \alpha_k d_k$.

Dynamic GD

```

if np.linalg.norm(grad_full(w)) < 1e-6:
    history.append(w.copy().tolist())
    batch_sizes.append(n)
    break

history.append(w.copy().tolist())
batch_sizes.append(n)

return history, batch_sizes

history, batch_sizes = dynamic_gd(w0, theta, max_iter, alpha, batch0)

```

Infine, si controlla la norma del gradiente esatto (calcolato su tutti i dati) tramite `grad_full(w)`: se $\|\nabla J(w_k)\|_2 < 10^{-6}$, si considera raggiunta la convergenza e si interrompe il ciclo, registrando l'ultimo punto. Altrimenti, si continua registrando la storia dei pesi e delle dimensioni del batch. Al termine, la funzione restituisce le liste `history` (contenente i pesi w_k a ogni iterazione) e `batch_sizes` (contenente le corrispondenti dimensioni n_k del batch).

B.2 Newton-CG con campionamento dinamico

Newton-CG

```
import numpy as np

def cg(A, b, gamma, maxcg):
    x = np.zeros_like(b)
    r = b - A(x)
    p = r.copy()
    rr = np.dot(r, r)
    for _ in range(maxcg):
        Ap = A(p)
        pHp = np.dot(p, Ap)
        if pHp <= 1e-14:
            break
        alpha = rr / pHp
        x = x + alpha * p
        r_new = r - alpha * Ap
        rr_new = np.dot(r_new, r_new)
        if rr_new <= gamma * np.dot(x, x) + 1e-16:
            return x
        beta = rr_new / rr
        p = r_new + beta * p
        r = r_new
        rr = rr_new
    return x
```

La funzione `cg` risolve $Ax = b$ con il metodo del gradiente coniugato, dove A è l'operatore Hessiana-vettore. Parte da $x_0 = 0$, residuo $r_0 = b$ e direzione $p_0 = r_0$. Ad ogni iterazione calcola Ap , la curvatura $pHp = p^T Ap$ (se troppo piccola, si ferma), aggiorna con $\alpha = rr/pHp$ e il residuo. Il test di arresto

$$\|r_{\text{new}}\|^2 \leq \gamma \|x\|^2 + 10^{-16}$$

è descritto nel paragrafo “Criterio di terminazione del gradiente coniugato” la soglia γ (calcolata dall'algoritmo esterno) bilancia l'errore di troncamento del CG con il rumore statistico del sottocampionamento dell'Hessiana. Se il test è soddisfatto, restituisce x ; altrimenti aggiorna $\beta = rr_{\text{new}}/rr$ e la direzione $p = r_{\text{new}} + \beta p$. Il ciclo termina al massimo dopo `maxcg` iterazioni.

Newton-CG

```
def newton_cg(w0, theta, max_iter, alpha, batch0, R, maxcg):
    w = np.array(w0, dtype=float)
    n = max(batch0, 2)
    history, batch_sizes = [w.copy().tolist()], [n]
```

La funzione principale `newton_cg` riceve il punto iniziale w_0 , la tolleranza θ per il controllo della varianza, il numero massimo di iterazioni esterne `max_iter`, il passo iniziale `alpha` per la line search, la dimensione del batch iniziale `batch0`, il rapporto R tra la dimensione del campione per l'Hessiana e quello per il gradiente, e il numero

massimo di iterazioni del CG `maxcg`. Il vettore dei pesi w viene convertito in array NumPy in doppia precisione, la dimensione del batch è forzata ad almeno 2 (per garantire che la varianza campionaria sia definita) e si inizializzano le liste `history` e `batch_sizes` che registreranno la traiettoria dei pesi e le dimensioni del batch a ogni iterazione.

Newton-CG

```
#
for k in range(max_iter):
    indices_S = np.random.choice(N, size=n, replace=False)
    g = np.mean([grad_i(w, i) for i in indices_S], axis=0)
```

Ad ogni iterazione esterna k , si estrae un campione casuale $\mathcal{S}_k$ di $n_k = n$ indici distinti (senza reinserimento) da $\{0, \dots, N-1\}$. Su questo campione si calcola il gradiente approssimato

$$g_k = \nabla J_{\mathcal{S}_k}(w_k) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla \ell(w_k; i),$$

che costituisce la stima stocastica del gradiente esatto $\nabla J(w_k)$.

Newton-CG

```
#
n_h = min(max(1, int(round(R * n))), N)
indices_H = np.random.choice(indices_S, size=n_h,
    replace=False)
Hv = lambda v: np.mean([hessvec_i(w, i, v) for i in
    indices_H], axis=0)
```

Si determina la dimensione del campione per l'Hessiana come $n_h = \lceil R n_k \rceil$, vincolata tra 1 e N . Viene quindi estratto un sottocampione $\mathcal{H}_k \subseteq \mathcal{S}_k$ (senza reinserimento) di dimensione n_h . L'operatore `Hv` definisce il prodotto Hessiana-vettore come

$$H_v(v) = \nabla^2 J_{\mathcal{H}_k}(w_k) v = \frac{1}{n_h} \sum_{i \in \mathcal{H}_k} \nabla^2 \ell(w_k; i) v.$$

Questo operatore viene passato al risolutore CG, che lo utilizzerà per calcolare i prodotti Ap senza mai costruire esplicitamente la matrice Hessiana.

Newton-CG

```
#
p0 = -g
p0_norm2 = np.dot(p0, p0)
gamma = 0.0
if p0_norm2 > 1e-16 and n_h > 1:
    Hp0 = np.array([hessvec_i(w, i, p0) for i in indices_H])
    gamma = np.sum(np.var(Hp0, axis=0, ddof=1)) / (n_h *
        p0_norm2)
```

Prima di chiamare il CG, si calcola il coefficiente γ che quantifica l'errore di approssimazione dell'Hessiana lungo la direzione iniziale $p_0 = -g_k$. Si valutano i prodotti Hessiana-vettore individuali $\nabla^2 \ell(w_k; i) p_0$ per ogni $i \in \mathcal{H}_k$; la loro varianza campionaria (componente per componente) viene aggregata in norma L_1 e normalizzata:

$$\gamma = \frac{\|\text{Var}_{i \in \mathcal{H}_k}(\nabla^2 \ell(w_k; i) p_0)\|_1}{n_h \|p_0\|_2^2}.$$

Se il gradiente è nullo (o quasi) o $n_h = 1$ (varianza non definita), si pone $\gamma = 0$, il che corrisponde a richiedere convergenza completa del CG. Questo parametro sarà utilizzato nel test di arresto del CG per fermarsi quando il residuo è confrontabile con il rumore statistico dell'Hessiana.

Newton-CG

```
#
d = cg(Hv, -g, gamma, maxcg)
```

Si risolve il sistema lineare di Newton approssimato

$$\nabla^2 J_{\mathcal{H}_k}(w_k) d = -g_k$$

chiamando la funzione `cg` con l'operatore `Hv`, il termine noto $-g_k$, il coefficiente γ e il limite di iterazioni `maxcg`. La soluzione d è la direzione di ricerca di Newton-CG, calcolata senza costruire l'Hessiana e con un criterio di arresto che bilancia l'accuratezza interna con l'incertezza statistica del campionamento.

Newton-CG

```
#
J_batch = lambda wc: np.mean([loss_i(wc, i) for i in
    indices_S])

c1, c2 = 1e-4, 0.9
step, J_w = alpha, J_batch(w)
gd = np.dot(g, d)
if gd >= 0:
    d = -g
    gd = -np.dot(g, g)
```

Si definisce la funzione obiettivo sul batch del gradiente, $J_{\mathcal{S}_k}(w)$, e si inizializza la ricerca lineare con passo `step = alpha`. Si calcola la derivata direzionale $gd = g_k^T d$. Se $gd \geq 0$, la direzione d non è di discesa; in tal caso si forza il fallback $d = -g_k$, che è sempre di discesa (a meno di gradiente nullo), e si ricalcola $gd = -\|g_k\|^2$.

Newton-CG

```
#
for _ in range(30):
    w_new = w + step * d
    if J_batch(w_new) <= J_w + c1 * step * gd:
```

```

        g_new = np.mean([grad_i(w_new, i) for i in
                           indices_S], axis=0)
        if np.dot(g_new, d) >= c2 * gd:
            break
        step *= 0.5
    else:
        step = 0.0
    w = w + step * d

```

La line search con backtracking cerca un passo α_k che soddisfi le condizioni di Wolfe. $J_{S_k}(w_k + \alpha_k d_k) \leq J_{S_k}(w_k) + c_1 \alpha_k g_k^T d_k$ con $c_1 = 10^{-4}$ e $\nabla J_{S_k}(w_k + \alpha_k d_k)^T d_k \geq c_2 g_k^T d_k$ con $c_2 = 0.9$ (Spiegazione nell'appendice). Se entrambe sono soddisfatte, il passo è accettato. In caso contrario, il passo viene dimezzato e si riprova, per al più 30 tentativi; se nessuno è accettabile, si pone $\alpha_k = 0$. L'aggiornamento dei pesi è quindi $w_{k+1} = w_k + \alpha_k d_k$.

Newton-CG

```

#
    history.append(w.copy().tolist())
    batch_sizes.append(n)

```

Si registrano il nuovo punto w_{k+1} e la dimensione corrente del batch n .

Newton-CG

```

#
    indices_new = np.random.choice(N, size=n, replace=False)
    g_new = np.mean([grad_i(w, i) for i in indices_new], axis=0)
    var_vec = np.var([grad_i(w, i) for i in indices_new], axis=0,
                      ddof=1) if n > 1 else np.zeros_like(g_new)
    V_norm1, gg_new = np.sum(var_vec), np.dot(g_new, g_new)
    if gg_new > 1e-16 and V_norm1 / n > theta**2 * gg_new:
        n = min(int(np.ceil(V_norm1 / (theta**2 * gg_new))) + 1,
                 N)

```

Dopo l'aggiornamento dei pesi, si verifica la Condizione di Controllo della Varianza (CCV) su un nuovo campione $\mathcal{S}_{k+1}$ di dimensione n (quella corrente) per decidere se aumentare il batch per l'iterazione successiva. Si calcolano i gradienti individuali sul nuovo campione, la loro varianza campionaria $\hat{\mathcal{V}}$ e la norma L_1 $V_{\text{norm1}} = \|\hat{\mathcal{V}}\|_1$. La CCV richiede

$$\frac{\|\hat{\mathcal{V}}\|_1}{n} \leq \theta^2 \|g_{\text{new}}\|_2^2.$$

Se non è soddisfatta, la nuova dimensione del batch viene stimata come

$$n \leftarrow \min \left\{ N, \left\lceil \frac{\|\hat{\mathcal{V}}\|_1}{\theta^2 \|g_{\text{new}}\|_2^2} \right\rceil + 1 \right\},$$

in modo che il batch cresca automaticamente quando la varianza dello stimatore è troppo alta rispetto alla norma del gradiente, ovvero quando ci si avvicina alla soluzione e serve maggiore precisione.

Newton-CG

```
#
    if np.linalg.norm(grad_full(w)) < 1e-6:
        break
    return history, batch_sizes

history, batch_sizes = newton_cg(w0, theta, max_iter, alpha, batch0,
    R, maxcg)
```

Infine, si controlla la norma del gradiente esatto (calcolato su tutto il dataset) tramite `grad_full`; se $\|\nabla J(w_{k+1})\|_2 < 10^{-6}$, si considera raggiunta la convergenza e si interrompe il ciclo. Al termine delle iterazioni, la funzione restituisce le liste `history` e `batch_sizes`, che contengono rispettivamente la traiettoria dei pesi w_k e le dimensioni del batch n_k a ogni iterazione. L'ultima riga mostra la chiamata tipica alla funzione, che produce le due liste per l'analisi successiva. L'algoritmo Newton CG è sostanzialmente Dynamic GD ma la direzione di discesa è quella di Newton anziché l'antigradiente

B.3 Newton-CG per problemi L_1 -regolarizzati

Newton-L1

```
import numpy as np

def newton_l1(w0, theta, max_iter, alpha, batch0, nu, sigma, maxcg,
    eta=0.5):
    w = np.array(w0, dtype=float)
    n = max(batch0, 2)
    history = [w.copy().tolist()]
    batch_sizes = [n]
```

La funzione principale `newton_l1` implementa il metodo Newton-CG con campionamento dinamico per problemi con regolarizzazione L_1 . Riceve il punto iniziale w_0 , la tolleranza θ per il controllo della varianza, il numero massimo di iterazioni esterne `max_iter`, il passo iniziale `alpha` per la ricerca lineare, la dimensione del batch iniziale `batch0`, il parametro di penalità $\nu > 0$, la costante $\sigma \in (0, 1)$ per la condizione di Armijo, il numero massimo di iterazioni del gradiente coniugato `maxcg` e il fattore di tolleranza $\eta \in (0, 1)$ per il criterio di arresto del CG. Il vettore dei pesi w viene convertito in array NumPy in doppia precisione, la dimensione del batch è forzata ad almeno 2 (per garantire che la varianza campionaria sia definita) e si inizializzano le liste `history` e `batch_sizes` che registreranno la traiettoria dei pesi e le dimensioni del batch a ogni iterazione.

Newton-L1

```
#
def F_batch(v, indices):
    Jb = np.mean([loss_i(v, i) for i in indices])
    return Jb + nu * np.sum(np.abs(v))
```

```

def subgrad_batch(v, indices):
    grads = np.array([grad_i(v, i) for i in indices])
    gJ = np.mean(grads, axis=0)
    g = np.zeros_like(v)
    for i in range(len(v)):
        if v[i] > 0:
            g[i] = gJ[i] + nu
        elif v[i] < 0:
            g[i] = gJ[i] - nu
        else:
            if gJ[i] < -nu:
                g[i] = gJ[i] + nu
            elif gJ[i] > nu:
                g[i] = gJ[i] - nu
            else:
                g[i] = 0.0
    return g
def project_orthant(v, z):
    res = v.copy()
    for i in range(len(v)):
        if z[i] != 0 and np.sign(res[i]) != z[i]:
            res[i] = 0.0
    return res

```

Vengono definite tre funzioni ausiliarie interne. `F_batch` calcola la funzione obiettivo regolarizzata sul batch $\mathcal{S}$:

$$F_{\mathcal{S}}(v) = J_{\mathcal{S}}(v) + \nu \|v\|_1 = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \ell(v; i) + \nu \sum_{j=1}^m |v_j|,$$

in accordo con la formulazione (6.2). `subgrad_batch` implementa il gradiente generalizzato $\tilde{\nabla} F_{\mathcal{S}}(v)$ componente per componente, seguendo la regola (6.3): se $v_i \neq 0$ si somma $\nu \text{sign}(v_i)$; se invece $v_i = 0$, si sceglie il subgradiente in $[-\nu, +\nu]$ che rende nullo (quando possibile) la componente del gradiente generalizzato, realizzando la proiezione $g_i = \arg \min_{g \in [-\nu, \nu]} |(\nabla J_{\mathcal{S}})_i + g|$. `project_orthant` implementa la proiezione $P(\cdot)$ sulla faccia ortante Ω_k definita in (6.5): azzerà ogni componente il cui segno non coincide con quello prescritto dal vettore z .

Newton-L1

```

#
for k in range(max_iter):
    indices_S = np.random.choice(N, size=n, replace=False)
    grads = np.array([grad_i(w, i) for i in indices_S])
    g_batch = np.mean(grads, axis=0)
    z = np.where(w > 0, 1,
                 np.where(w < 0, -1,
                          np.where(g_batch < -nu, 1,
                                   np.where(g_batch > nu, -1, 0))))

```

Ad ogni iterazione esterna k , si estrae un campione casuale $\mathcal{S}_k$ di $n_k = n$ indici distinti (senza reinserimento) da $\{0, \dots, N-1\}$. Su questo campione si calcola il gradiente della parte liscia

$$g_k = \nabla J_{\mathcal{S}_k}(w_k) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla \ell(w_k; i),$$

che costituisce la stima stocastica del gradiente esatto $\nabla J(w_k)$. Quindi si determina il vettore $z_k \in \{-1, 0, +1\}^m$ che caratterizza la faccia ortante secondo la (6.4): $z_k^i = \text{sign}(w_k^i)$ se $w_k^i \neq 0$; se invece $w_k^i = 0$, la componente viene posta a ± 1 quando il gradiente della parte liscia supera in valore assoluto la soglia ν (indicando che conviene “liberare” la variabile), e a 0 altrimenti (la variabile resta bloccata nell’active set).

Newton-L1

```
#
sg = subgrad_batch(w, indices_S)
sgn = np.linalg.norm(sg)
if sgn < 1e-10:
    history.append(w.copy().tolist())
    batch_sizes.append(n)
    break
n_h = max(1, int(round(R * n)))
n_h = min(n_h, N)
indices_H = np.random.choice(indices_S, size=n_h,
    replace=False)
free = (z != 0)
d = np.zeros_like(w)
if np.any(free):
    g_free = sg[free]
    # Hessian free: prodotto H.v senza costruire H
    def Hv(v_full):
        return np.mean([hessvec_i(w, i, v_full) for i in
            indices_H], axis=0)
    def Hv_free(v_free):
        v_full = np.zeros_like(w)
        v_full[free] = v_free
        Hv_full = Hv(v_full)
        return Hv_full[free]
    tol_cg = eta * np.linalg.norm(g_free)
    d_free = np.zeros(np.sum(free))
    r = -g_free.copy()
    p = r.copy()
    rr = np.dot(r, r)
    for _ in range(maxcg):
        Hp = Hv_free(p)
        pHp = np.dot(p, Hp)
        if pHp <= 1e-14:
            if np.linalg.norm(d_free) < 1e-14:
                d_free = -g_free.copy()
            break
```

```

alpha_cg = rr / pHp
d_free = d_free + alpha_cg * p
r_new = r - alpha_cg * Hp
rr_new = np.dot(r_new, r_new)
if np.sqrt(rr_new) <= tol_cg:
    r = r_new
    rr = rr_new
    break
beta = rr_new / rr
p = r_new + beta * p
r = r_new
rr = rr_new
d[free] = d_free

```

Si calcola il subgradiente $\tilde{\nabla} F_{\mathcal{S}_k}(w_k)$ tramite `subgrad_batch` e la sua norma euclidea; se questa è trascurabile ($< 10^{-10}$) si considera raggiunta la convergenza e si interrompe il ciclo, registrando l'ultimo punto. Altrimenti si determina la dimensione del campione per l'Hessiana come $n_h = \lceil R n_k \rceil$, vincolata tra 1 e N , e si estrae il sottocampione $\mathcal{H}_k \subseteq \mathcal{S}_k$ (senza reinserimento) di dimensione n_h . Il vettore booleano `free` individua le variabili libere (quelle con $z_k^i \neq 0$), mentre le rimanenti appartengono all'active set $\mathcal{A}_k$ definito in (6.6). Sul sottospazio delle variabili libere si risolve il sistema lineare di Newton ridotto

$$[Y_k^T \nabla^2 J_{\mathcal{H}_k}(w_k) Y_k] d^Y = -Y_k^T \tilde{\nabla} F_{\mathcal{S}_k}(w_k),$$

dove Y_k è la matrice dei versori delle coordinate libere, mediante il metodo del gradiente coniugato Hessian free. Gli operatori `Hv` e `Hv_free` calcolano il prodotto $\nabla^2 J_{\mathcal{H}_k}(w_k) v$ senza costruire esplicitamente la matrice Hessiana. Il CG viene arrestato quando il residuo soddisfa il criterio (6.11),

$$\|r_j\|_2 \leq \eta \|g_{\text{free}}\|_2.$$

o quando incontra curvatura non positiva ($\text{pHp} \leq 10^{-14}$), nel qual caso si effettua il fallback sulla direzione dell'antigradiente ridotto $-g_{\text{free}}$.

Newton-L1

```

#
step = alpha
F_w = F_batch(w, indices_S)
sg_d = np.dot(sg, d)
if sg_d >= 0:
    d = -sg
    sg_d = -np.dot(sg, sg)
w_new = w.copy()
for _ in range(20):
    w_trial = project_orthant(w + step * d, z)
    if F_batch(w_trial, indices_S) <= F_w + sigma * step *
        sg_d:
        w_new = w_trial
        break

```

```

step *= 0.5
if step < 1e-12:
    w_new = w.copy()
    break
w = w_new

```

Si inizializza la ricerca lineare proiettata con passo $\text{step} = \alpha$. Si calcola il valore dell'obiettivo $F_{\mathcal{S}_k}(w_k)$ e la derivata direzionale $\text{sg_d} = \tilde{\nabla} F_{\mathcal{S}_k}(w_k)^T d$. Se questa non è negativa, la direzione d non è di discesa e si forza il fallback $d = -\tilde{\nabla} F_{\mathcal{S}_k}(w_k)$, che è sempre di discesa (a meno di subgradiente nullo). La line search cerca il più grande passo α_k nella sequenza $1, \frac{1}{2}, \frac{1}{4}, \dots$ tale che

$$F_{\mathcal{S}_k}(P[w_k + \alpha_k d]) \leq F_{\mathcal{S}_k}(w_k) + \sigma \alpha_k \tilde{\nabla} F_{\mathcal{S}_k}(w_k)^T (P[w_k + \alpha_k d] - w_k),$$

dove P è la proiezione `project_orthant`. Questo corrisponde alla condizione (6.10). Se lo step diventa troppo piccolo ($< 10^{-12}$) si rifiuta l'aggiornamento e si mantiene il punto corrente.

Newton-L1

```

#
indices_new = np.random.choice(N, size=n, replace=False)
grads_new = np.array([grad_i(w, i) for i in indices_new])
g_new = np.mean(grads_new, axis=0)
if n > 1:
    var_vec = np.var(grads_new, axis=0, ddof=1)
else:
    var_vec = np.zeros_like(g_new)
V_norm1 = np.sum(var_vec)
gg_new = np.dot(g_new, g_new)
if gg_new > 1e-16:
    if V_norm1 / n > theta**2 * gg_new:
        n_new = int(np.ceil(V_norm1 / (theta**2 * gg_new))) + 1
        n = min(n_new, N)
history.append(w.copy().tolist())
batch_sizes.append(n)
if np.linalg.norm(grad_full(w)) < 1e-6:
    break
return history, batch_sizes

history, batch_sizes = newton_l1(w0, theta, max_iter, alpha, batch0,
    nu, sigma, maxcg, eta)

```

Dopo l'aggiornamento dei pesi, si verifica la Condizione di Controllo della Varianza (CCV) su un nuovo campione $\mathcal{S}_{k+1}$ della stessa dimensione n_k corrente per decidere se aumentare il batch per l'iterazione successiva. Si calcolano i gradienti individuali della parte liscia sul nuovo campione, la loro varianza campionaria $\hat{\mathcal{V}}$ e la norma L_1 $V_{\text{norm1}} = \|\hat{\mathcal{V}}\|_1$. La CCV richiede

$$\frac{\|\hat{\mathcal{V}}\|_1}{n} \leq \theta^2 \|g_{k+1}\|_2^2,$$

dove g_{k+1} è il gradiente della parte liscia sul nuovo campione. Se la condizione non è soddisfatta, la nuova dimensione del batch viene stimata come

$$n \leftarrow \min \left\{ N, \left\lceil \frac{\|\hat{\mathcal{V}}\|_1}{\theta^2 \|g_{k+1}\|_2^2} \right\rceil + 1 \right\}.$$

Questa è un'estensione dinamica rispetto al paper originale, dove il campione per il caso L_1 era mantenuto fisso. Si registrano il nuovo punto w_{k+1} e la dimensione del batch, quindi si controlla la norma del gradiente esatto (calcolato su tutto il dataset) tramite `grad_full`; se $\|\nabla J(w_{k+1})\|_2 < 10^{-6}$ si interrompe il ciclo. Al termine delle iterazioni, la funzione restituisce le liste `history` e `batch_sizes`. L'ultima riga mostra la chiamata tipica alla funzione, che produce le due liste per l'analisi successiva.

B.4 Metodo BB-CCV (Barzilai–Borwein con campionamento dinamico)

Il metodo BB-CCV sostituisce il passo fisso del Dynamic GD con un passo adattivo di Barzilai–Borwein, che stima la curvatura di J usando solo i gradienti già calcolati, mantenendo la Condizione di Controllo della Varianza per il campionamento dinamico del batch (Appendice E).

BB-CCV

```
def bb_dynamic_gd(w0, theta, max_iter, alpha, batch0):
    w = np.array(w0, dtype=float)
    n = max(batch0, 2)
    history      = [w.copy().tolist()]
    batch_sizes  = [n]

    w_prev = w.copy()
    g_prev = None
```

La funzione riceve gli stessi argomenti del Dynamic GD (Sezione B.1): il punto iniziale $w_0 \in \mathbb{R}^d$, la tolleranza θ della CCV, il numero massimo di iterazioni, il passo base α e la dimensione iniziale del batch $n_0 = \max(\text{batch0}, 2)$. Oltre alle liste `history` (pesi w_k) e `batch_sizes` (dimensioni n_k), si inizializza lo stato del passo di Barzilai–Borwein: w_{prev} e g_{prev} , cioè il punto e il gradiente dell'iterazione precedente, usati per costruire le differenze s_k e y_k . Alla prima iterazione $g_{\text{prev}} = \text{None}$ e il passo sarà semplicemente α .

BB-CCV

```
for k in range(max_iter):
    indices = np.random.choice(N, size=n, replace=False)
    grads = np.array([grad_i(w, i) for i in indices])
    g = np.mean(grads, axis=0)
```

Come nel Dynamic GD (Sezione B.1) si estrae il batch $\mathcal{S}_k = \{i_1, \dots, i_n\}$ senza reinserimento e si calcola il gradiente campionato $g_k = \nabla J_{\mathcal{S}_k}(w_k) = \frac{1}{n_k} \sum_{i \in \mathcal{S}_k} \nabla \ell(w_k; i)$.

BB-CCV

```

if k > 0 and g_prev is not None:
    s = w - w_prev
    y = g - g_prev
    sy = np.dot(s, y)
    if abs(sy) > 1e-14:
        step_bb = np.dot(s, s) / sy
        step = np.clip(step_bb, alpha / 20.0, alpha * 5.0)
    else:
        step = alpha
else:
    step = alpha

w_prev = w.copy()
g_prev = g.copy()

```

Il passo di Barzilai–Borwein stima la curvatura di J lungo la direzione percorsa usando le differenze tra iterazioni consecutive:

$$s_k = w_k - w_{k-1}, \quad y_k = g_k - g_{k-1}, \quad \alpha_k^{\text{BB}} = \frac{s_k^\top s_k}{s_k^\top y_k}.$$

Per una quadratica vale $y_k = \nabla^2 J s_k$; quindi α_k^{BB} coincide con l'inverso di una media di Rayleigh della Hessiana lungo s_k , cioè un passo di Newton scalare a costo nullo, poiché s_k e y_k sono quantità già disponibili. Per robustezza sui problemi non quadratici il passo è salvaguardato nell'intervallo $[\alpha/20, 5\alpha]$ tramite `np.clip`; se $s_k^\top y_k$ è numericamente nullo si torna al passo base α . Alla prima iterazione ($k = 0$) la coppia (s, y) non esiste ancora e si usa $\alpha_0 = \alpha$. Infine si aggiorna lo stato precedente $w_{\text{prev}} \leftarrow w_k$, $g_{\text{prev}} \leftarrow g_k$.

BB-CCV

```

def J_batch(w_curr):
    return np.mean([loss_i(w_curr, i) for i in indices])

c1 = 1e-4
J_curr = J_batch(w)
g_norm2 = np.dot(g, g)

if g_norm2 > 1e-16:
    for _ in range(30):
        w_new = w - step * g
        if J_batch(w_new) <= J_curr - c1 * step * g_norm2:
            break
        step *= 0.5
    else:
        step = 0.0

w = w - step * g

```

Il candidato è $w_k - \alpha_k g_k$: il passo di Barzilai–Borwein viene rifinito da una line search di Armijo con backtracking sulla loss del batch, come nel Dynamic GD. La

condizione di sufficiente decremento, con $c_1 = 10^{-4}$, è

$$J_{S_k}(w_k - \alpha_k g_k) \leq J_{S_k}(w_k) - c_1 \alpha_k \|g_k\|_2^2,$$

che è la condizione di Armijo della Sezione 5.1.5 con direzione $d_k = -g_k$. Se dopo 30 dimezzamenti la condizione non è soddisfatta si pone $\alpha_k = 0$. L'aggiornamento è quindi $w_{k+1} = w_k - \alpha_k g_k$.

BB-CCV

```
if n > 1:
    var_vec = np.var(grads, axis=0, ddof=1)
else:
    var_vec = np.zeros_like(g)
V_norm1 = np.sum(var_vec)

gg = np.dot(g, g)
if gg > 1e-16:
    if V_norm1 / n > theta**2 * gg:
        n_new = int(np.ceil(V_norm1 / (theta**2 * gg))) + 1
        n = min(n_new, N)
```

La gestione del batch è identica al Dynamic GD (Sezione B.1): si calcola la varianza campionaria dei gradienti individuali sul batch $\hat{\mathcal{V}} = \text{Var}_{i \in S_k}(\nabla \ell(w_k; i))$, la sua norma $V_{\text{norm1}} = \|\hat{\mathcal{V}}\|_1$ e si verifica la CCV $\|\hat{\mathcal{V}}\|_1/n_k \leq \theta^2 \|g_k\|_2^2$. Se violata, il batch viene aumentato a $n \leftarrow \min\{N, \lceil \|\hat{\mathcal{V}}\|_1/(\theta^2 \|g_k\|_2^2) \rceil + 1\}$.

BB-CCV

```
history.append(w.copy().tolist())
batch_sizes.append(n)

if np.linalg.norm(grad_full(w)) < 1e-6:
    break

return history, batch_sizes

history, batch_sizes = bb_dynamic_gd(w0, theta, max_iter, alpha,
    batch0)
```

Si registrano il nuovo punto w_{k+1} e la dimensione del batch e si verifica la convergenza sulla norma del gradiente esatto $\|\nabla J(w_{k+1})\|_2 < 10^{-6}$. Al termine la funzione restituisce le liste `history` e `batch_sizes`; l'ultima riga mostra la chiamata tipica.

C Istruzioni per l'esecuzione

In questa appendice sono raccolte le istruzioni per eseguire il codice Python presentato nell'Appendice B e per avviare l'applicazione web interattiva descritta nella Sezione 6.2.

C.1 Dipendenze

Il codice richiede Python 3.9 o successivo e le seguenti librerie:

- `numpy` (versione ≥ 1.21) – calcolo numerico vettoriale;
- `matplotlib` (versione ≥ 3.5) – generazione dei grafici di convergenza e dei percorsi di ottimizzazione;
- `plotly` (versione ≥ 5.0) – visualizzazioni interattive nell'applicazione web;
- `pyodide` – runtime Python nel browser per l'esecuzione del codice degli algoritmi (usato dall'applicazione web).

C.2 Installazione

Si consiglia di creare un ambiente virtuale dedicato e installare le dipendenze con `pip`:

```
python3 -m venv .venv
source .venv/bin/activate          # su Windows: .venv\Scripts\activate
pip install --upgrade pip
pip install numpy matplotlib plotly
```

C.3 Esecuzione degli esperimenti numerici

Gli esperimenti del Capitolo 6 sono condotti con l'applicazione web interattiva descritta nella Sezione 6.2: i valori riportati nel capitolo (tabelle e figure) sono quelli mostrati dal pannello *Analisi* dell'applicazione, con i preset, i parametri e il seme descritti nella Sezione 6.1. Aprendo `visualizzazione.html` in un browser e selezionando il preset, l'algoritmo e i parametri indicati, i risultati sono riproducibili a parità di seed.

Per eseguire i metodi anche senza browser, lo script `sim_exp.py` implementa in forma standalone il gradiente a campione dinamico e Newton-CG su problemi test sintetici (tra cui la regressione lineare), generando figure in formato PDF/PNG e tabelle dei risultati in formato LaTeX:

```
python3 sim_exp.py
```

Lo script stampa a video, per ogni problema test, il numero di iterazioni, il tempo di esecuzione, l'errore finale $\|w_k - w^*\|$ e la dimensione massima del batch raggiunta.

C.4 Esecuzione dei singoli algoritmi

I codici dell'Appendice B possono essere eseguiti anche in modo isolato, previa definizione delle funzioni ausiliarie `loss_i`, `grad_i`, `hessvec_i`, `grad_full` e della costante N , che dipendono dal dataset. Un esempio completo per la regressione lineare è incluso nello script `sim_exp.py`.

C.5 Avvio dell'applicazione web

L'applicazione web interattiva è un'applicazione a pagina singola (HTML + JavaScript) che esegue il codice Python nel browser tramite Pyodide: non è necessario un server dedicato. È sufficiente aprire il file HTML in un browser moderno (Chrome, Firefox, Safari, Edge). Per lo sviluppo locale si può utilizzare un semplice server HTTP statico:

```
python3 -m http.server 8000
```

e quindi visitare `http://localhost:8000`.

C.6 Requisiti di sistema

Gli esperimenti riportati in questo elaborato sono stati eseguiti su un computer portatile con processore Apple Silicon (8 core) e 16 GB di RAM, in ambiente macOS, utilizzando Python 3.14, NumPy 2.4 e Matplotlib 3.11. Il costo complessivo della simulazione (tre problemi test, tre valori di θ e quattro algoritmi) è inferiore a due minuti; nessun acceleratore GPU è richiesto, poiché tutti i calcoli sono operazioni vettoriali su array NumPy di dimensione al più $N \times m$. Per la riproducibilità esatta dei risultati, il generatore di numeri casuali è stato inizializzato con seme fisso (seed 42); a parità di codice e versioni delle librerie, i valori riportati sono riproducibili.

D Schemi dei metodi a varianza ridotta

I metodi a varianza ridotta SVRG e SAGA, discussi nella Sezione 4, combinano la stima stocastica del gradiente con una correzione basata su quantità globali: lo stimatore g_k resta non distorto, $\mathbb{E}_{i_k}[g_k] = \nabla J(w_k)$, e la sua varianza è ridotta rispetto al gradiente stocastico puro $\nabla \ell(w_k; i_k)$. Nella notazione del documento, $J(w) = \frac{1}{N} \sum_{i=1}^N \ell(w; i)$ è la perdita media empirica, $\nabla \ell(w; i)$ il gradiente sull'esempio i -esimo, i_k l'indice estratto uniformemente a caso da $\{1, \dots, N\}$ all'iterazione k e α_k il passo.

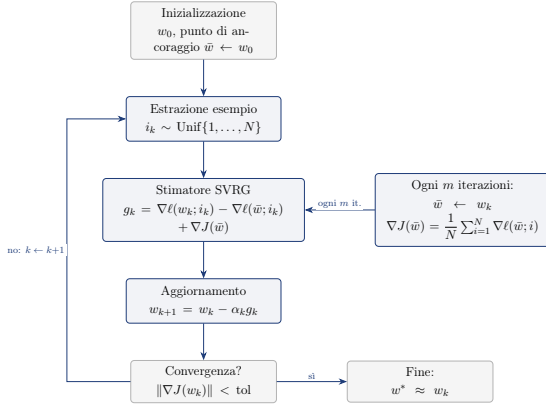


Figura D.1: Schema del metodo SVRG di Johnson e Zhang [9]: il punto di ancoraggio $\bar{w}$ viene aggiornato ogni m iterazioni con il gradiente esatto $\nabla J(\bar{w})$, e lo stimatore corregge il gradiente stocastico corrente con la differenza $\nabla \ell(w_k; i_k) - \nabla \ell(\bar{w}; i_k)$. Lo stimatore è non distorto, $\mathbb{E}_{i_k}[g_k] = \nabla J(w_k)$, e quando w_k è vicino a $\bar{w}$ i due termini stocastici si elidono quasi del tutto, riducendo la varianza.

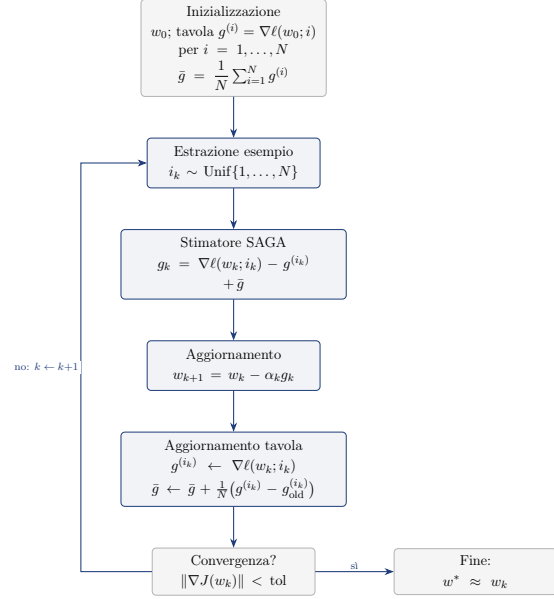


Figura D.2: Schema del metodo SAGA di Defazio, Bach e Lacoste-Julien [10]: la tavola dei gradienti $g^{(i)}$ memorizza l'ultimo gradiente calcolato per ogni esempio ed è aggiornata a ogni iterazione; lo stimatore usa la media $\bar{g}$ della tavola come correzione. Anche in questo caso $\mathbb{E}_{i_k}[g_k] = \nabla J(w_k)$ e la varianza è ridotta rispetto al gradiente stocastico puro.

E Schema dell'algoritmo BB-CCV

Questa appendice illustra il passo di Barzilai–Borwein su cui si basa il metodo BB-CCV e riporta lo schema a blocchi del metodo aggiuntivo usato per i test comparativi della Sezione 6.3: BB-CCV (gradiente a campione dinamico con passo di Barzilai–Borwein).

E.1 Il metodo di Barzilai–Borwein

Il metodo BB-CCV sostituisce il passo fisso del Dynamic GD con un passo adattivo di Barzilai–Borwein, che stima la curvatura di J lungo la direzione percorsa usando soltanto le quantità già disponibili, senza richiedere la Hessiana.

Consideriamo due iterazioni consecutive, w_k e $w_{k+1} = w_k + \Delta w_k$. Lo sviluppo al primo ordine del gradiente $g(w) = \nabla J(w)$ attorno a w_k è

$$g(w_{k+1}) \approx g(w_k) + \nabla^2 J(w_k) \Delta w_k. \quad (\text{E.1})$$

Definiamo lo spostamento e la variazione del gradiente:

$$s_k \triangleq \Delta w_k = w_{k+1} - w_k, \quad y_k \triangleq g(w_{k+1}) - g(w_k). \quad (\text{E.2})$$

Dalla (E.1) si ottiene la relazione fondamentale

$$y_k \approx \nabla^2 J(w_k) s_k. \quad (\text{E.3})$$

I metodi quasi Newton approssimano la Hessiana con B_{k+1} imponendo l'equazione della secante $B_{k+1}s_k = y_k$. Barzilai–Borwein estremizza l'approssimazione: pone $B_{k+1} = \alpha_{k+1}^{-1}I$, riducendo la secante a

$$\alpha^{-1}s \approx y,$$

un sistema di n equazioni scalari nell'incognita α : in generale nessun singolo scalare può soddisfarle esattamente, quindi il problema si risolve ai minimi quadrati. Dalla relazione $\alpha^{-1}s \approx y$ si ottengono due formule classiche. La prima, minimizzando $\|\alpha y - s\|_2^2$, dà

$$\alpha_k^{(1)} = \frac{s_k^\top y_k}{y_k^\top y_k}; \quad (\text{E.4})$$

la seconda, minimizzando $\|\alpha^{-1}s - y\|_2^2$, dà

$$\alpha_k^{(2)} = \frac{s_k^\top s_k}{s_k^\top y_k}. \quad (\text{E.5})$$

La seconda formula è più aggressiva della prima: produce passi più grandi quando il gradiente cambia poco lungo la direzione percorsa (cioè quando la funzione è piatta in quella direzione), e passi più piccoli quando il gradiente cambia molto. Tuttavia, se lo spostamento s e la variazione del gradiente y sono quasi perpendicolari (cioè $s^\top y \approx 0$), il denominatore si annulla e il passo diventa enorme, facendo esplodere l'iterazione. Per questo BB-CCV introduce la salvaguardia $\text{clip}(\cdot, \alpha/20, 5\alpha)$, che mantiene il passo in un intervallo ragionevole anche in presenza di curvature anomale.

Sulle funzioni quadratiche, dove $y = As$ è esatta e A è definita positiva, il passo di Barzilai–Borwein è ottimale e il metodo converge in poche iterazioni. Sulle funzioni generali, invece, le proprietà precedenti vengono meno: la relazione secante è solo locale, l'Hessiana varia nel tempo, il denominatore $s^\top y$ può diventare negativo o nullo (curvatura negativa o Hessiana singolare), e la presenza di derivate terze rende il passo non più ottimale. BB-CCV gestisce queste difficoltà con la salvaguardia e la line search di Armijo.

Caratteristica	Funzione quadratica	Funzione generale
Hessiana	costante, $\nabla^2 J = A$	varia nel tempo, $\nabla^2 J(w)$
Relazione fondamentale	$y = As$, esatta e globale	approssimata: media della Hessiana lungo il percorso
Passo calcolato	inverso del quoziente di Rayleigh (ottimale)	media degli ultimi passi, obsoleta al passo successivo
Denominatore $s^\top y$	sempre positivo, stabile	può essere ≤ 0 o ≈ 0 : serve la salvaguardia
Convergenza	superlineare (in due dimensioni bastano 2–3 passi)	lineare lenta, o divergente senza salvaguardia

Tabella E.1: Il passo di Barzilai–Borwein sulle funzioni quadratiche e generali.

Nella pratica, sui problemi della Sezione 6.3 la salvaguardia di BB-CCV è sufficiente: il metodo risulta il più veloce tra quelli testati e raggiunge la precisione macchina sui problemi mal condizionati.

E.2 Schema dell'algoritmo BB-CCV

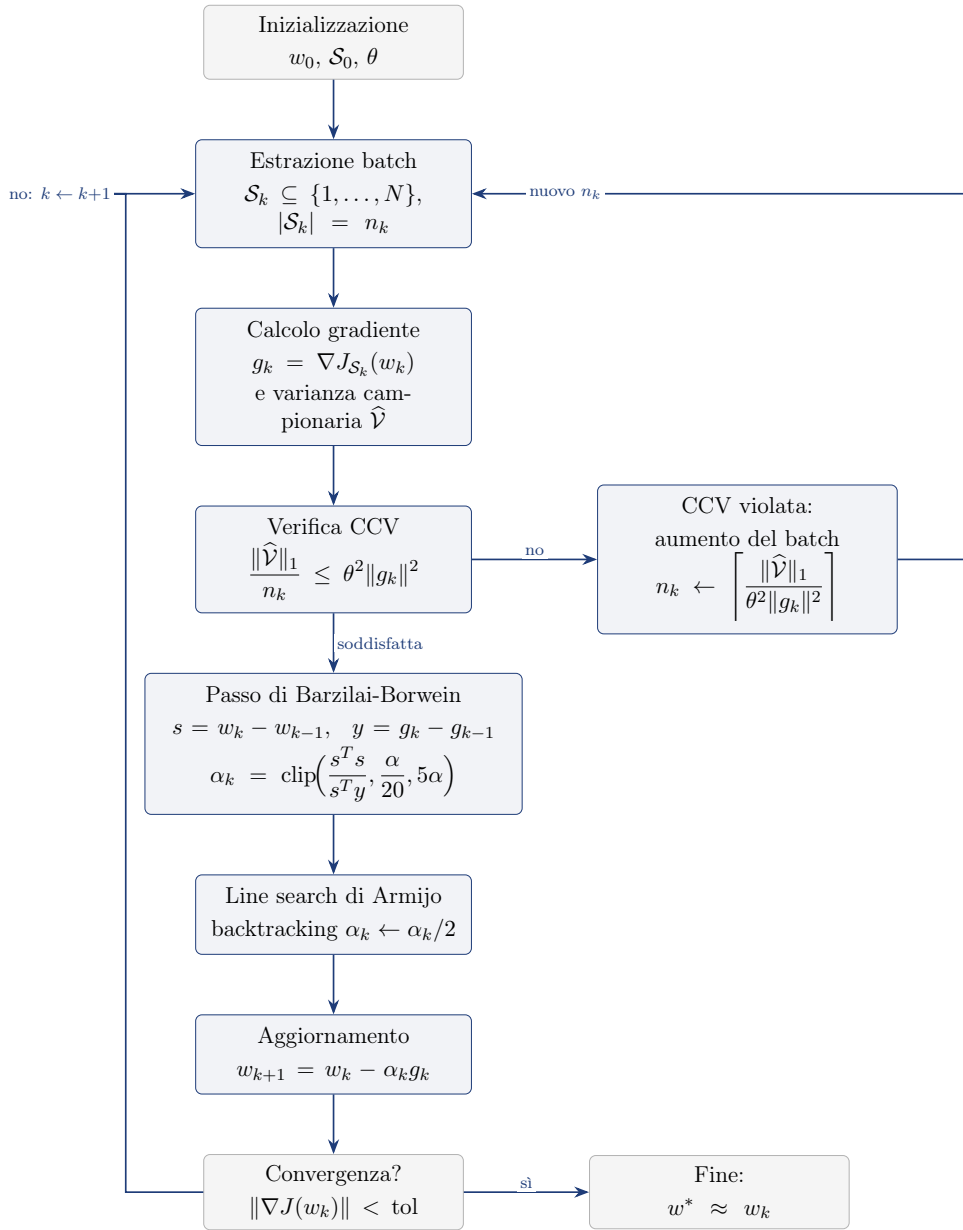


Figura E.1: Schema dell'algoritmo *BB-CCV* (gradiente a campione dinamico con passo di Barzilai–Borwein). Il flusso è quello del Dynamic GD (Figura 5.1): estrazione del batch $\mathcal{S}_k$, calcolo di gradiente e varianza campionaria $\hat{\mathcal{V}}$, verifica della CCV con eventuale aumento di n_k . La differenza è nel passo: per $k > 0$ si stima la curvatura di J usando solo i gradienti già calcolati, $s = w_k - w_{k-1}$ e $y = g_k - g_{k-1}$, con la formula $\alpha_k^{BB} = \frac{s^T s}{s^T y}$ di Barzilai–Borwein (per $k = 0$, $\alpha_0 = \alpha$); il passo è salvaguardato in $[\alpha/20, 5\alpha]$ per la stabilità sui problemi non quadratici e rifinito da una line search di Armijo con backtracking. BB usa solo quantità già disponibili, quindi non richiede la Hessiana: nei tre esperimenti della Sezione 6 il metodo impiega in media ≈ 13 iterazioni (12, 11 e 16 sui tre problemi) contro le 30 del GD, e sui problemi mal condizionati raggiunge precisioni 10^{-8} – 10^{-14} invece di 10^{-1} – 10^{-2} .

Riferimenti bibliografici

- [1] R. H. Byrd, G. M. Chin, J. Nocedal, and Y. Wu, “Sample size selection in optimization methods for machine learning,” *Mathematical Programming, Series A*, vol. 134, no. 1, pp. 127–155, 2012.
- [2] L. Bottou and O. Bousquet, “The tradeoffs of large scale learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 20, 2008, pp. 161–168.
- [3] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer Series in Operations Research and Financial Engineering, New York: Springer, 2006.
- [4] R. H. Byrd, G. M. Chin, W. Neveitt, and J. Nocedal, “On the use of stochastic Hessian information in optimization methods for machine learning,” *SIAM Journal on Optimization*, vol. 21, no. 3, pp. 894–1009, 2011.
- [5] L. Bottou, F. E. Curtis, and J. Nocedal, “Optimization methods for large-scale machine learning,” *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018.
- [6] H. Robbins and S. Monro, “A stochastic approximation method,” *The Annals of Mathematical Statistics*, vol. 22, no. 3, pp. 400–407, 1951.
- [7] B. T. Polyak, “Gradient methods for the minimization of functionals,” *USSR Computational Mathematics and Mathematical Physics*, vol. 3, no. 4, pp. 864–878, 1963 (traduzione inglese, 1964).
- [8] H. Karimi, J. Nutini, and M. Schmidt, “Linear convergence of gradient and proximal-gradient methods under the Polyak–Łojasiewicz condition,” in *European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases (ECML PKDD)*, 2016, pp. 795–811.
- [9] R. Johnson and T. Zhang, “Accelerating stochastic gradient descent using predictive variance reduction,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 26, 2013, pp. 315–323.
- [10] A. Defazio, F. Bach, and S. Lacoste-Julien, “SAGA: A fast incremental gradient method with support for non-strongly convex composite objectives,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27, 2014, pp. 1646–1654.
- [11] R. S. Dembo and T. Steihaug, “Truncated-Newton algorithms for large-scale unconstrained optimization,” *Mathematical Programming*, vol. 26, no. 2, pp. 190–212, 1983.
- [12] J. R. Shewchuk, “An introduction to the conjugate gradient method without the agonizing pain,” Technical Report CMU-CS-94-125, Carnegie Mellon University, 1994.
- [13] J. Engel, C. Resnick, A. Roberts, S. Dieleman, M. Norouzi, D. Eck, and K. Simonyan, “Neural audio synthesis of musical notes with WaveNet autoencoders,” in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.